CYBERSÉCURITÉ OPÉRATIONNELLE

1. Déploiement d’une solution de supervision de sécurité avec Wazuh

Dans le cadre du projet de cybersécurité opérationnelle, une plateforme Wazuh est mise en place afin de centraliser la supervision de sécurité des machines du laboratoire.

Wazuh permet notamment de collecter et d’analyser les événements provenant des postes supervisés, de détecter certaines anomalies de sécurité, de contrôler la configuration des systèmes et de centraliser les alertes dans une interface d’administration.

Le déploiement retenu repose sur une architecture **single-node conteneurisée avec Docker Compose**. Elle regroupe les principaux composants de Wazuh :

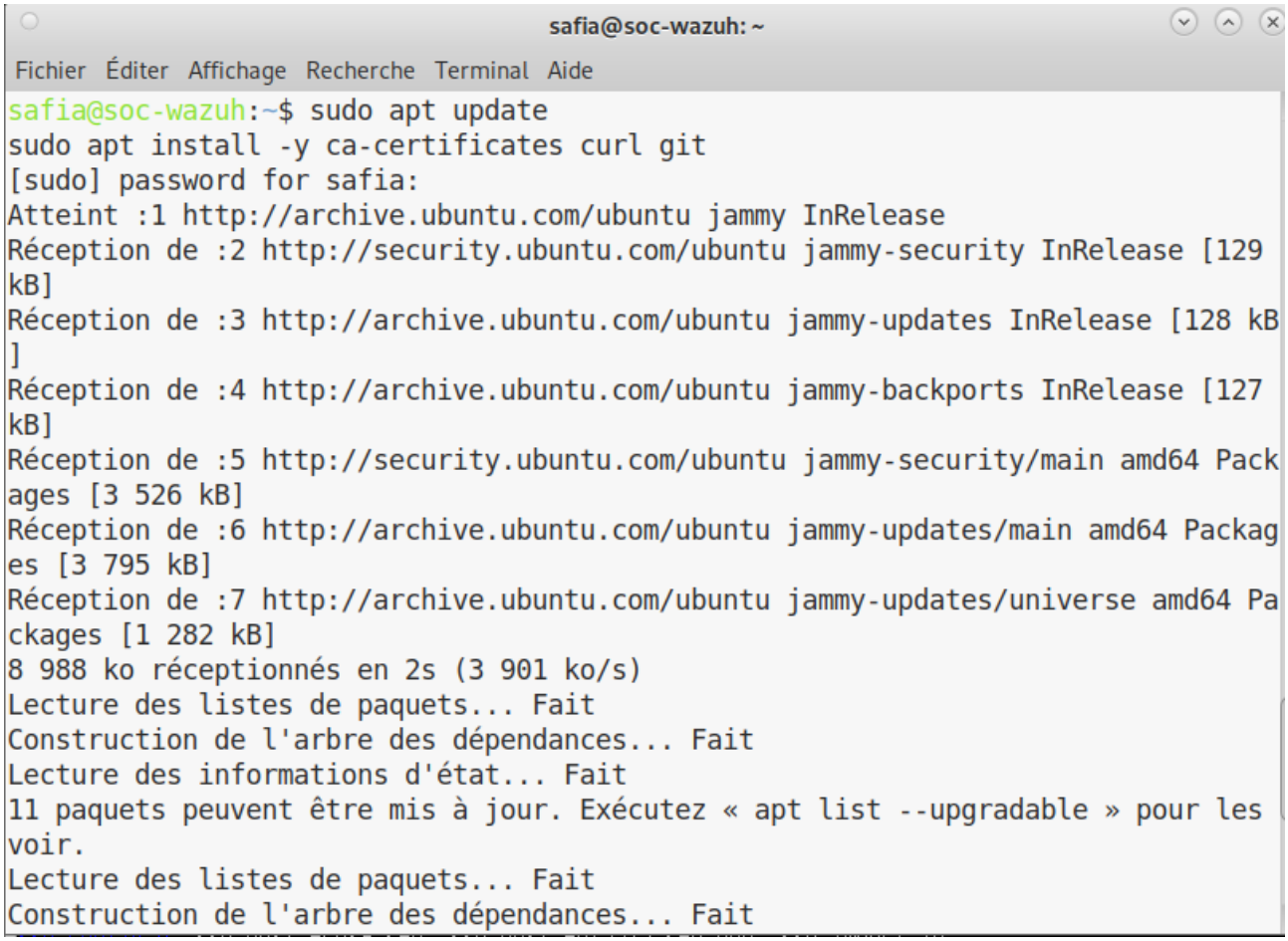
- **Wazuh Manager** : collecte et analyse les événements provenant des agents ;
- **Wazuh Indexer** : stocke et indexe les données de sécurité ;
- **Wazuh Dashboard** : fournit l’interface Web de consultation et d’analyse.

Un agent Wazuh est ensuite installé sur une machine cliente afin de vérifier la communication entre l’équipement supervisé et le serveur Wazuh.

Installation de Docker et Docker compose

1.1. Préparer les outils nécessaires :

sudo apt update
sudo apt install -y ca-certificates curl git



1.2. Ajouter la clé du dépôt officiel Docker :

sudo install -m 0755 -d /etc/apt/keyrings

sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc

sudo chmod a+r /etc/apt/keyrings/docker.asc

1.3. Déclarer le dépôt pour Ubuntu 22.04 AMD64 :

sudo tee /etc/apt/sources.list.d/docker.sources > /dev/null <<'EOF'

Types: deb

URIs: https://download.docker.com/linux/ubuntu

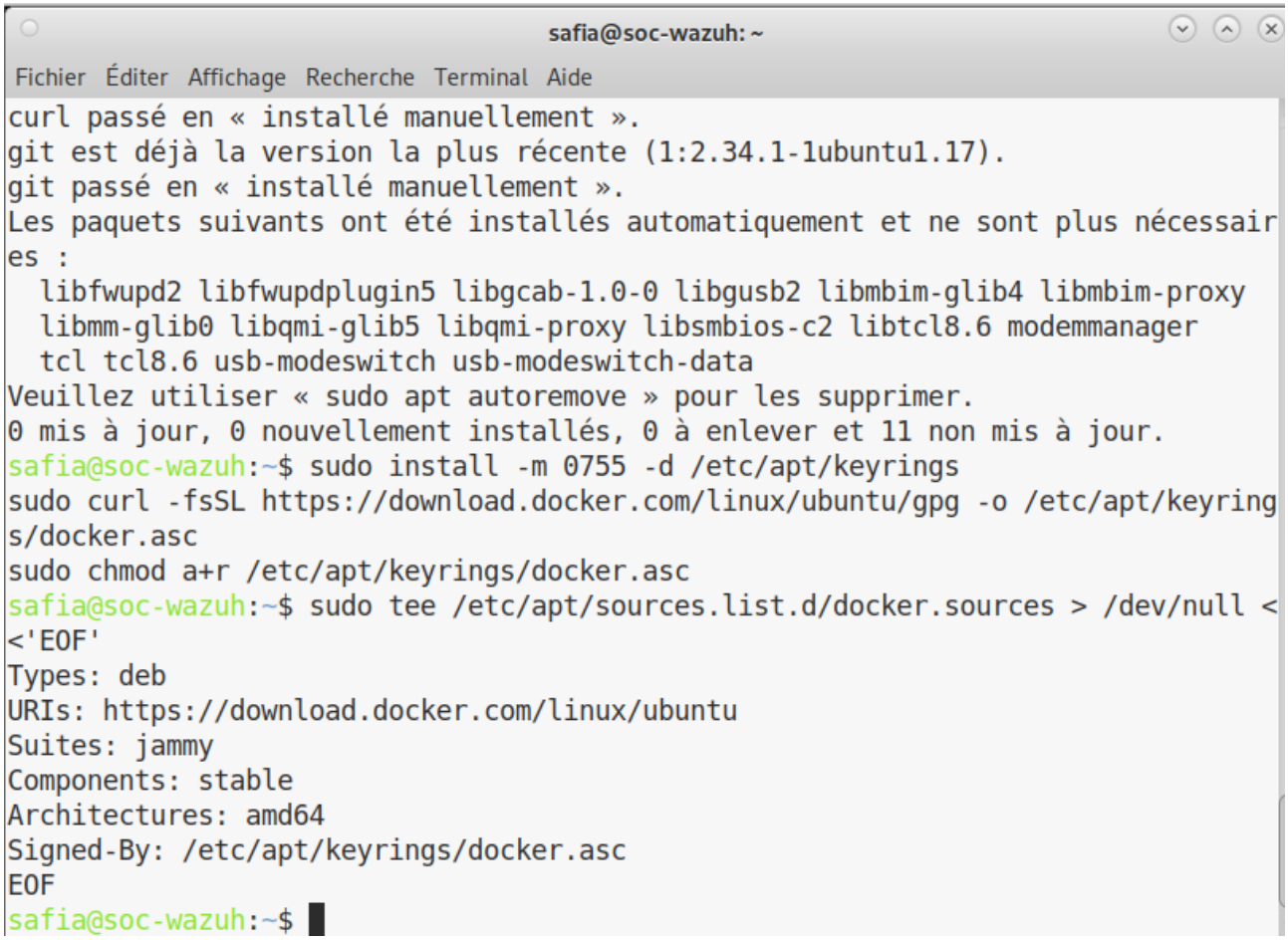
Suites: jammy

Components: stable

Architectures: amd64

Signed-By: /etc/apt/keyrings/docker.asc

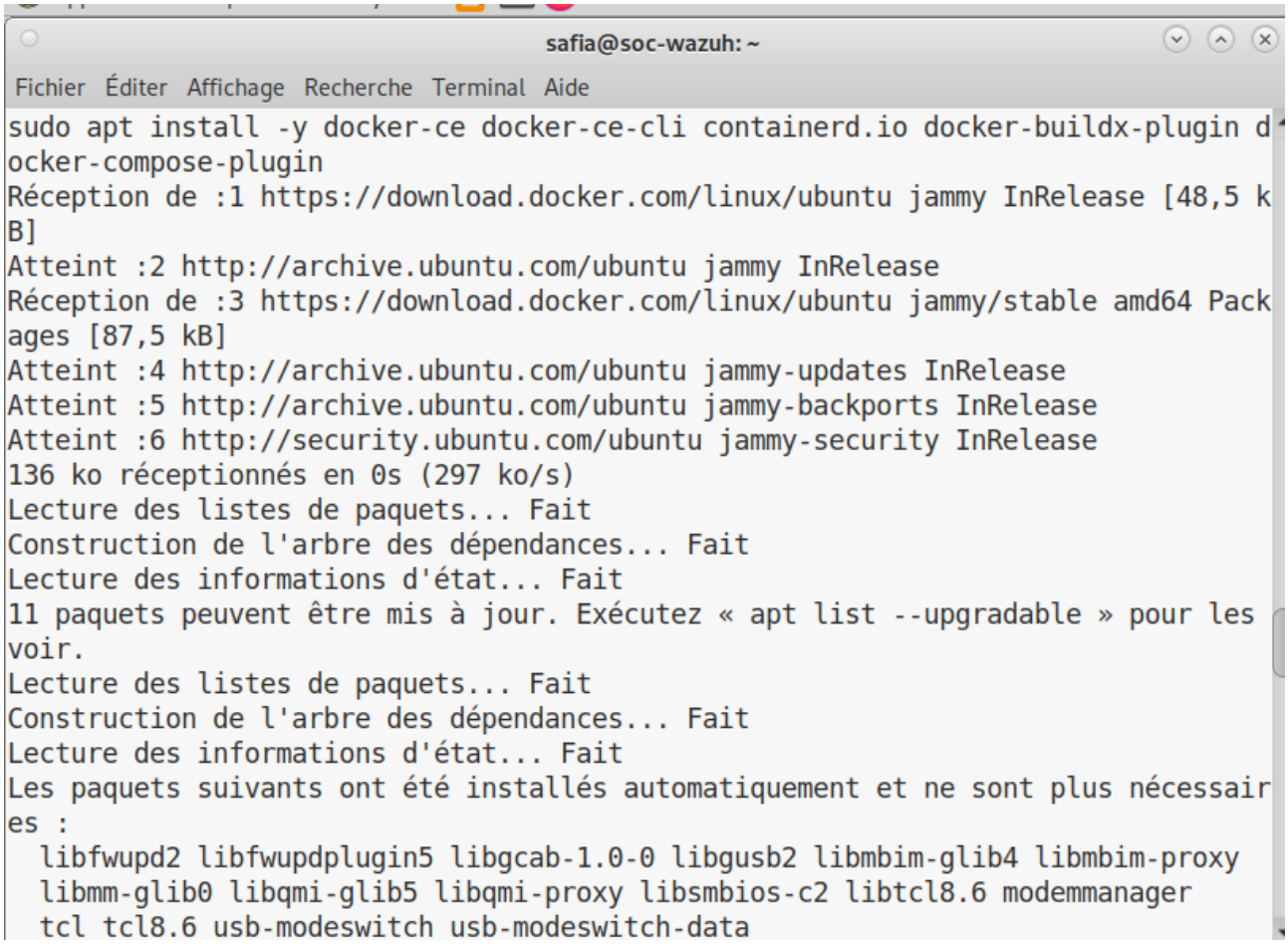
EOF

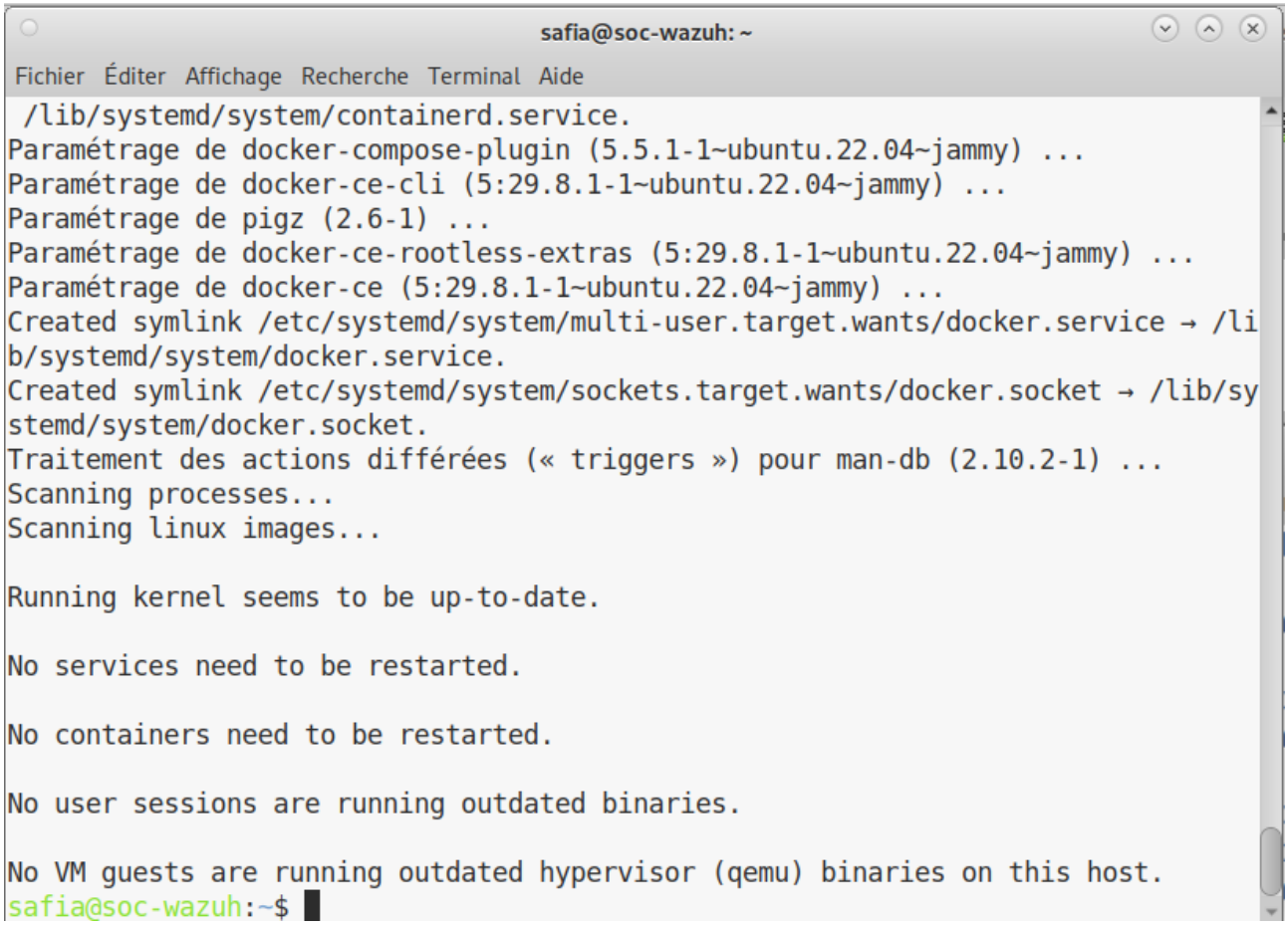


1.4. Installation de Docker et Docker Compose :

Le déploiement officiel de Wazuh utilisé dans ce laboratoire repose sur des conteneurs Docker. Docker et le plugin Docker Compose sont donc installés sur le serveur soc-wazuh.

sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin


safia@soc-wazuh: ~
Fichier Éditer Affichage Recherche Terminal Aide
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin d
ocker-compose-plugin
Réception de :1 https://download.docker.com/linux/ubuntu jammy InRelease [48,5 k
B]
Atteint :2 http://archive.ubuntu.com/ubuntu jammy InRelease
Réception de :3 https://download.docker.com/linux/ubuntu jammy/stable amd64 Pack
ages [87,5 kB]
Atteint :4 http://archive.ubuntu.com/ubuntu jammy-updates InRelease
Atteint :5 http://archive.ubuntu.com/ubuntu jammy-backports InRelease
Atteint :6 http://security.ubuntu.com/ubuntu jammy-security InRelease
136 ko réceptionnés en 0s (297 ko/s)
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
11 paquets peuvent être mis à jour. Exécutez « apt list --upgradable » pour les
voir.
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
Les paquets suivants ont été installés automatiquement et ne sont plus nécessair
es :
libfwupd2 libfwupdplugin5 libgcab-1.0-0 libgusb2 libmbim-glib4 libmbim-proxy
libmm-glib0 libqmi-glib5 libqmi-proxy libsmbios-c2 libtcl8.6 modemmanager
tcl tcl8.6 usb-modeswitch usb-modeswitch-data



L'installation s'est terminée correctement. Le service Docker est maintenant disponible et pourra être utilisé pour déployer les différents composants de Wazuh.

2. Vérification des ressources et préparation du système

2.1. Configurer le paramètre système nécessaire à l’indexeur.

Avant le déploiement, les ressources mémoire disponibles sur le serveur sont vérifiées avec la commande free -h. La machine dispose d'environ 7,7 Gio de mémoire vive, ce qui permet de poursuivre l'installation du laboratoire Wazuh.

echo 'vm.max_map_count=262144' | sudo tee /etc/sysctl.d/99-wazuh.conf
sudo sysctl -p /etc/sysctl.d/99-wazuh.conf

```
safia@soc-wazuh:~$ free -h
               total        used        free      shared  buff/cache   available
Mem:           7,7Gi       373Mi       6,4Gi       1,0Mi       947Mi       7,1Gi
Swap:          3,8Gi          0B       3,8Gi
```

Le paramètre vm.max_map_count est ensuite configuré à 262144. Ce paramètre détermine le nombre maximal de zones mémoire pouvant être mappées par un processus et est nécessaire au bon fonctionnement de l'indexeur utilisé par Wazuh.

Le réglage est enregistré dans /etc/sysctl.d/99-wazuh.conf afin d'être conservé après un redémarrage du serveur.

2.2. Récupérer les fichiers officiels de déploiement :

cd ~
git clone --depth 1 --branch v4.14.7 https://github.com/wazuh/wazuh-docker.git
cd ~/wazuh-docker/single-node

```
safia@soc-wazuh:~$ cd ~
git clone --depth 1 --branch v4.14.7 https://github.com/wazuh/wazuh-docker.git
cd ~/wazuh-docker/single-node
Cloning into 'wazuh-docker'...
remote: Enumerating objects: 152, done.
remote: Counting objects: 100% (152/152), done.
remote: Compressing objects: 100% (121/121), done.
remote: Total 152 (delta 23), reused 83 (delta 19), pack-reused 0 (from 0)
Receiving objects: 100% (152/152), 93.69 KiB | 3.35 MiB/s, done.
Resolving deltas: 100% (23/23), done.
Note: switching to 'adcc5b57d2f7edfcbe6c399272dc76fbdf12b623'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

  git switch -c <new-branch-name>

Or undo this operation with:

  git switch -

Turn off this advice by setting config variable advice.detachedHead to false
```

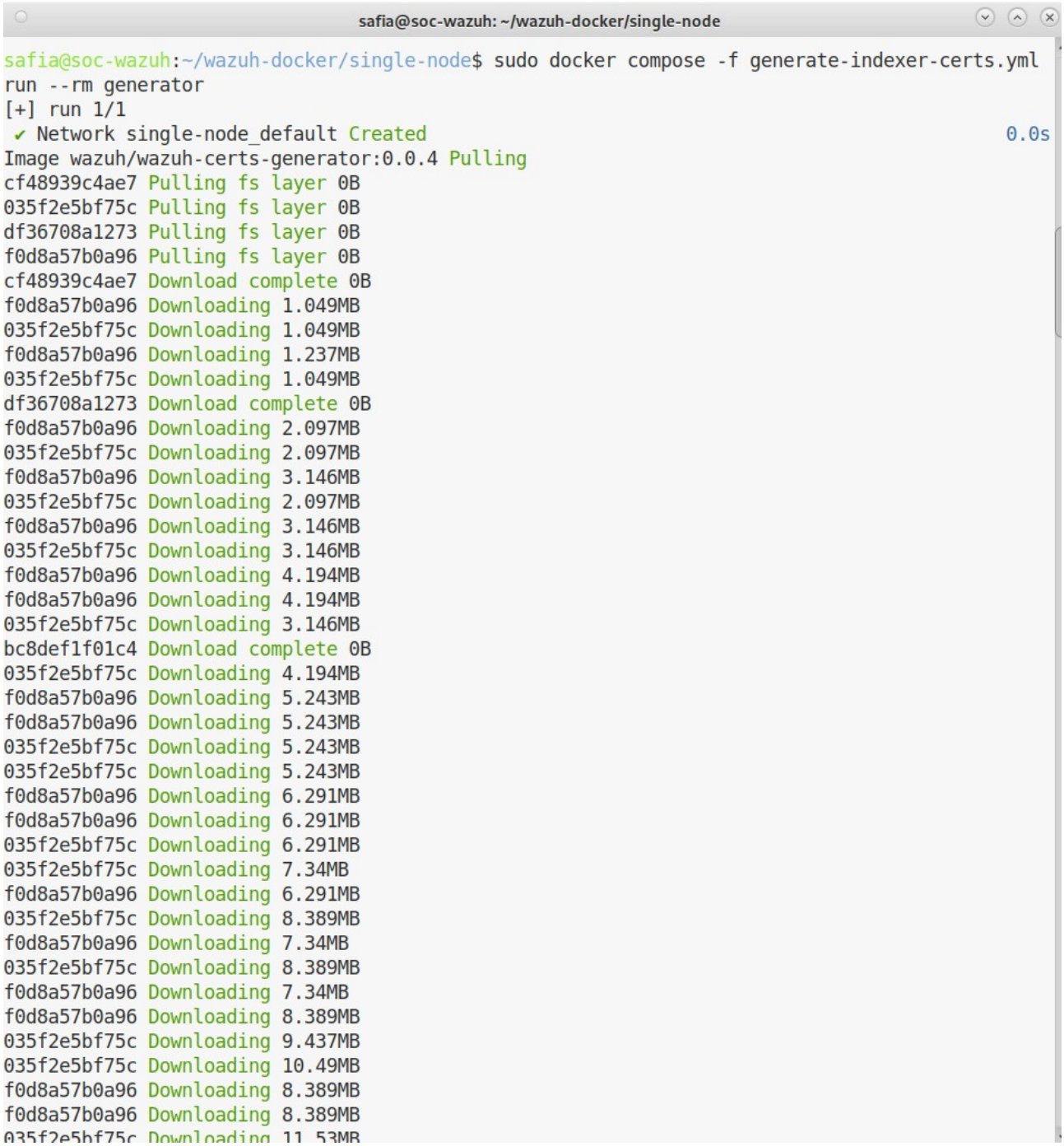
Le dépôt officiel wazuh-docker est récupéré depuis GitHub. La version 4.14.7 est utilisée pour ce laboratoire.

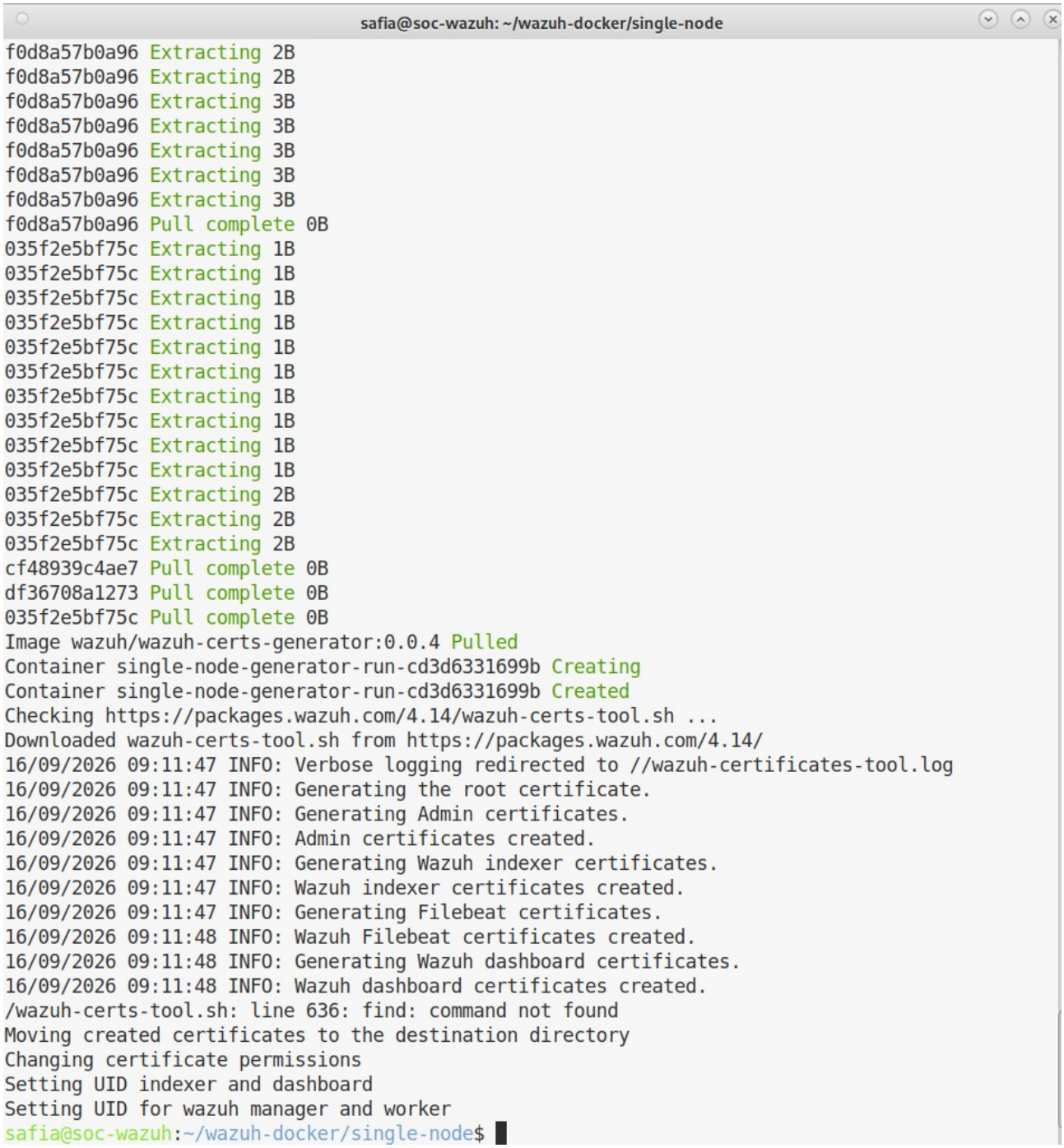
Le répertoire single-node est sélectionné car l'ensemble des composants Wazuh sera hébergé sur un seul serveur.

3. Génération des certificats de sécurité

Wazuh utilise des certificats pour sécuriser les communications entre ses différents composants. Avant de démarrer la plateforme, les certificats nécessaires à l'indexeur et au Dashboard sont donc générés.

sudo docker compose -f generate-indexer-certs.yml run --rm generator


safia@soc-wazuh: ~/wazuh-docker/single-node
safia@soc-wazuh:~/wazuh-docker/single-node$ sudo docker compose -f generate-indexer-certs.yml run --rm generator
[+] run 1/1
✔ Network single-node_default Created 0.0s
Image wazuh/wazuh-certs-generator:0.0.4 Pulling
cf48939c4ae7 Pulling fs layer 0B
035f2e5bf75c Pulling fs layer 0B
df36708a1273 Pulling fs layer 0B
f0d8a57b0a96 Pulling fs layer 0B
cf48939c4ae7 Download complete 0B
f0d8a57b0a96 Downloading 1.049MB
035f2e5bf75c Downloading 1.049MB
f0d8a57b0a96 Downloading 1.237MB
035f2e5bf75c Downloading 1.049MB
df36708a1273 Download complete 0B
f0d8a57b0a96 Downloading 2.097MB
035f2e5bf75c Downloading 2.097MB
f0d8a57b0a96 Downloading 3.146MB
035f2e5bf75c Downloading 2.097MB
f0d8a57b0a96 Downloading 3.146MB
035f2e5bf75c Downloading 3.146MB
f0d8a57b0a96 Downloading 4.194MB
f0d8a57b0a96 Downloading 4.194MB
035f2e5bf75c Downloading 3.146MB
bc8def1f01c4 Download complete 0B
035f2e5bf75c Downloading 4.194MB
f0d8a57b0a96 Downloading 5.243MB
f0d8a57b0a96 Downloading 5.243MB
035f2e5bf75c Downloading 5.243MB
035f2e5bf75c Downloading 5.243MB
f0d8a57b0a96 Downloading 6.291MB
f0d8a57b0a96 Downloading 6.291MB
035f2e5bf75c Downloading 6.291MB
035f2e5bf75c Downloading 7.34MB
f0d8a57b0a96 Downloading 6.291MB
035f2e5bf75c Downloading 8.389MB
f0d8a57b0a96 Downloading 7.34MB
035f2e5bf75c Downloading 8.389MB
f0d8a57b0a96 Downloading 7.34MB
f0d8a57b0a96 Downloading 8.389MB
035f2e5bf75c Downloading 9.437MB
035f2e5bf75c Downloading 10.49MB
f0d8a57b0a96 Downloading 8.389MB
f0d8a57b0a96 Downloading 8.389MB
035f2e5bf75c Downloading 11.53MB



La génération se termine sans erreur. Les certificats nécessaires au déploiement peuvent maintenant être utilisés par les différents conteneurs.

4. Démarrage de la plateforme Wazuh

Les trois principaux composants de la plateforme sont ensuite démarrés en arrière-plan avec Docker Compose.

sudo docker compose up -d
sudo docker compose ps -a



```
safia@soc-wazuh:~/wazuh-docker/single-node$ sudo docker compose up -d
sudo docker compose ps -a
[+] up 62/63
 ✔ Image wazuh/wazuh-indexer:4.14.7           Pulled                                  228.4s
 ✔ Image wazuh/wazuh-dashboard:4.14.7         Pulled                                  222.4s
 ✔ Image wazuh/wazuh-manager:4.14.7           Pulled                                  232.3s
 ✔ Volume single-node_filebeat_etc            Created                                   0.0s
 ✔ Volume single-node_wazuh_etc               Created                                   0.0s
 ✔ Volume single-node_wazuh-indexer-data      Created                                   0.0s
 ✔ Volume single-node_wazuh_active_response   Created                                   0.0s
 ✔ Volume single-node_wazuh-dashboard-config  Created                                   0.0s
 ✔ Volume single-node_wazuh_wodles            Created                                   0.0s
 ✔ Volume single-node_wazuh-dashboard-custom  Created                                   0.0s
 ✔ Volume single-node_wazuh_agentless         Created                                   0.0s
 ✔ Volume single-node_wazuh_api_configuration Created                                   0.0s
 ✔ Volume single-node_wazuh_var_multigroups   Created                                   0.0s
 ✔ Volume single-node_wazuh_integrations      Created                                   0.0s
 ✔ Volume single-node_wazuh_logs              Created                                   0.0s
 ✔ Volume single-node_wazuh_queue             Created                                   0.0s
 ✔ Volume single-node_filebeat_var            Created                                   0.0s
 ✔ Container single-node-wazuh.indexer-1      Started                                   1.2s
 ✔ Container single-node-wazuh.manager-1      Started                                   1.6s
 ✔ Container single-node-wazuh.dashboard-1    Started                                   1.2s
NAME                              IMAGE                                COMMAND                  SERVIC
E           CREATED         STATUS                  PORTS
single-node-wazuh.dashboard-1   wazuh/wazuh-dashboard:4.14.7   "/entrypoint.sh"         wazuh.
dashboard   1 second ago    Up Less than a second   443/tcp, 0.0.0.0:443->5601/tcp, [::]:443->
5601/tcp
single-node-wazuh.indexer-1     wazuh/wazuh-indexer:4.14.7     "/entrypoint.sh open…"   wazuh.
indexer     2 seconds ago   Up 1 second             0.0.0.0:9200->9200/tcp, [::]:9200->9200/tc
p
single-node-wazuh.manager-1     wazuh/wazuh-manager:4.14.7     "/init"                  wazuh.
manager     2 seconds ago   Up Less than a second   0.0.0.0:1514-1515->1514-1515/tcp, [::]:151
4-1515->1514-1515/tcp, 0.0.0.0:514->514/udp, [::]:514->514/udp, 0.0.0.0:55000->55000/tcp, [::]
:55000->55000/tcp, 1516/tcp
safia@soc-wazuh:~/wazuh-docker/single-node$
```

La commande docker compose ps -a permet de vérifier l'état des conteneurs.

Les trois services principaux sont présents :

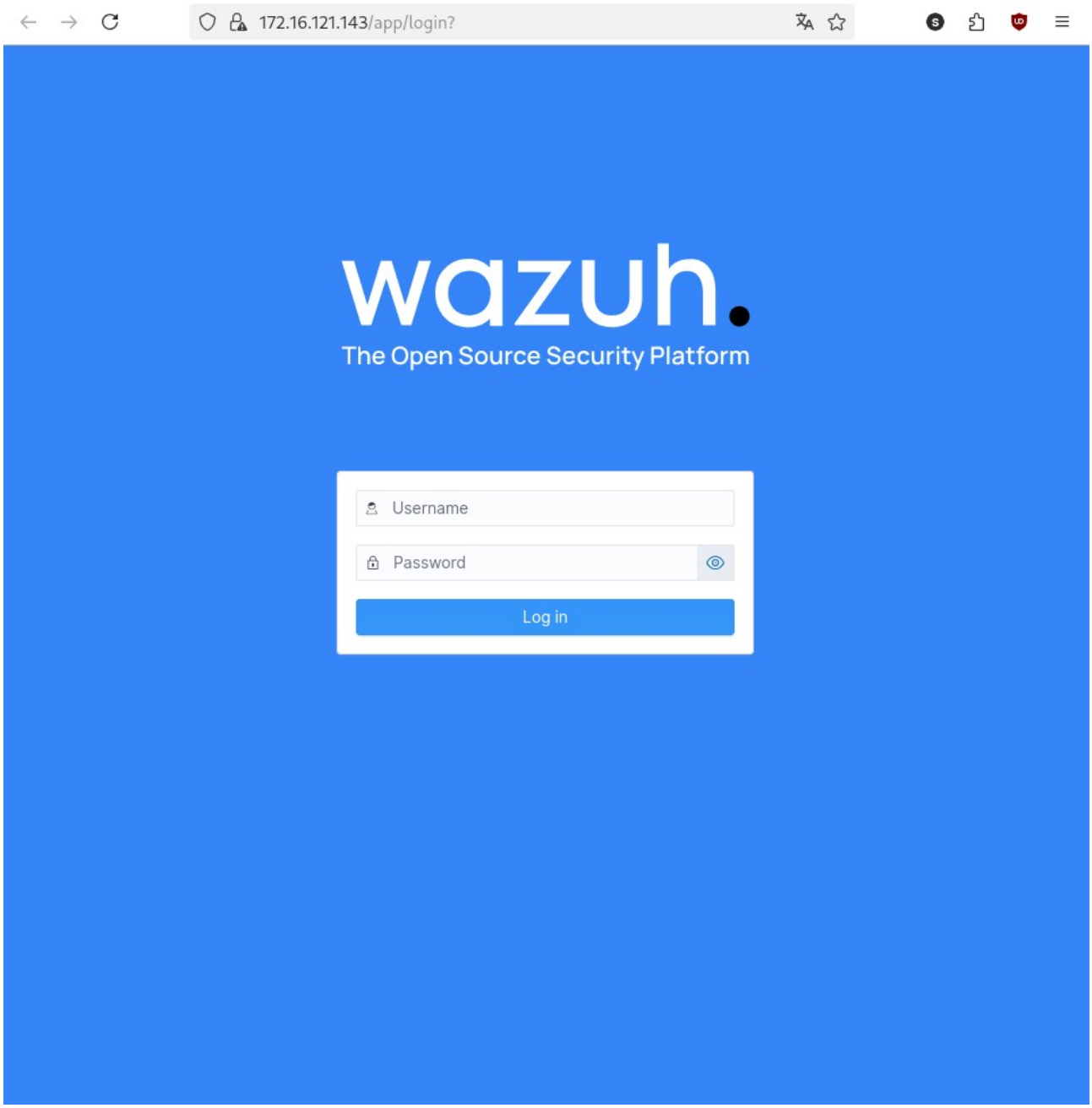
- wazuh.manager
- wazuh.indexer
- wazuh.dashboard

Leur état confirme que la plateforme Wazuh a correctement démarré.

5. Vérification de l'accès au Dashboard Wazuh

Après le démarrage des conteneurs, l'interface Web du serveur Wazuh est accessible depuis le navigateur.

L'affichage de la page d'authentification confirme que le service Wazuh Dashboard est opérationnel.



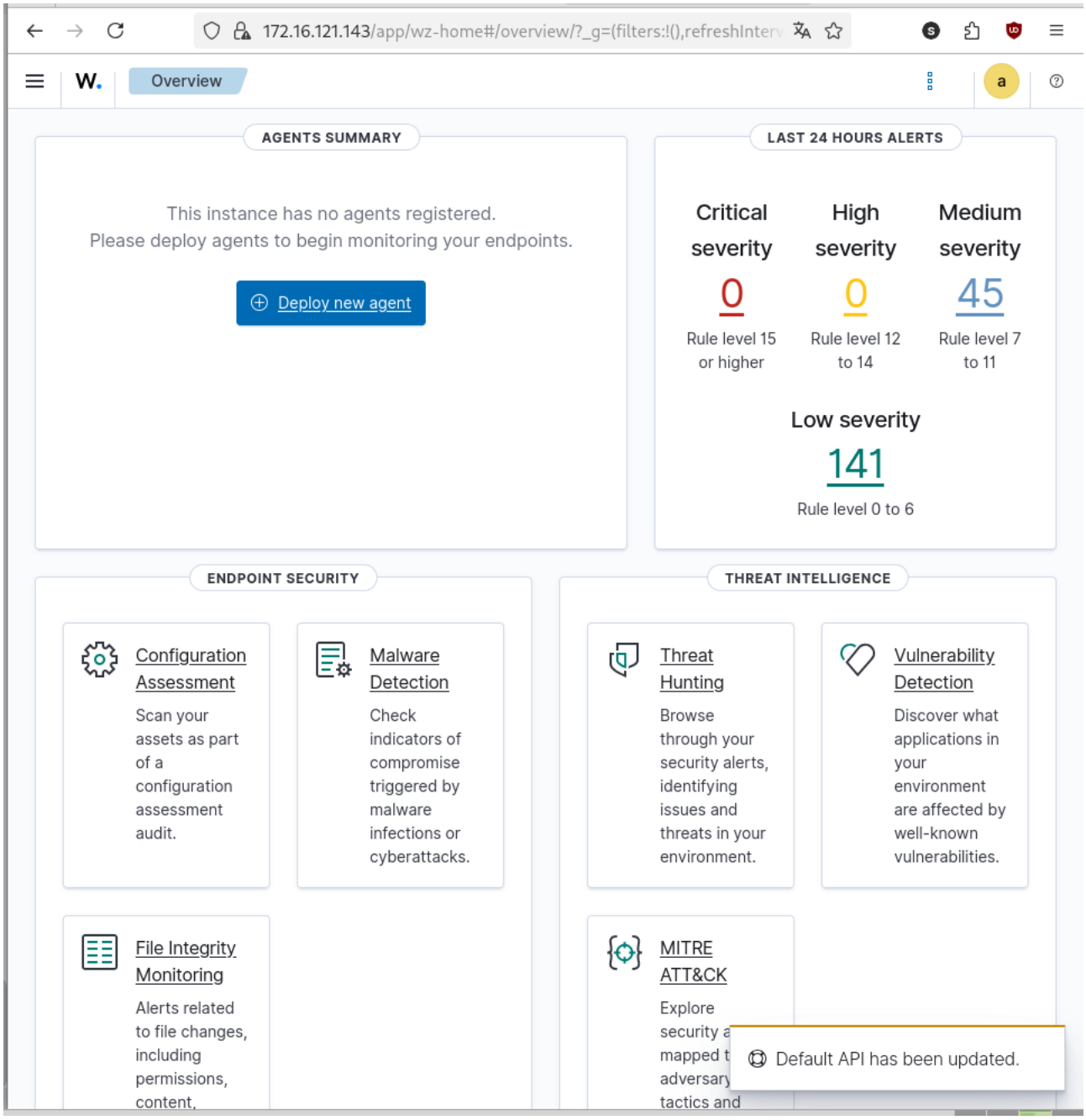
Après authentification, le tableau de bord Wazuh est accessible. À ce stade, aucun agent n'est encore enregistré sur le serveur. La prochaine étape consiste donc à déployer un agent sur une machine cliente afin de commencer la supervision.

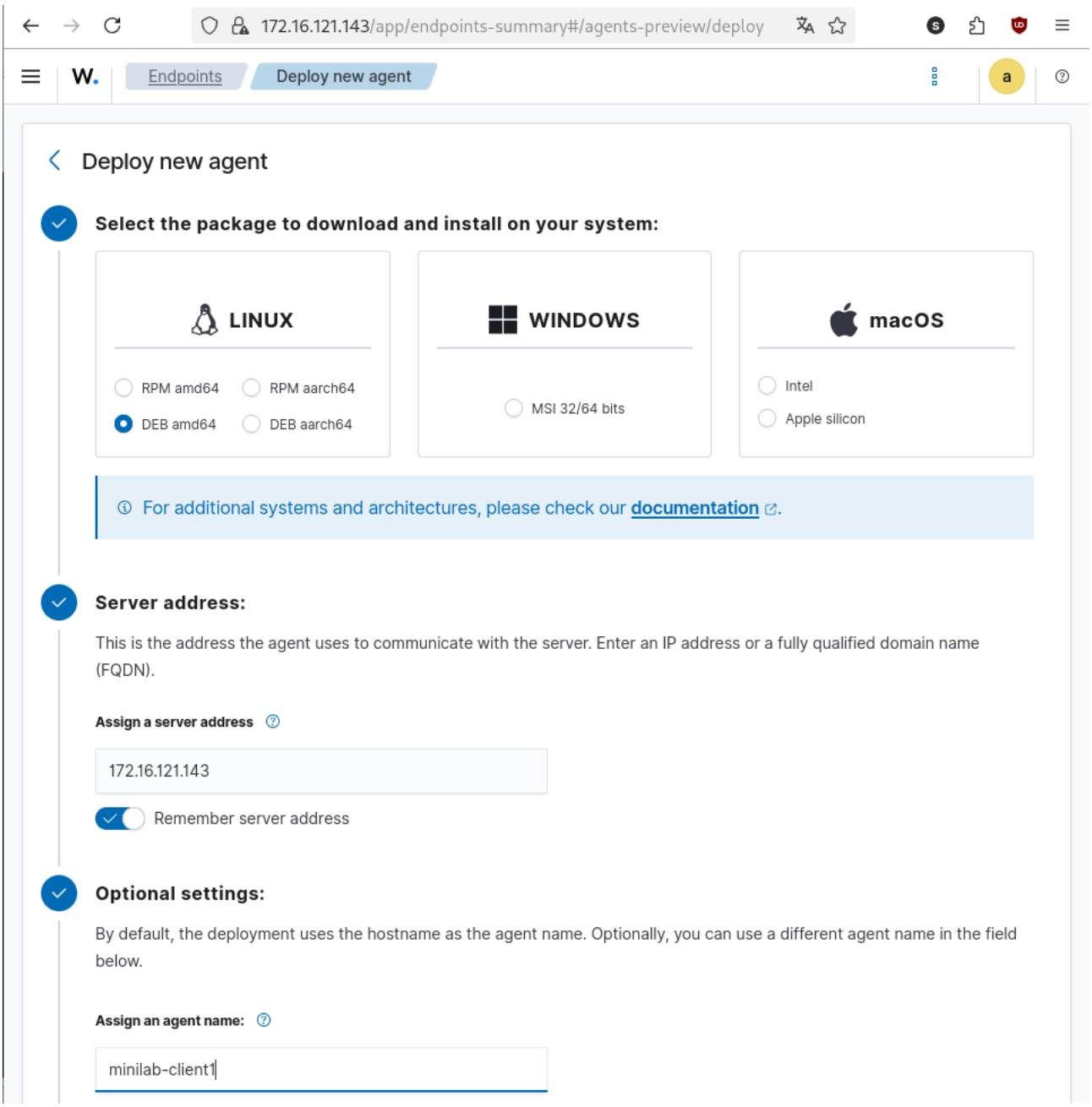
6. Déploiement d'un agent Wazuh sur le client Debian

Pour permettre la supervision d'un équipement, un agent Wazuh doit être installé sur celui-ci.

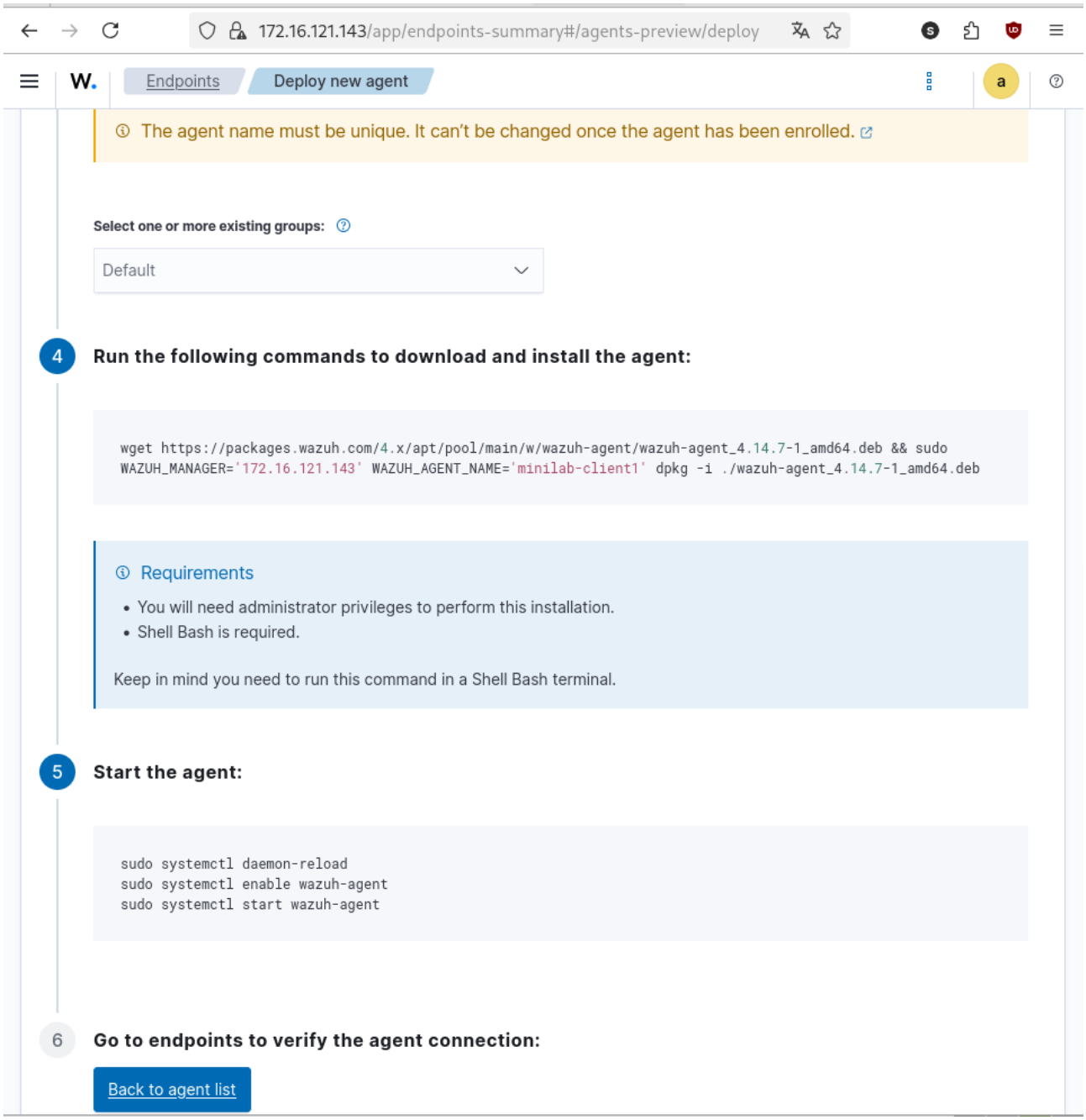
Depuis le Dashboard, l'assistant Deploy new agent est utilisé. Le système Linux/Debian est sélectionné, puis l'adresse du serveur Wazuh 172.16.121.143 est renseignée.

Le nom minilab-client1 est attribué à l'agent afin de pouvoir identifier facilement la machine supervisée dans l'interface.



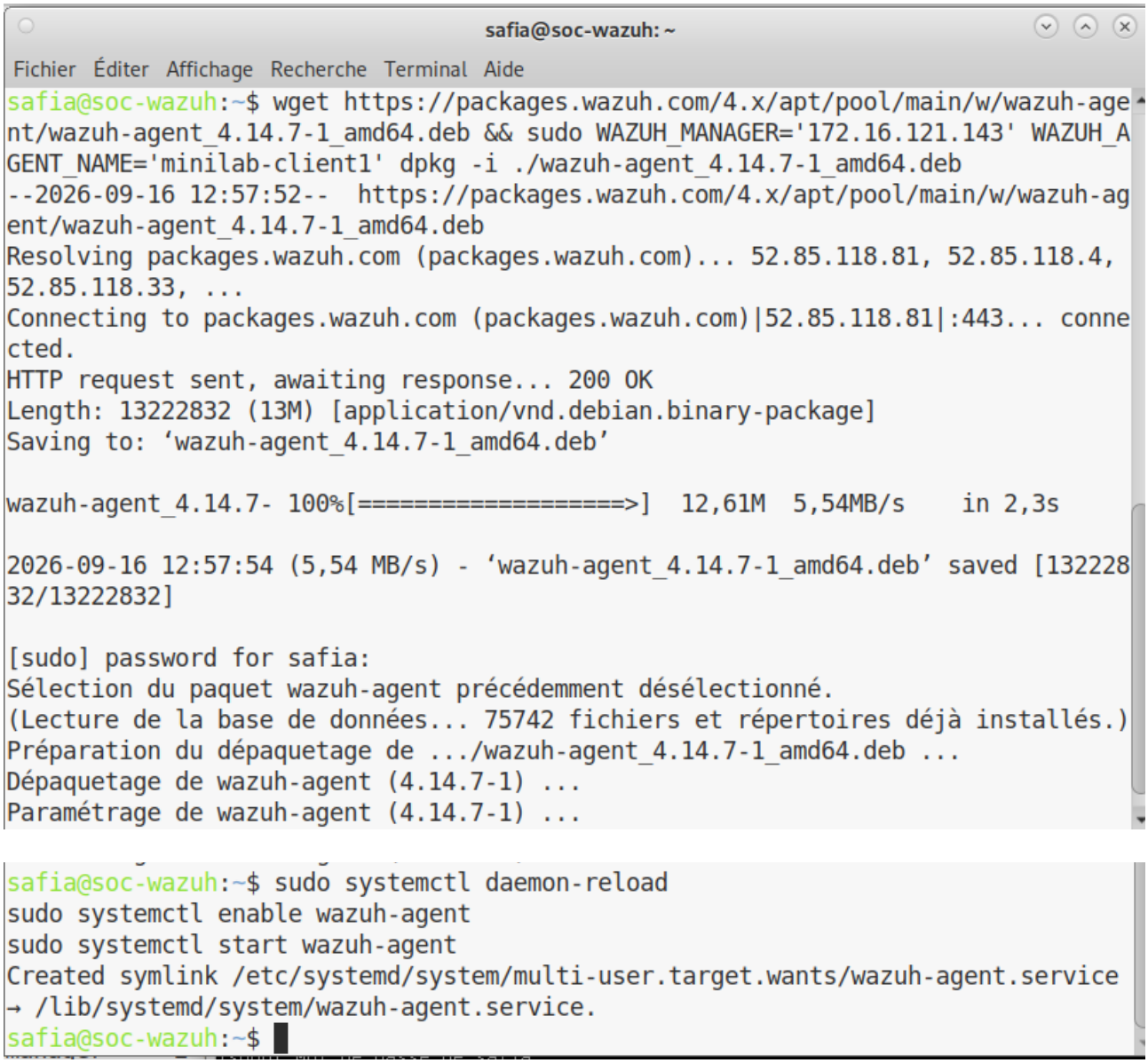


Wazuh génère automatiquement la commande permettant de télécharger et d'installer le paquet correspondant sur le poste client.

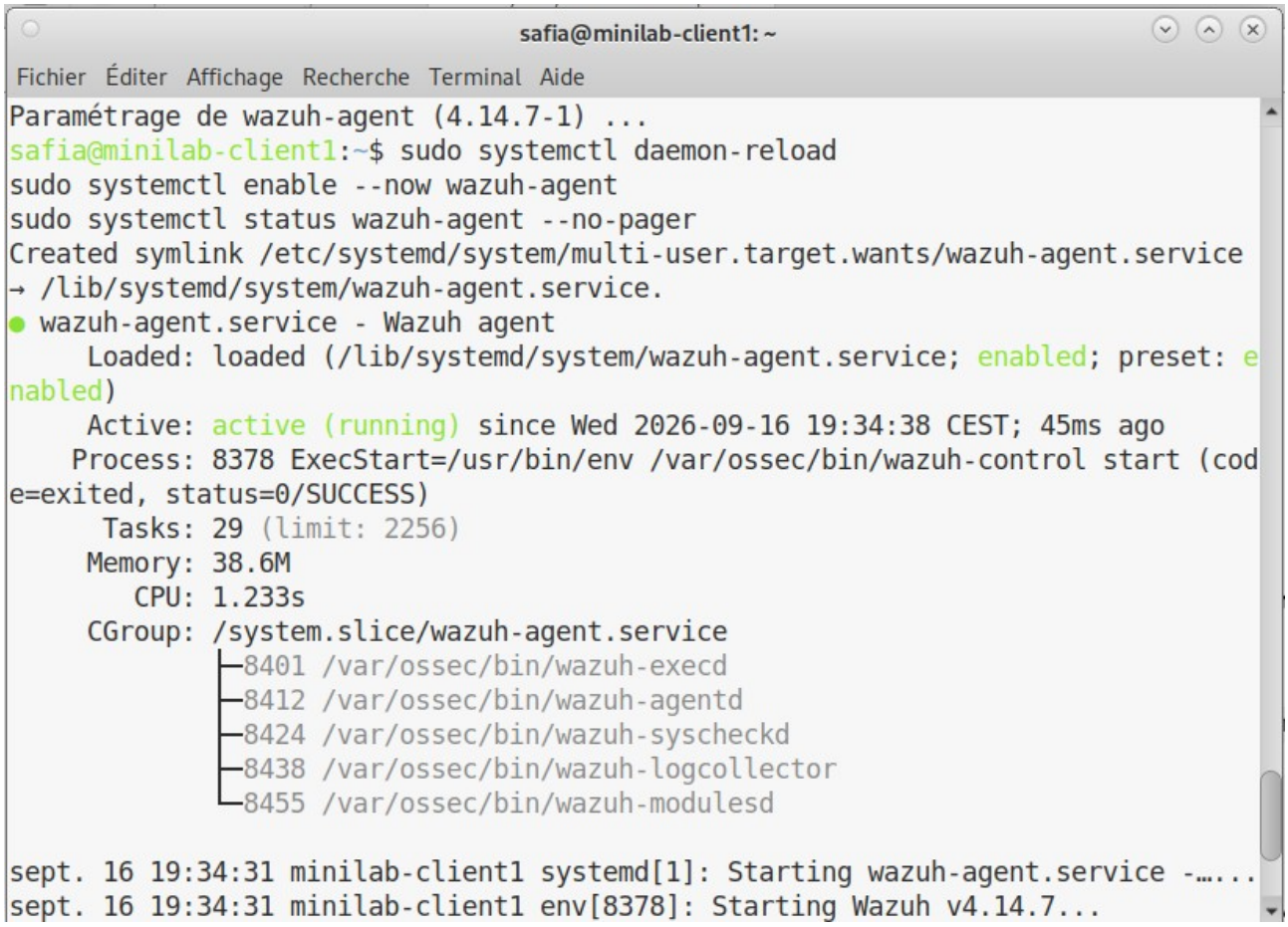


Une fois le paquet installé, le service wazuh-agent est activé et démarré :

sudo systemctl daemon-reload
sudo systemctl enable wazuh-agent
sudo systemctl start wazuh-agent


safia@soc-wazuh: ~
Fichier Éditer Affichage Recherche Terminal Aide
safia@soc-wazuh:~$ wget https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.14.7-1_amd64.deb && sudo WAZUH_MANAGER='172.16.121.143' WAZUH_AGENT_NAME='minilab-client1' dpkg -i ./wazuh-agent_4.14.7-1_amd64.deb
--2026-09-16 12:57:52-- https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.14.7-1_amd64.deb
Resolving packages.wazuh.com (packages.wazuh.com)... 52.85.118.81, 52.85.118.4, 52.85.118.33, ...
Connecting to packages.wazuh.com (packages.wazuh.com)|52.85.118.81|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 13222832 (13M) [application/vnd.debian.binary-package]
Saving to: ‘wazuh-agent_4.14.7-1_amd64.deb’
wazuh-agent_4.14.7- 100%[==========>] 12,61M 5,54MB/s in 2,3s
2026-09-16 12:57:54 (5,54 MB/s) - ‘wazuh-agent_4.14.7-1_amd64.deb’ saved [13222832/13222832]
[sudo] password for safia:
Sélection du paquet wazuh-agent précédemment désélectionné.
(Lecture de la base de données... 75742 fichiers et répertoires déjà installés.)
Préparation du dépaquetage de .../wazuh-agent_4.14.7-1_amd64.deb ...
Dépaquetage de wazuh-agent (4.14.7-1) ...
Paramétrage de wazuh-agent (4.14.7-1) ...

safia@soc-wazuh:~$ sudo systemctl daemon-reload
sudo systemctl enable wazuh-agent
sudo systemctl start wazuh-agent
Created symlink /etc/systemd/system/multi-user.target.wants/wazuh-agent.service → /lib/systemd/system/wazuh-agent.service.
safia@soc-wazuh:~$

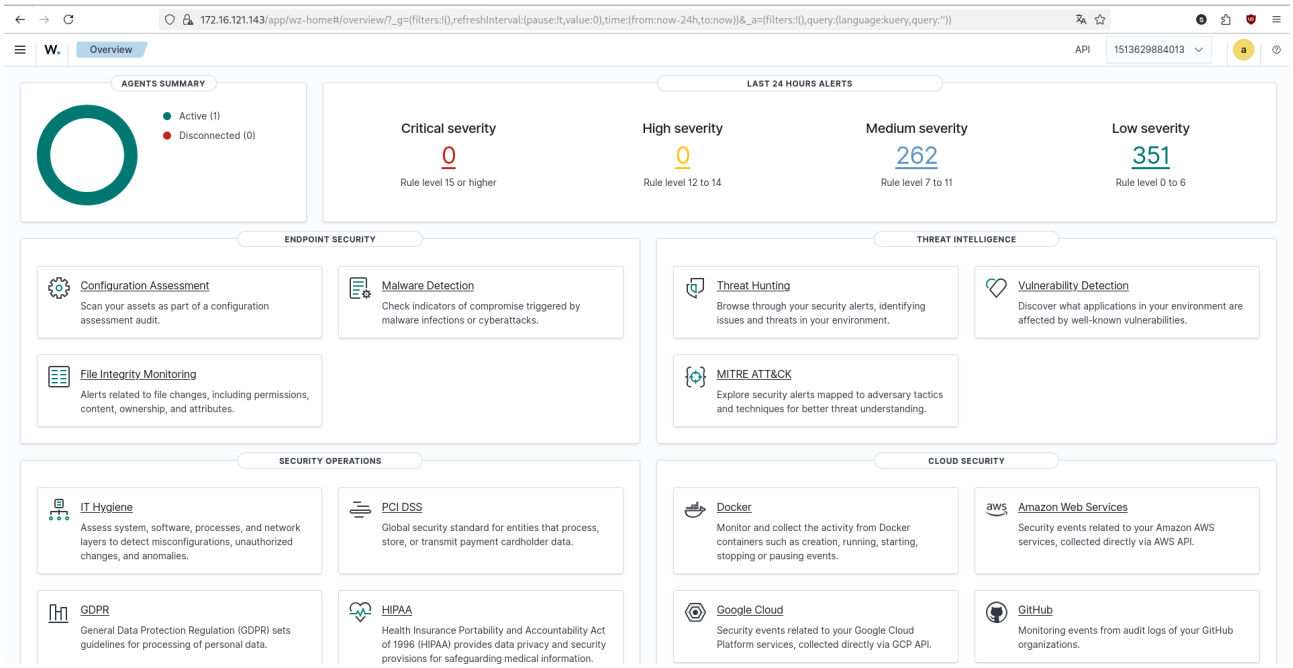


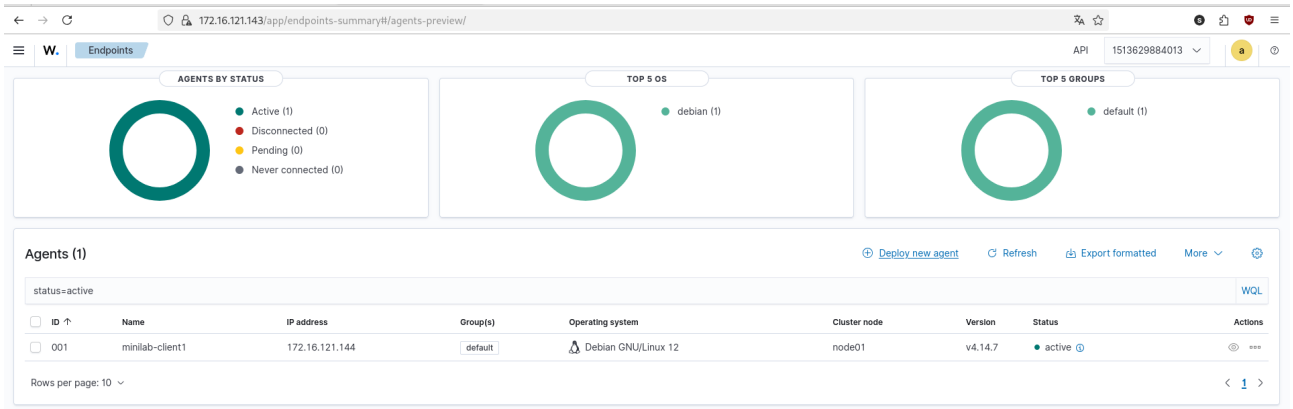
La commande de vérification indique que le service wazuh-agent est **active (running)**. L'agent est donc correctement démarré sur minilab-client1.

7. Validation de la supervision

Le Dashboard Wazuh affiche désormais un agent actif : minilab-client1. Cette information confirme que la communication entre le poste Debian et le serveur Wazuh fonctionne correctement.

L'agent peut maintenant transmettre au manager les informations nécessaires à la supervision et aux contrôles de sécurité.



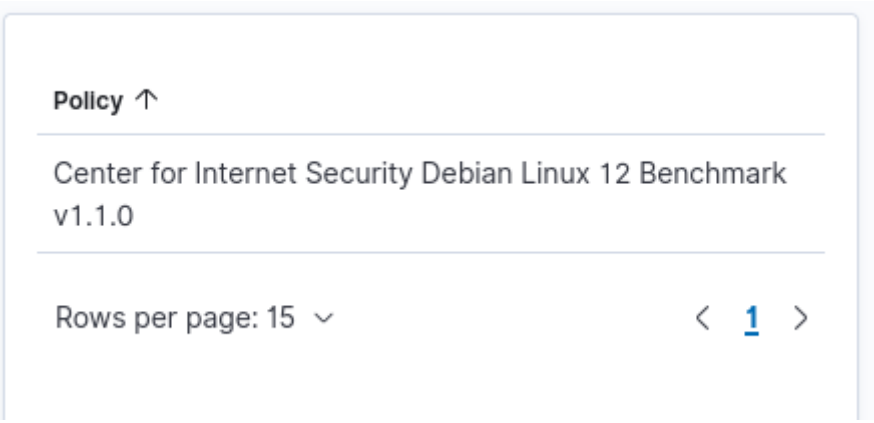


8. Évaluation de la configuration de sécurité

Wazuh intègre également des fonctions d'évaluation de la configuration de sécurité des systèmes supervisés.

Pour l'agent Debian, une politique basée sur le **CIS Debian Linux 12 Benchmark v1.1.0** est disponible. Elle permet de comparer certains paramètres de configuration du système avec les recommandations de sécurité définies par le CIS.

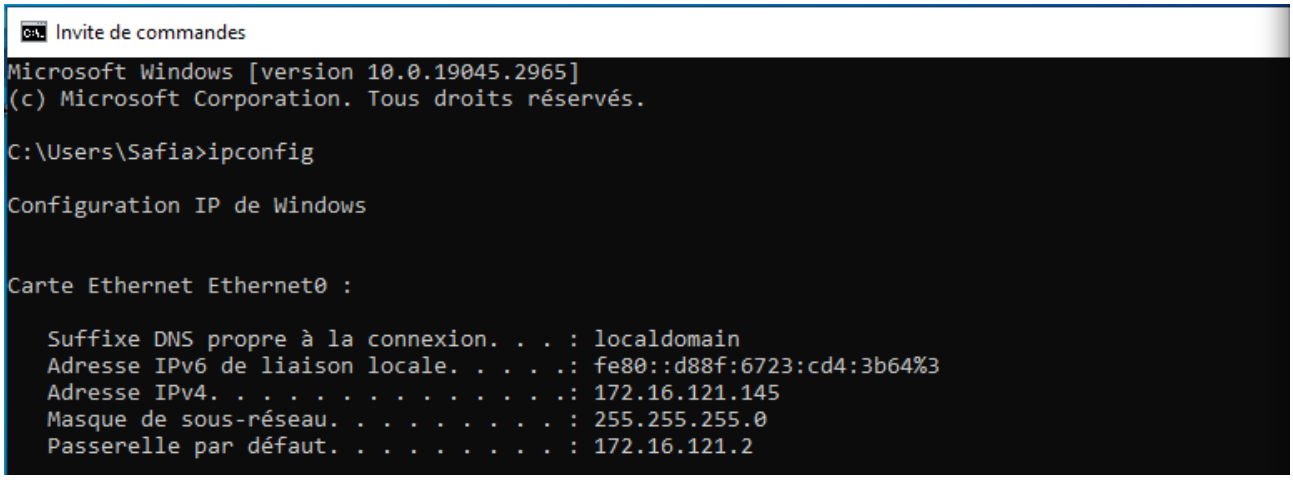
Cette fonctionnalité permettra par la suite d'identifier les contrôles conformes ou non conformes et de proposer des mesures de durcissement adaptées.



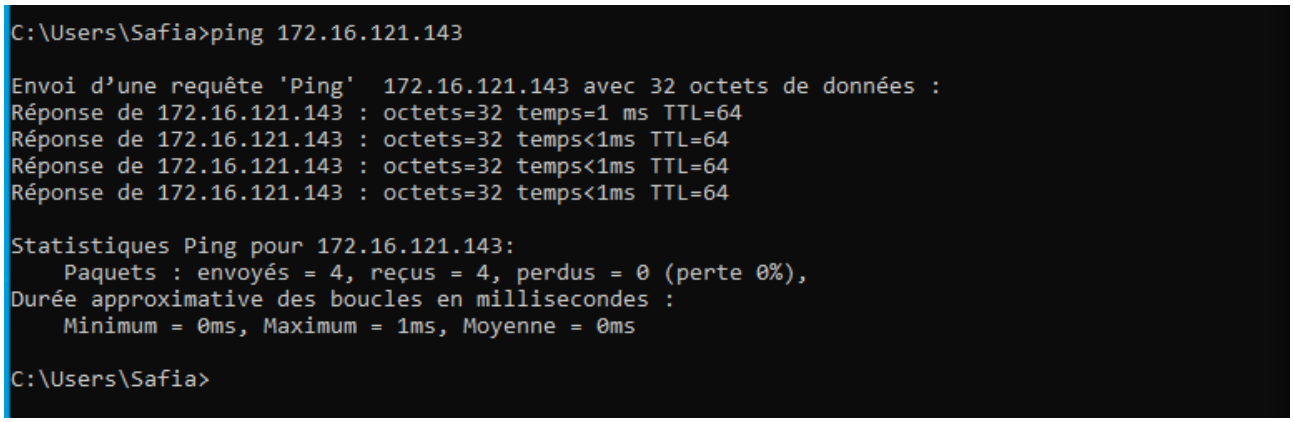
9. Installation des agents Windows

9.1. Paramétrage

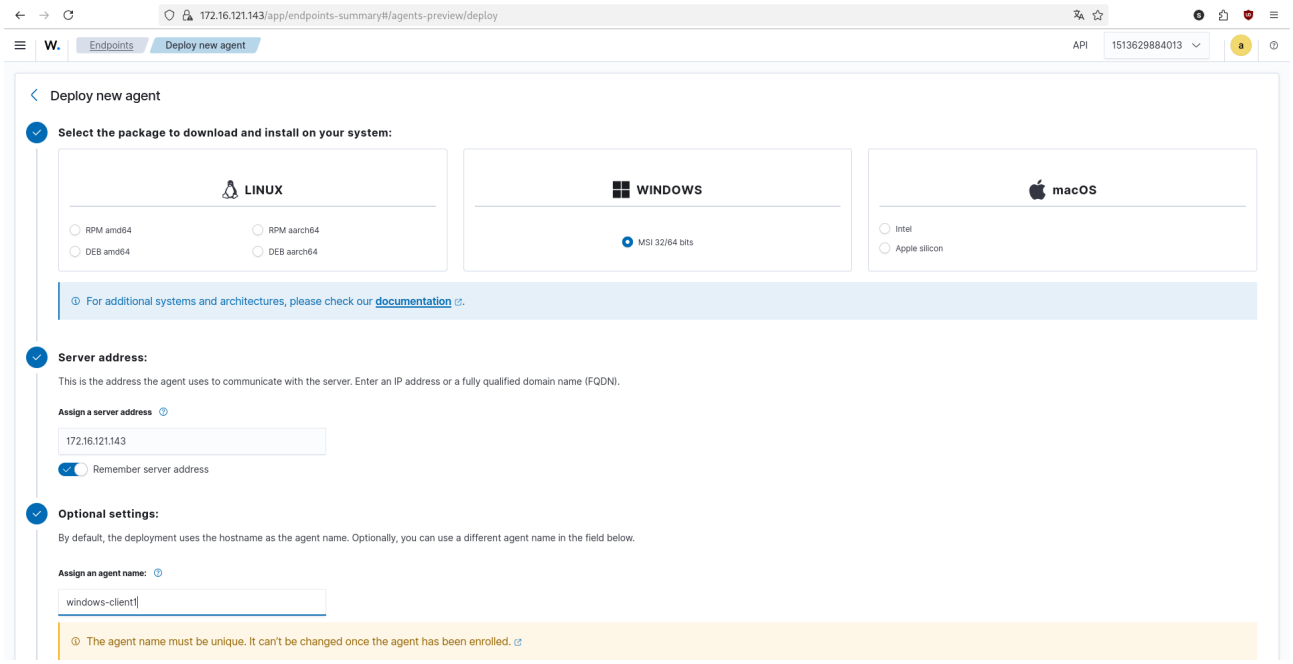
On instlle une VM Windows 10, windows-client1, qui servira d'agent Wazuh. Puis on vérifie la connectivité avec Le serveur Wazuh. Avec la commande Ipconfig pour obtenir l'IP de la nouvelle VM.

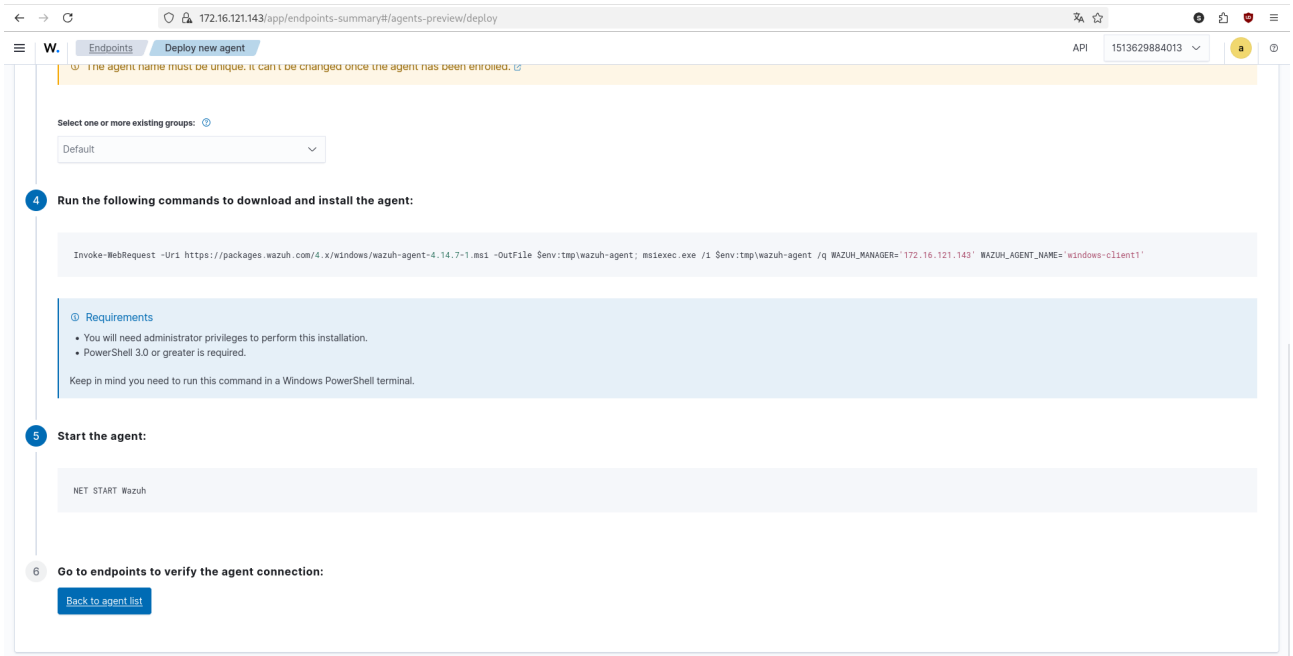


On teste la connectivité avec le manager Wazuh soc-wazuh avec la commande :
ping 172.16.121.143



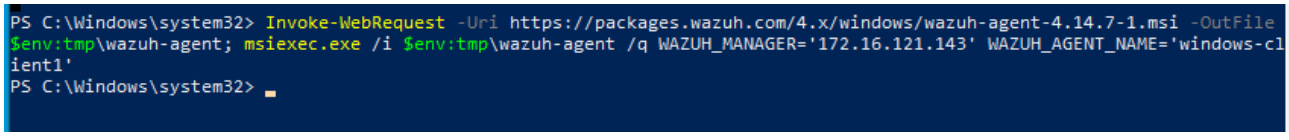
9.2. Enregistrement de windows-client1 dans l'endpoint





On exécute ensuite la commande pour lancer l'agent wazuh :

Invoke-WebRequest -Uri https://packages.wazuh.com/4.x/windows/wazuh-agent-4.14.7-1.msi -OutFile $env:tmp\wazuh-agent; msiexec.exe /i $env:tmp\wazuh-agent /q WAZUH_MANAGER='172.16.121.143' WAZUH_AGENT_NAME='windows-client1'



Puis on démarre le service avec la commande :

Start-Service wazuhsvc

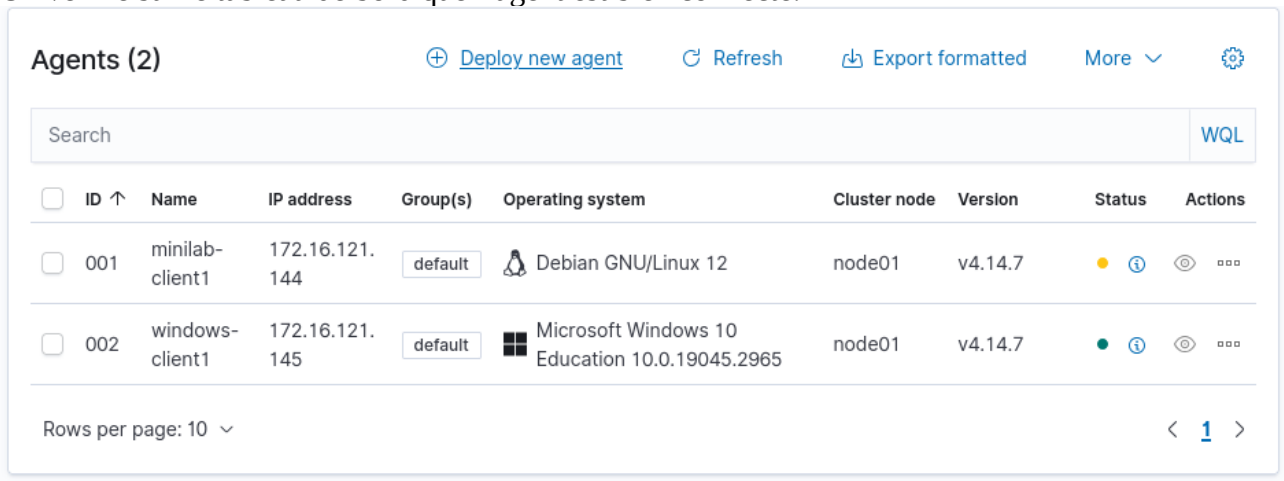
Enfin, on vérifie que le service est démarré avecla commande :
Get-Service wazuhsvc

```
PS C:\Windows\system32> Start-Service wazuhsvc
PS C:\Windows\system32> Get-Service wazuhsvc

Status   Name               DisplayName
------   ----               -----------
Running  wazuhsvc           Wazuh

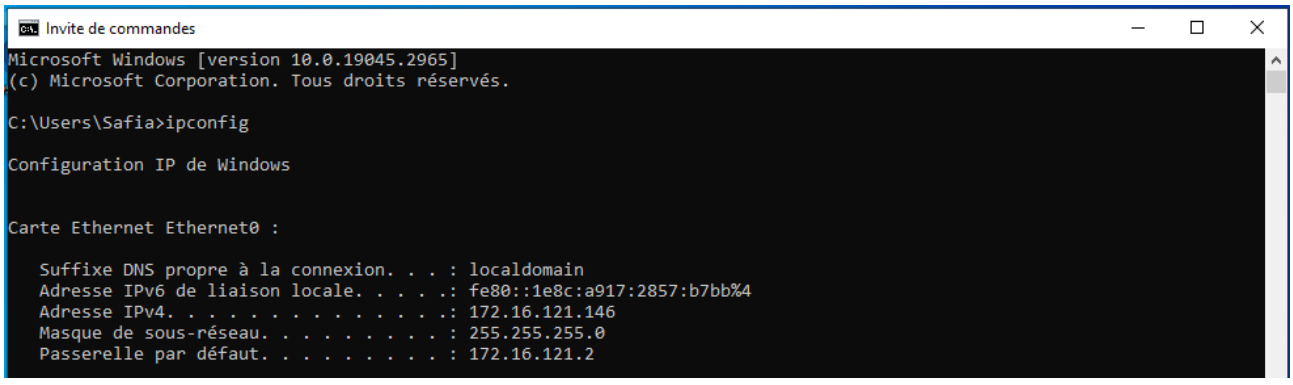

PS C:\Windows\system32>
```

On vérifie sur le tableau de bord que l'agent est bien connecté.



10. Ajout de l'agent windows-2

10.1. Création de la VM windows-client2 :



Vérification de la connectivité avec le manager Wazuh :
ping 172.16.121.143

```
C:\Users\Safia>ping 172.16.121.143

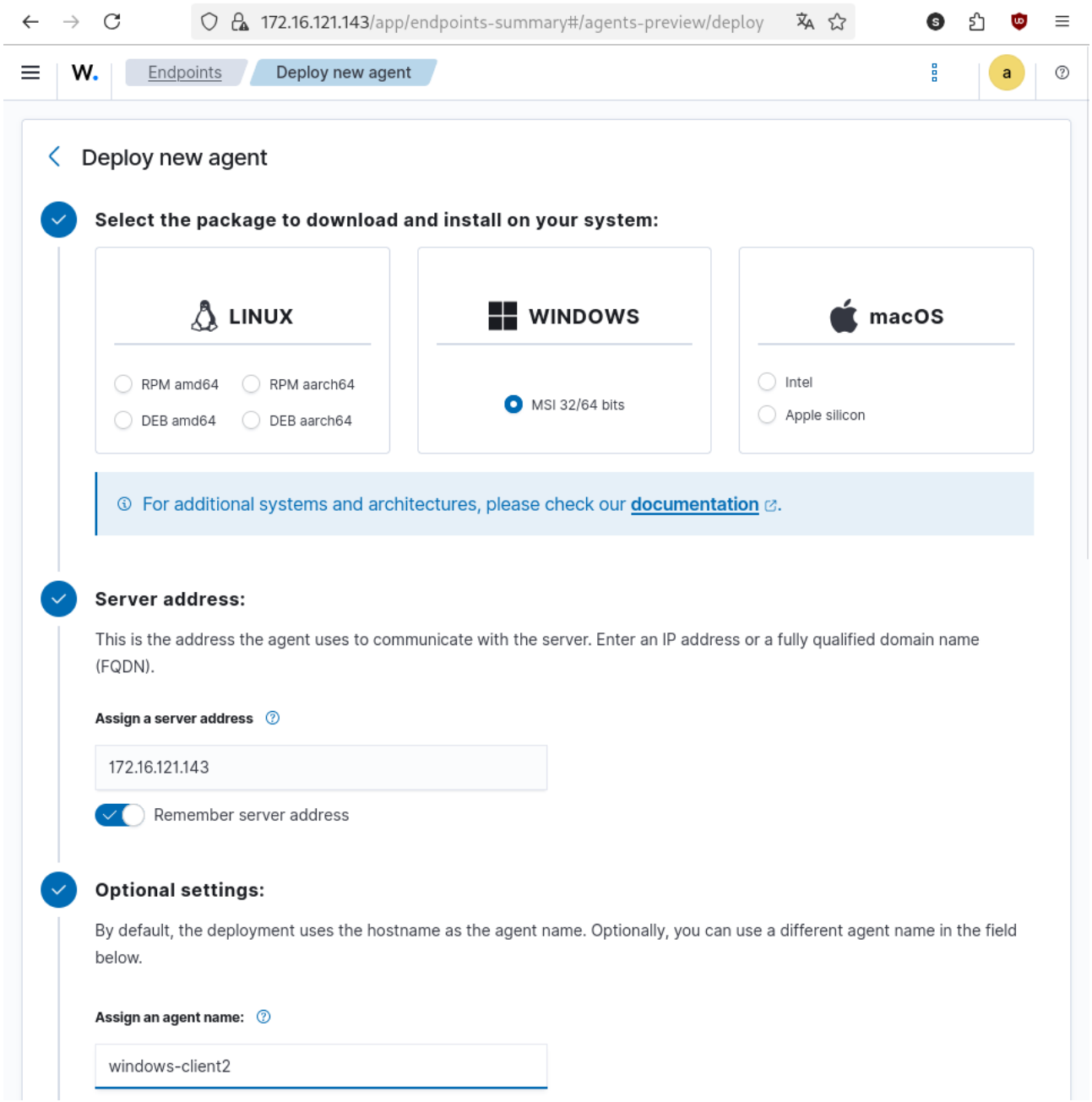
Envoi d'une requête 'Ping'  172.16.121.143 avec 32 octets de données :
Réponse de 172.16.121.143 : octets=32 temps<1ms TTL=64
Réponse de 172.16.121.143 : octets=32 temps<1ms TTL=64
Réponse de 172.16.121.143 : octets=32 temps<1ms TTL=64
Réponse de 172.16.121.143 : octets=32 temps=1 ms TTL=64

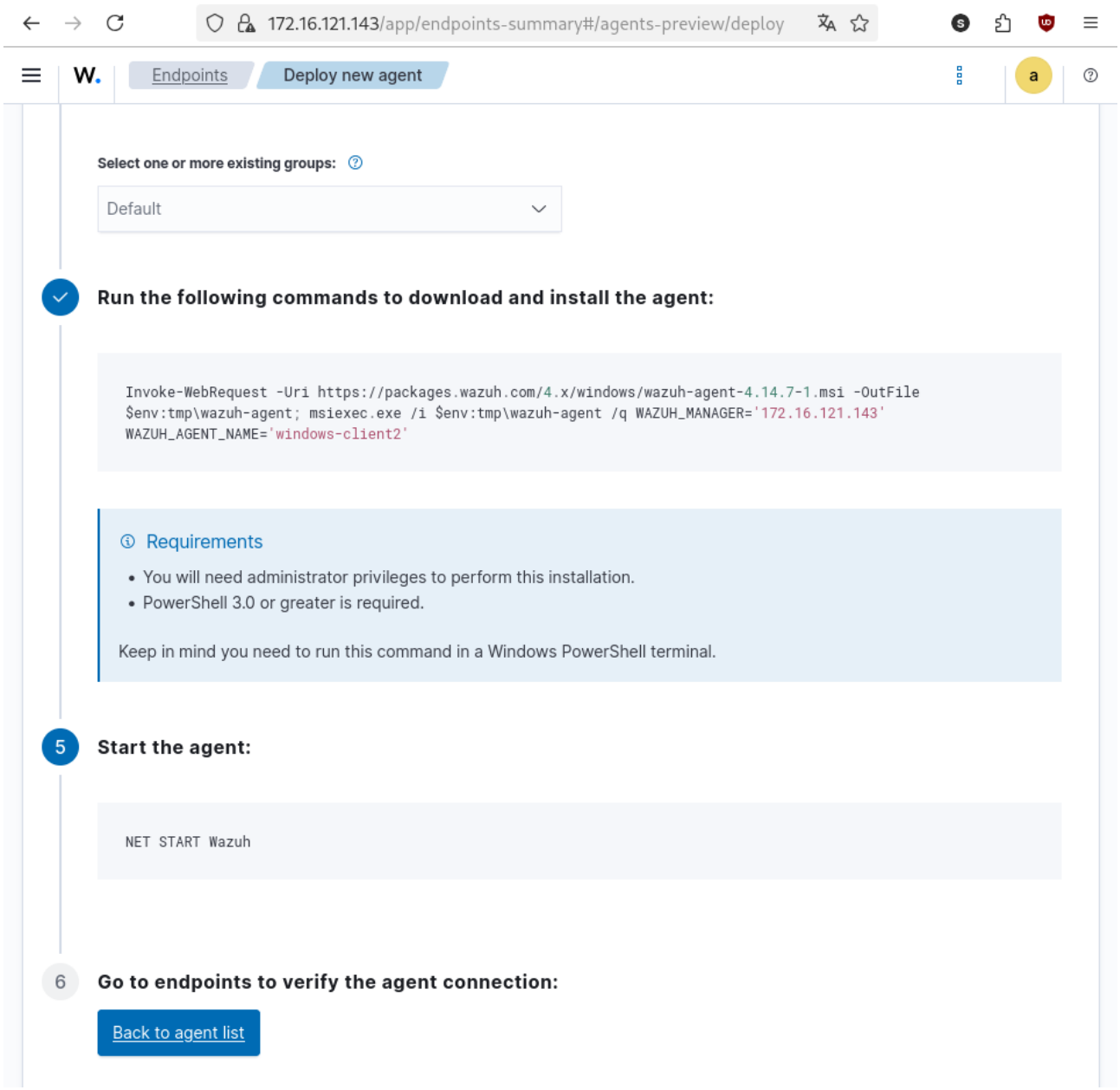
Statistiques Ping pour 172.16.121.143:
    Paquets : envoyés = 4, reçus = 4, perdus = 0 (perte 0%),
Durée approximative des boucles en millisecondes :
    Minimum = 0ms, Maximum = 1ms, Moyenne = 0ms

C:\Users\Safia>
```

10.2. Enregistrement de l'agent wazuh windows-client2

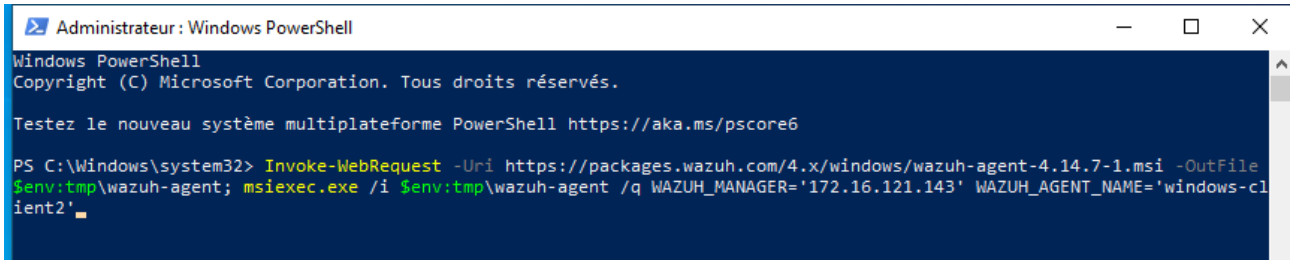
On ajoute la VM sur le tableau de bord de l'interface Wazuh





Depuis la VM que l'on vient de créer, windows-client2, on ouvre un terminal PowerShell et on copie-colle la commande suivante :

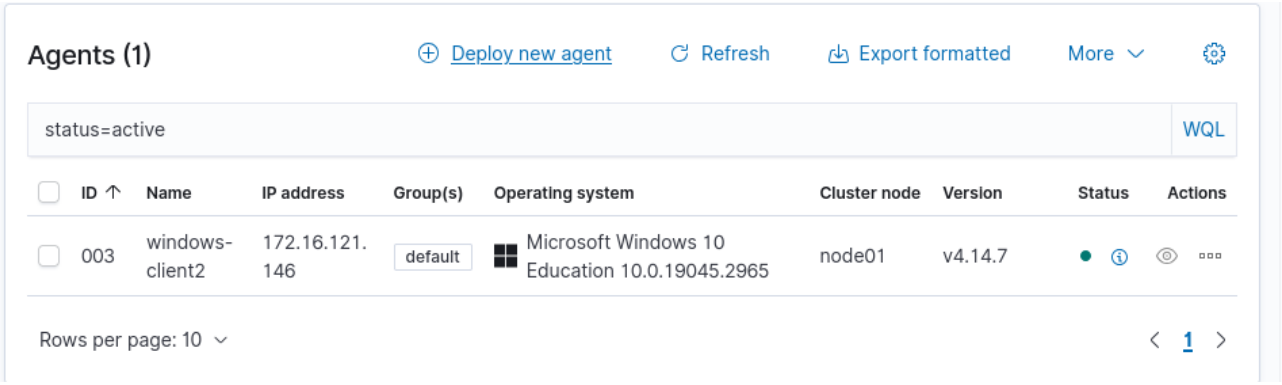
Invoke-WebRequest -Uri https://packages.wazuh.com/4.x/windows/wazuh-agent-4.14.7-1.msi -OutFile $env:tmp\wazuh-agent; msiexec.exe /i $env:tmp\wazuh-agent /q WAZUH_MANAGER='172.16.121.143' WAZUH_AGENT_NAME='windows-client2'



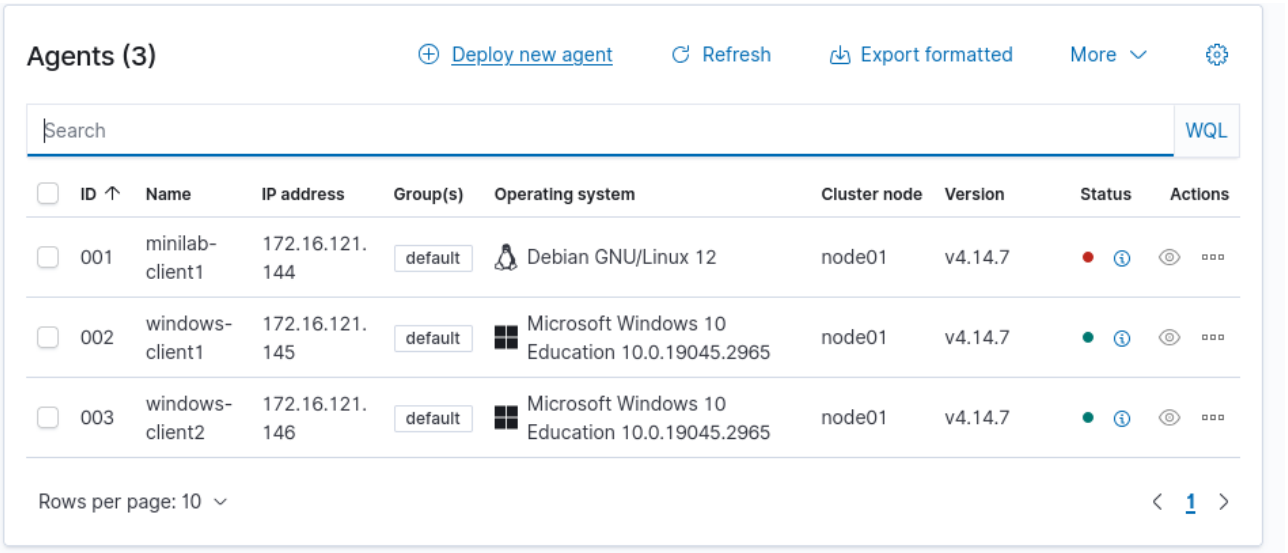
Ensuite on démarre le service avec la commande :
NET START Wazuh



Sur l'interface graphique de la page web on retrouve l'agent enregistré :



On peut voir les 3 agents enregistrés :



Les trois agents ont été déployés et validés individuellement. En raison des ressources mémoire limitées de l'hôte de virtualisation, les trois machines clientes n'ont pas été maintenues simultanément en fonctionnement. Chaque endpoint a néanmoins été démarré et sa communication avec le manager Wazuh a été vérifiée.

11. Configuration fonctionnelle de Wazuh

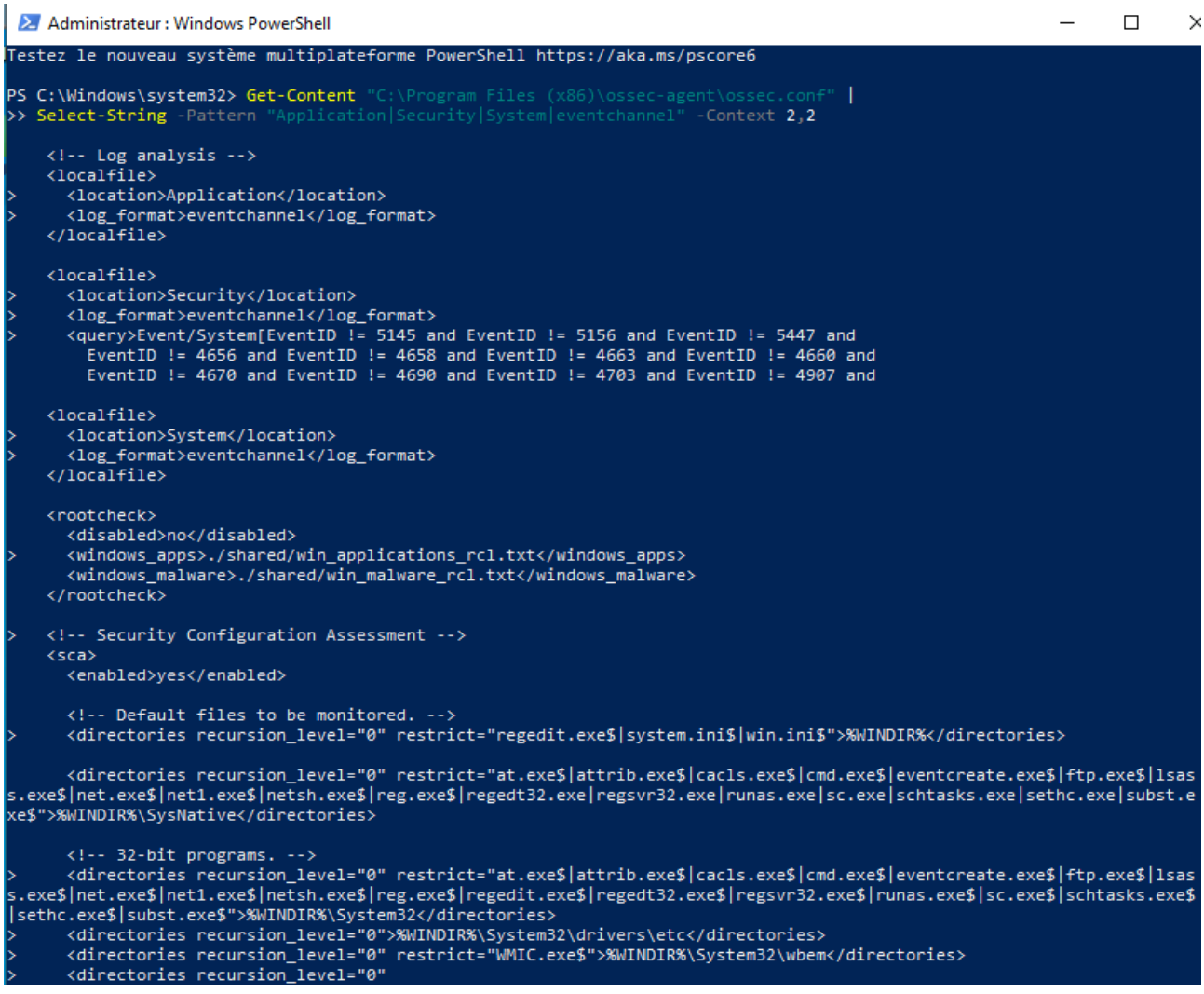
Afin de collecter les journaux Windows Security, System, Application, puis d'activer FIM et SCA.

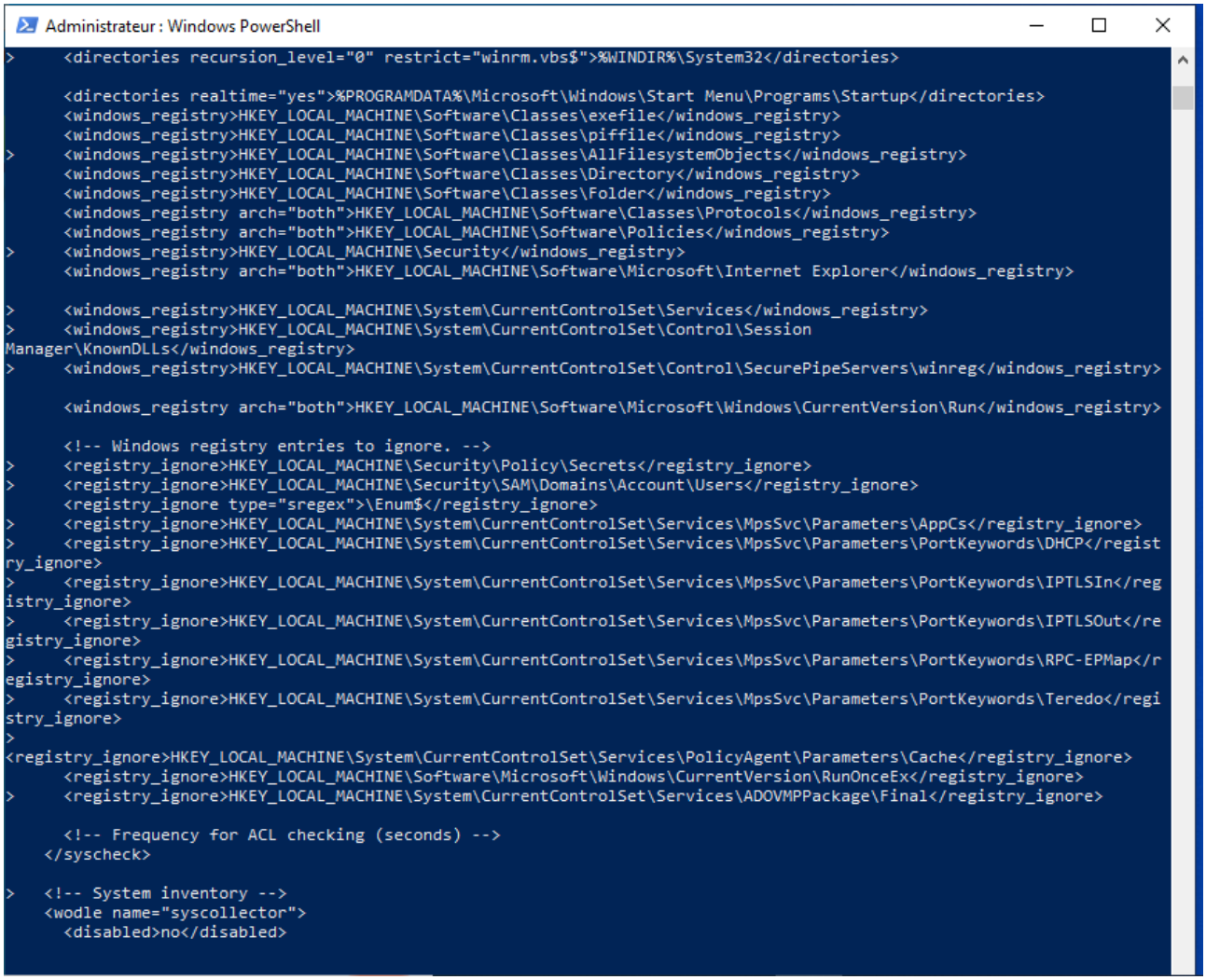
11.1. Collecte des informations :

11.1.2. Sur windows-client1

On ouvre PowerShell en administrateur et on tape la commande :

Get-Content "C:\Program Files (x86)\ossec-agent\ossec.conf" |
Select-String -Pattern "Application|Security|System|eventchannel" -Context 2,2



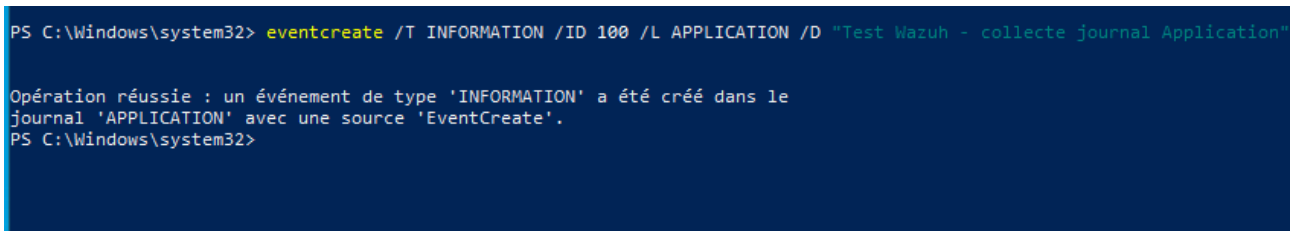


L’agent Wazuh Windows est déjà configuré pour collecter les trois journaux : Application, Security et System.

SCA est également déjà activé.

On lance la commande sur PowerShell :

eventcreate /T INFORMATION /ID 100 /L APPLICATION /D "Test Wazuh - collecte journal Application"

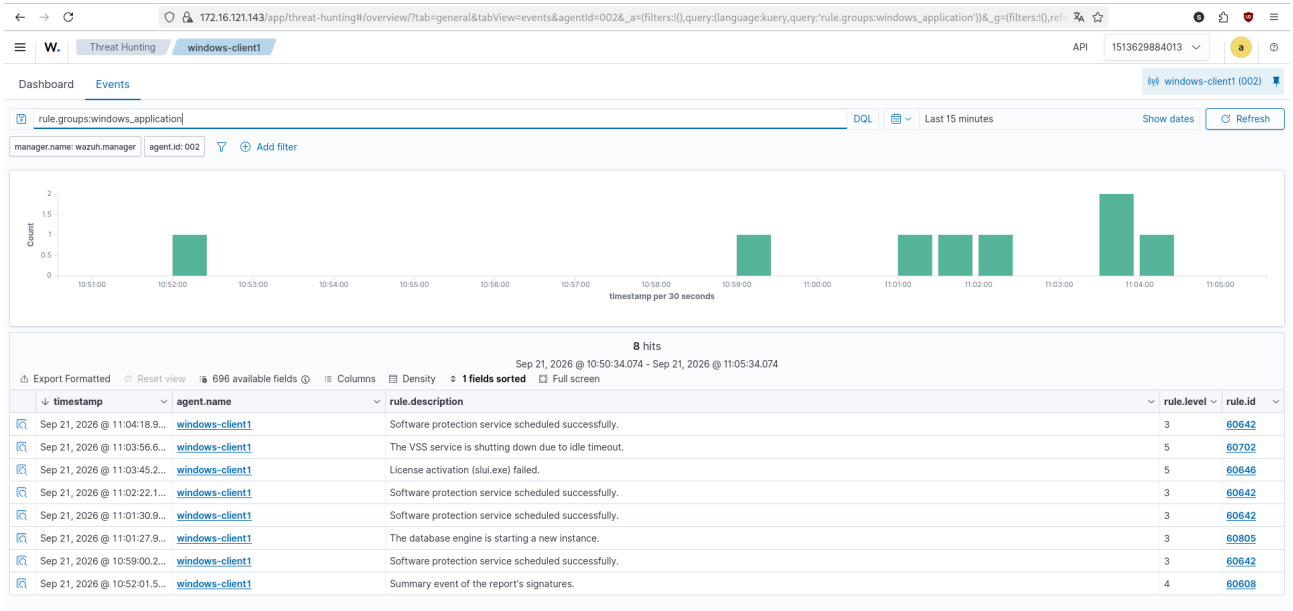


On crée un évènement pour voir si Wazuh l’enregistre dans le journal :

eventcreate /T INFORMATION /ID 100 /L APPLICATION /D "Test Wazuh - collecte journal Application"

```
PS C:\Windows\system32> eventcreate /T INFORMATION /ID 100 /L APPLICATION /D "Test Wazuh - collecte journal Application"

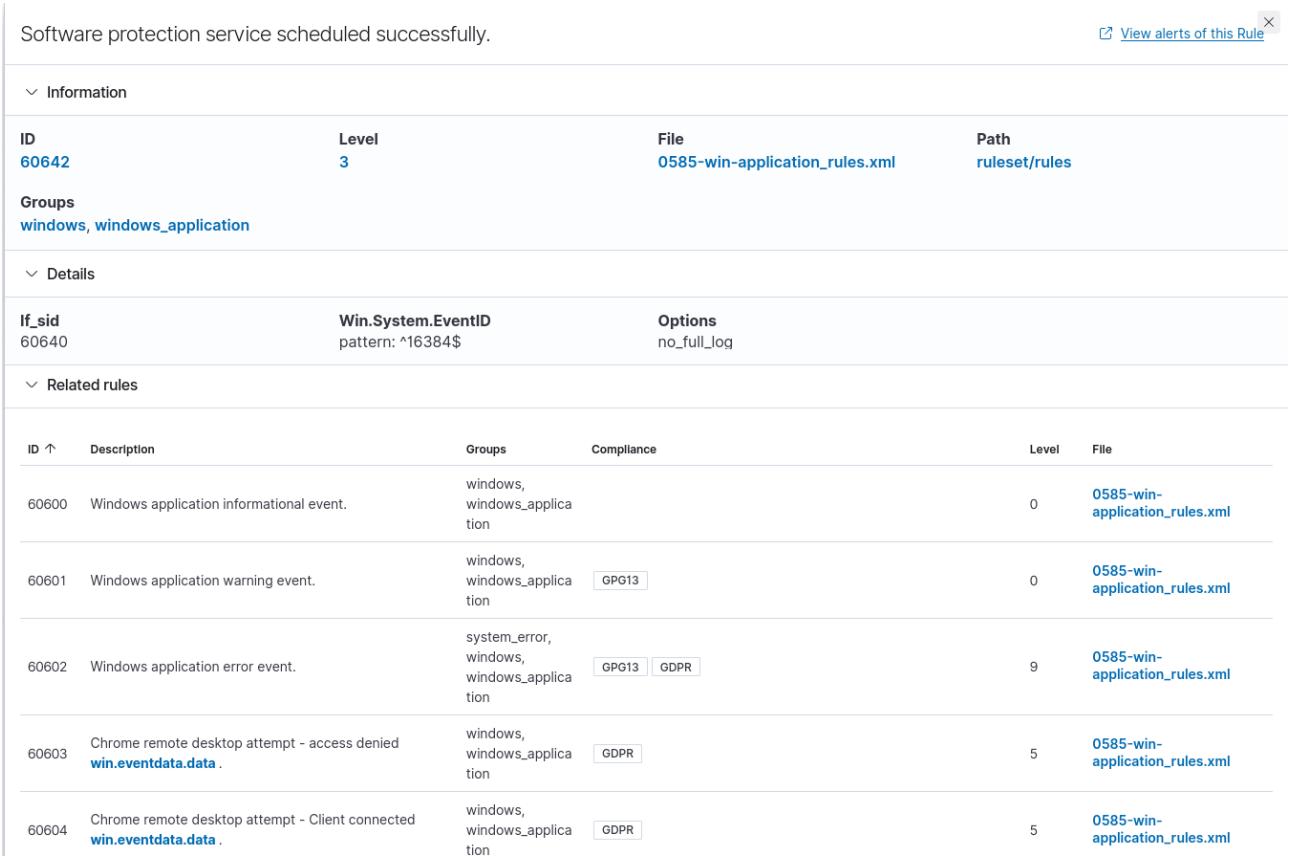
Opération réussie : un événement de type 'INFORMATION' a été créé dans le
journal 'APPLICATION' avec une source 'EventCreate'.
PS C:\Windows\system32>
```



Wazuh retourne **8 événements** provenant de windows-client1 sur les 15 dernières minutes.

On voit notamment des événements comme :

- Software protection service scheduled successfully
- The VSS service is shutting down due to idle timeout
- License activation (slui.exe) failed
- The database engine is starting a new instance
- Summary event of the report's signatures

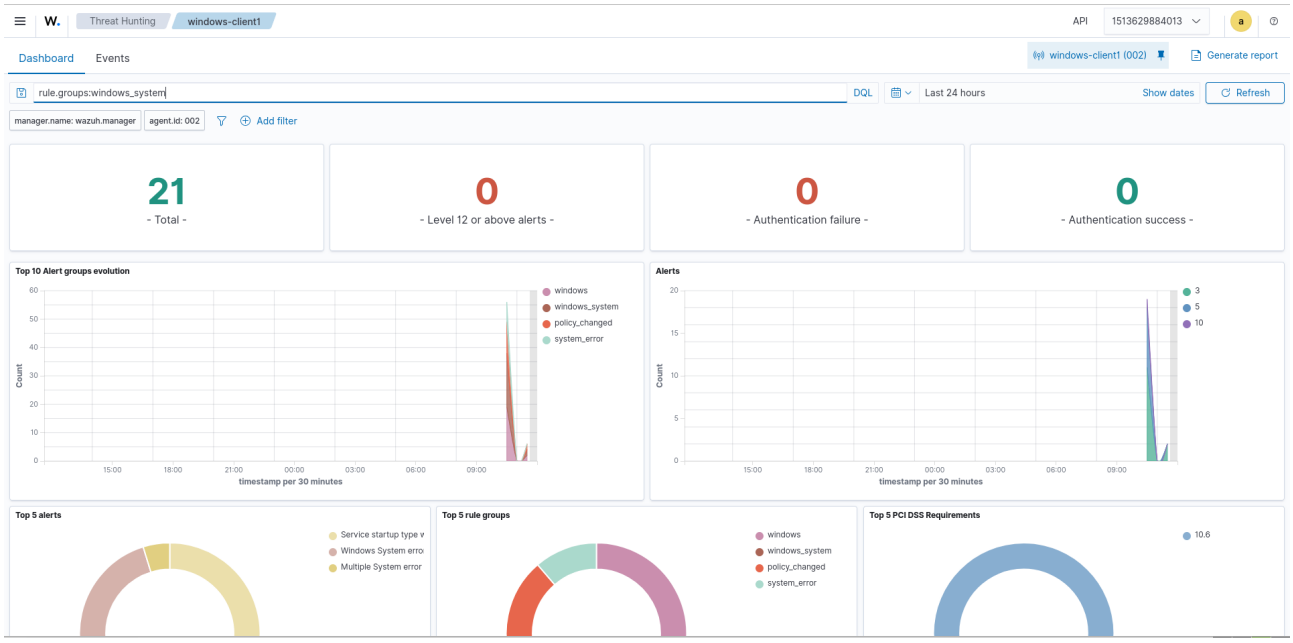


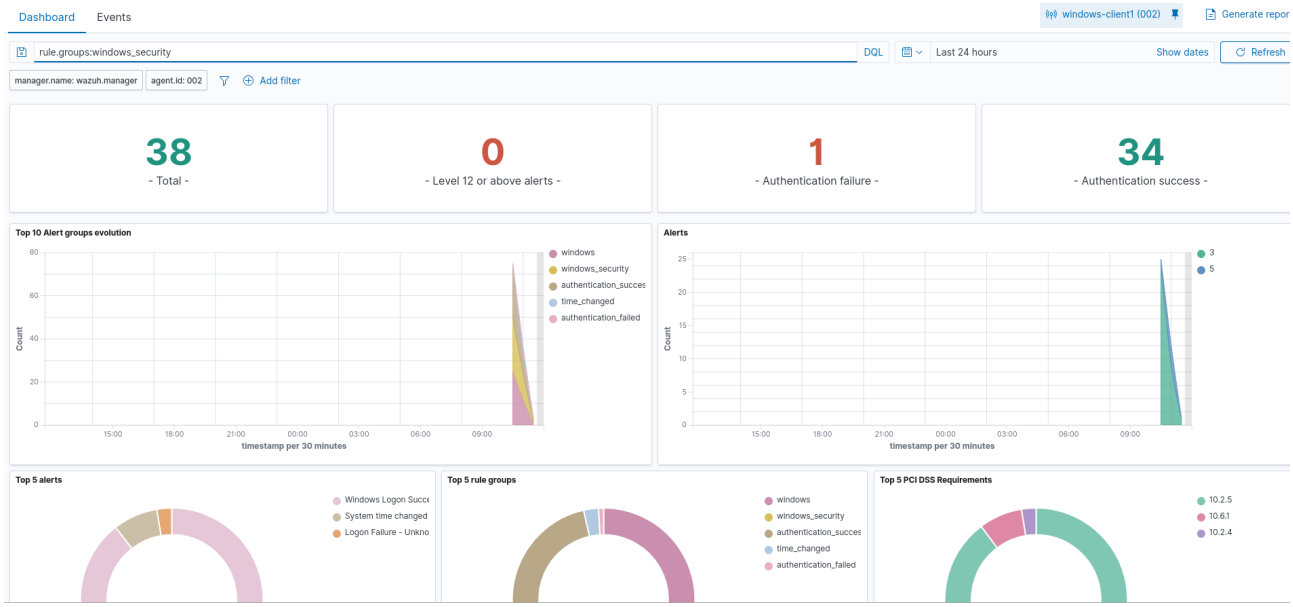
Cette fenêtre montre que Wazuh a pris un événement de windows-client1 et l'a traité avec une **règle dédiée au journal Windows Application**. On a plusieurs preuves directement visibles :

- **Rule ID : 60642**
- **Level : 3**
- fichier de règles : 0585-win-application_rules.xml
- groupes : **windows et windows_application**
- description : **"Software protection service scheduled successfully."**
- événement Windows associé : **EventID 16384**

Validation de la collecte du journal Application

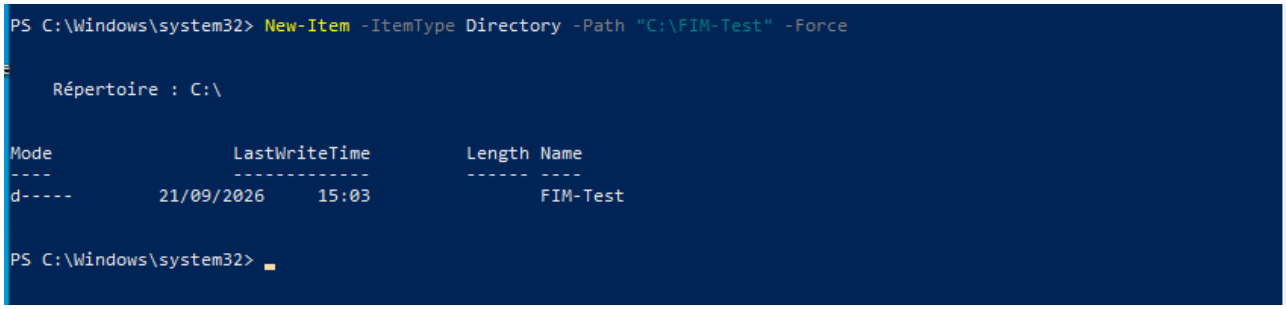
La configuration de l'agent Windows prévoit la collecte du journal Application au format eventchannel. Une recherche dans Threat Hunting avec le filtre rule.groups:windows_application permet d'identifier plusieurs événements provenant de windows-client1. L'examen détaillé d'une alerte montre son rattachement au groupe windows_application et au fichier de règles 0585-win-application_rules.xml, confirmant que les événements du journal Application sont correctement transmis et analysés par Wazuh.



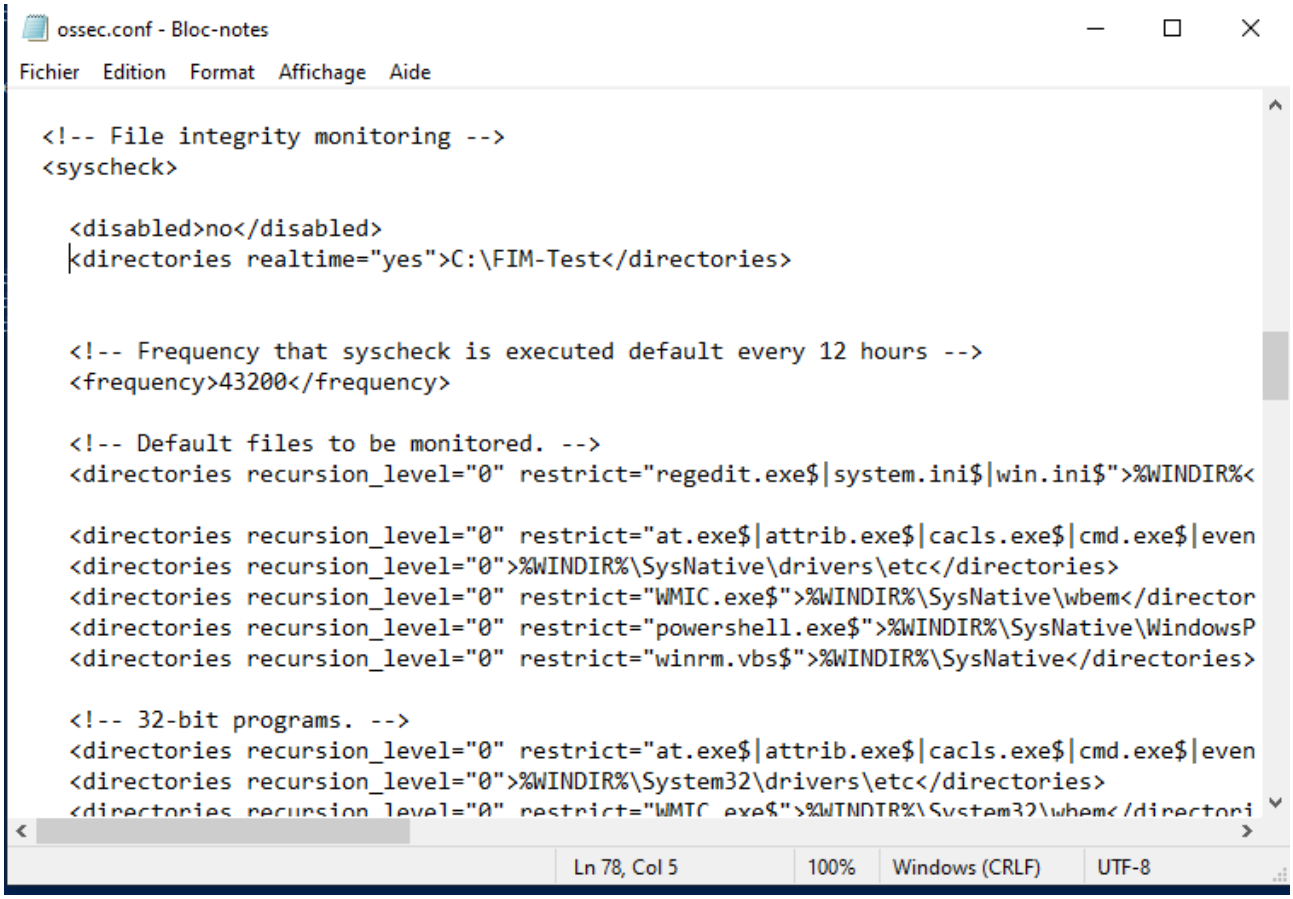


12. FIM : création d'un fichier → modification → suppression → vérification dans Wazuh.

Sur PowerShell de Windows-client1, on tape :
New-Item -ItemType Directory -Path "C:\FIM-Test" -Force



On ouvre le fichier de configuration : notepad "C:\Program Files (x86)\ossec-agent\ossec.conf"



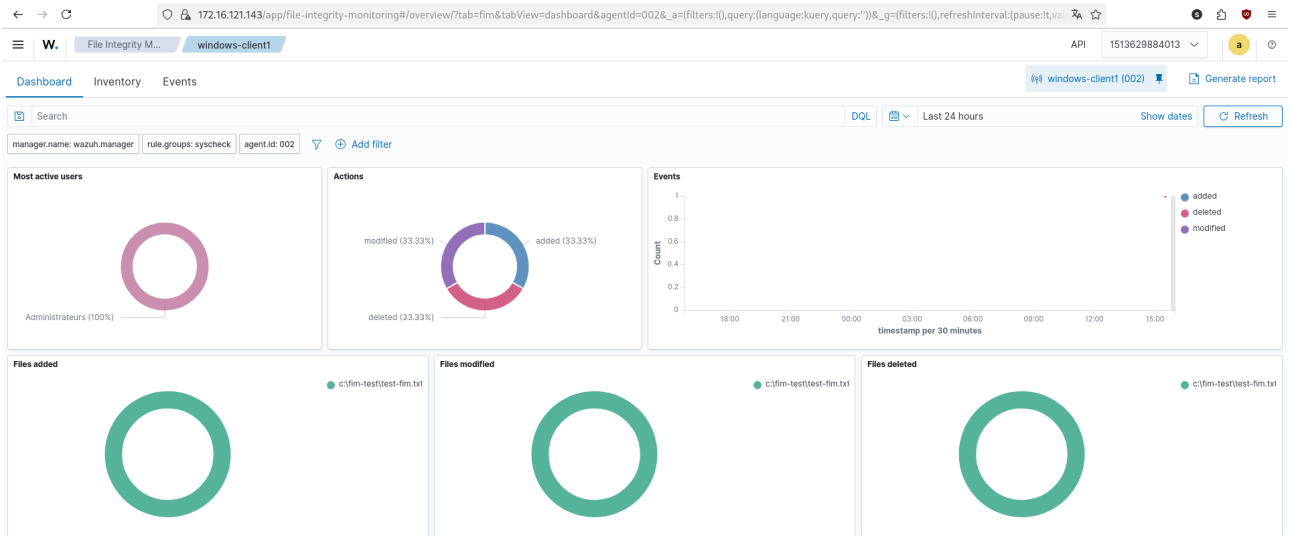
On ajoute la ligne : <directories realtime="yes">C:\FIM-Test</directories> dans le fichier.
On relance le service : Restart-Service wazuhsvc

```
PS C:\Windows\system32> Select-String -Path "C:\Program Files (x86)\ossec-agent\ossec.conf" -Pattern "FIM-Test"

C:\Program Files (x86)\ossec-agent\ossec.conf:78:    <directories realtime="yes">C:\FIM-Test</directories>


PS C:\Windows\system32> "Premier contenu FIM" | Out-File "C:\FIM-Test\test-fim.txt"
PS C:\Windows\system32> "Modification du fichier FIM" | Add-Content "C:\FIM-Test\test-fim.txt"
PS C:\Windows\system32> Remove-Item "C:\FIM-Test\test-fim.txt"
PS C:\Windows\system32>
```

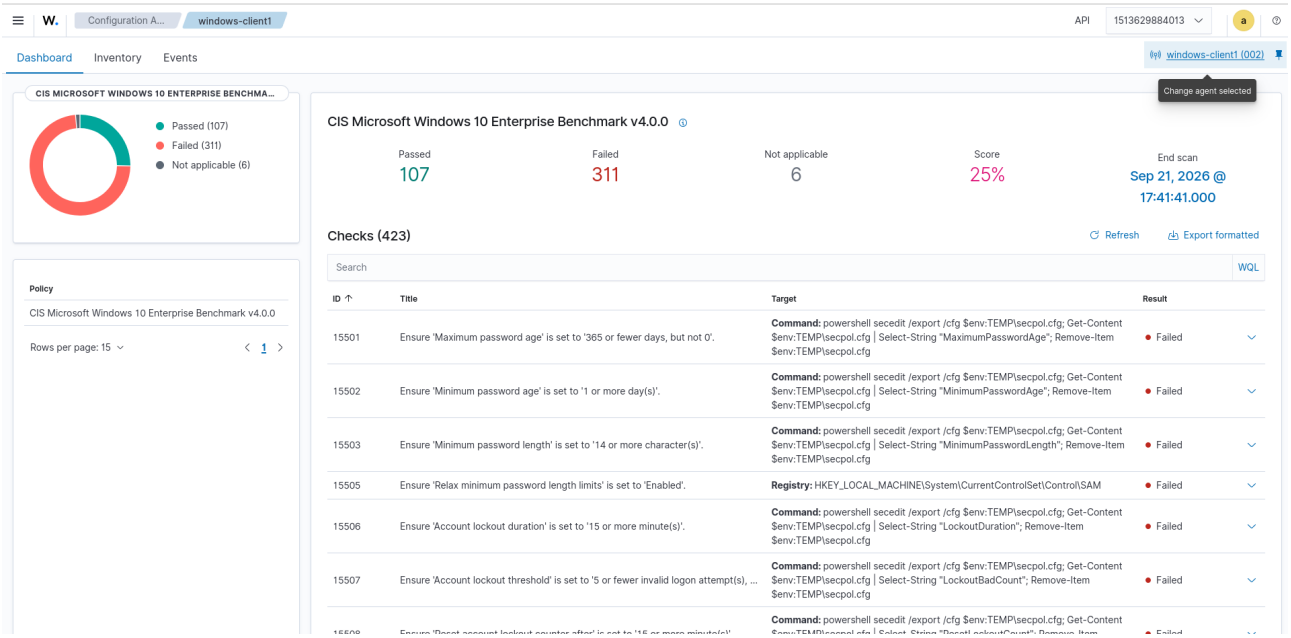
Ce test va provoquer trois types d'événements que FIM sait surveiller : **création, modification et suppression**. Le module FIM compare les attributs et empreintes des fichiers avec sa base locale et génère une alerte lorsqu'il détecte une différence.



La capture FIM montre exactement added / modified / deleted à 33,33 % chacun pour C:\FIM-Test\test-fim.txt. On peut donc considérer le test FIM comme validé.

13. SCA sur Windows, Security Configuration Assessment

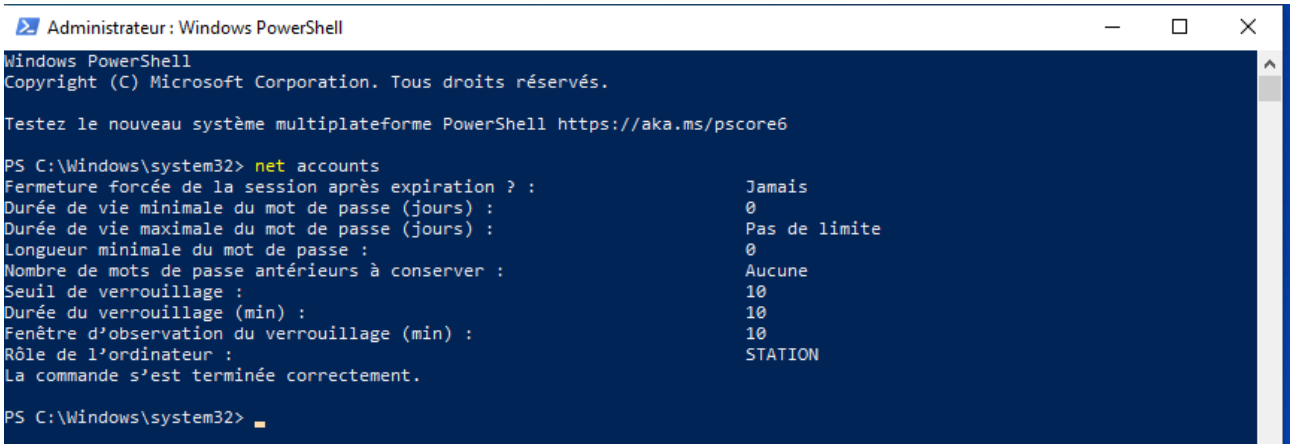
Pour vérifier si la configuration du système est correctement durcie.
On voit la politique CIS Microsoft Windows 10 Enterprise Benchmark v4.0.0 appliquée à windows-client1, avec 423 contrôles : 107 réussis, 311 en échec, 6 non applicables, pour un score affiché de 25 %.



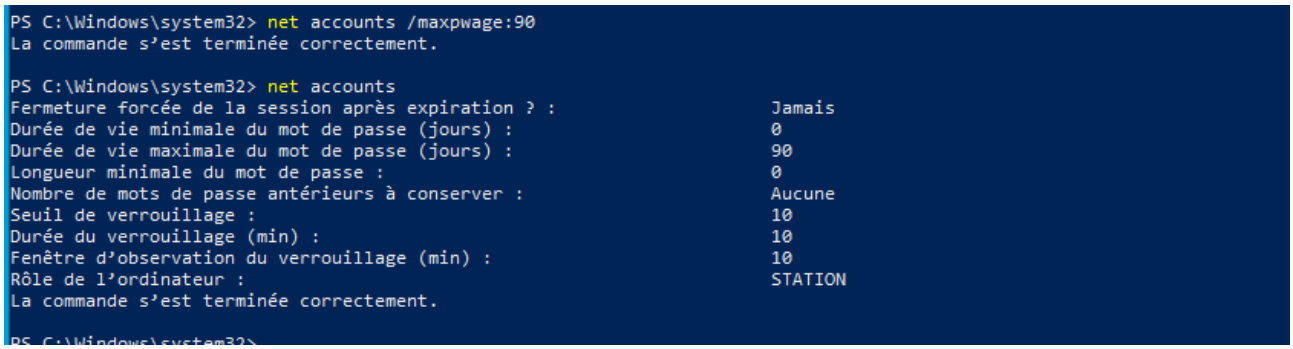
On ne corrige pas les 311 échecs mais le plus simple comme :

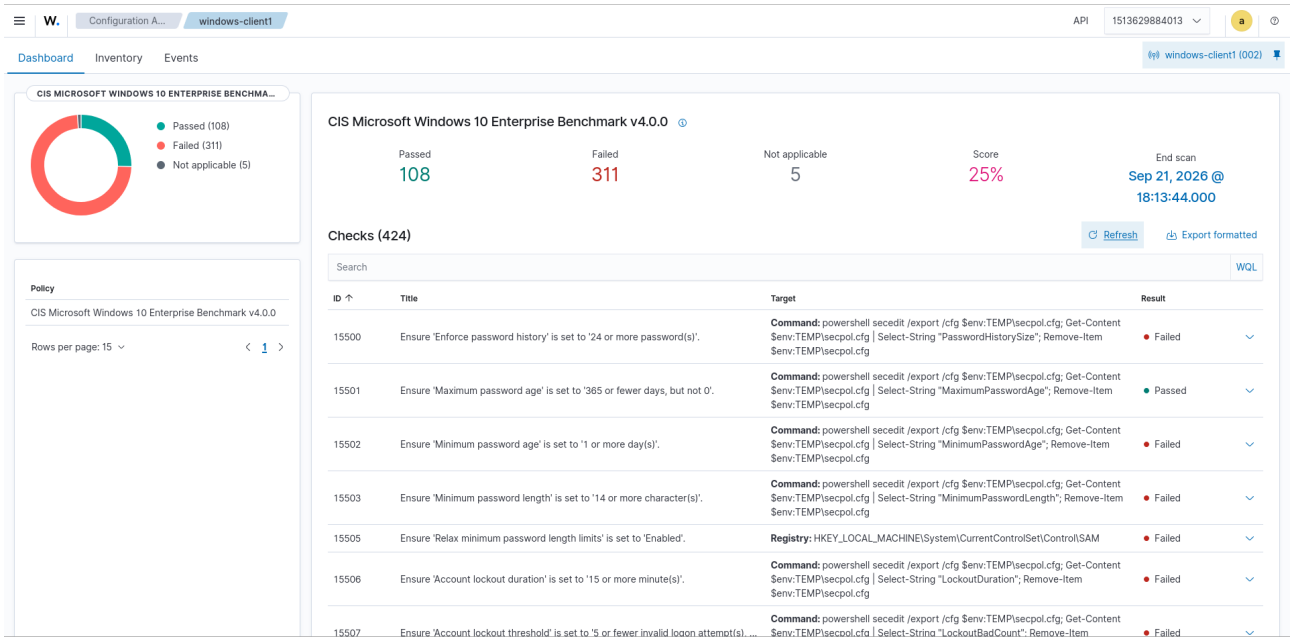
15501 — Ensure “Maximum password age” is set to “365 or fewer days, but not 0”
Vulnerability Detection

Sur PowerShell on lance la commande : net accounts



On applique une valeur de 90 jours : net accounts /maxpwage:90
Puis on vérifie : net accounts



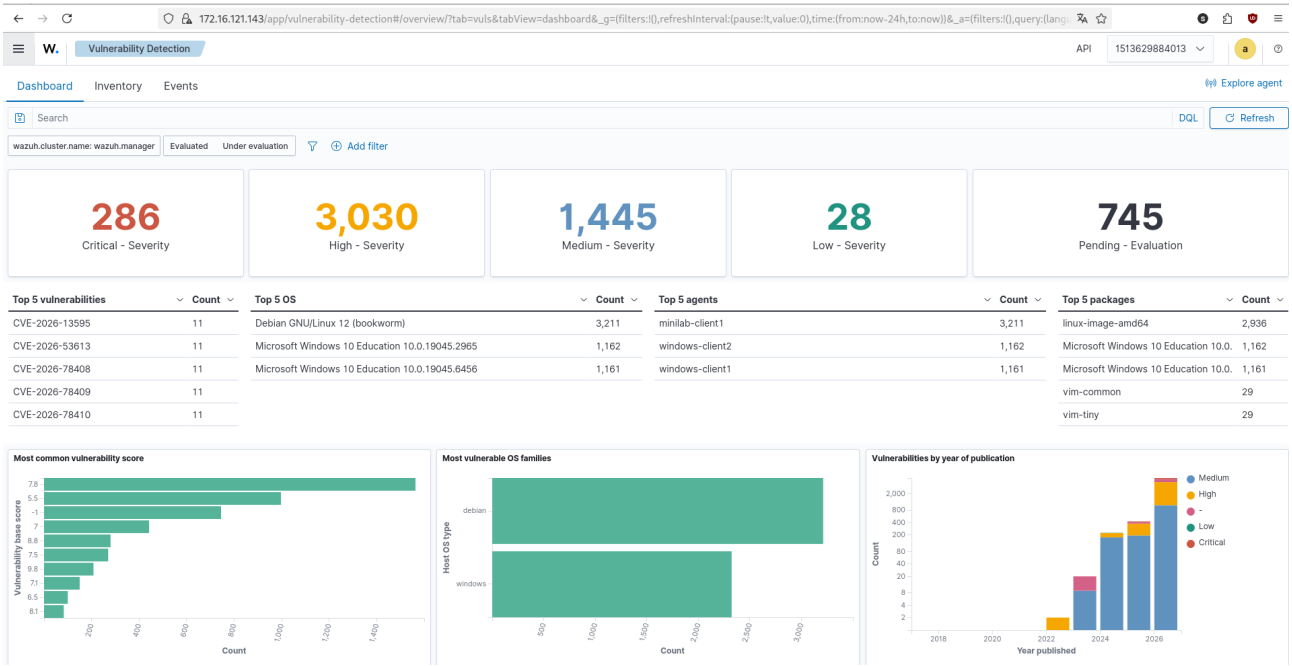


Remédiation SCA Windows

Le contrôle CIS 15501 relatif à la durée maximale de validité des mots de passe était initialement en échec. La politique locale a été modifiée afin de définir une durée maximale de 90 jours. Après redémarrage de l'agent et nouvelle évaluation SCA, le contrôle 15501 est passé à l'état Passed, confirmant la prise en compte de la mesure de durcissement.

14. Vulnerability Detection

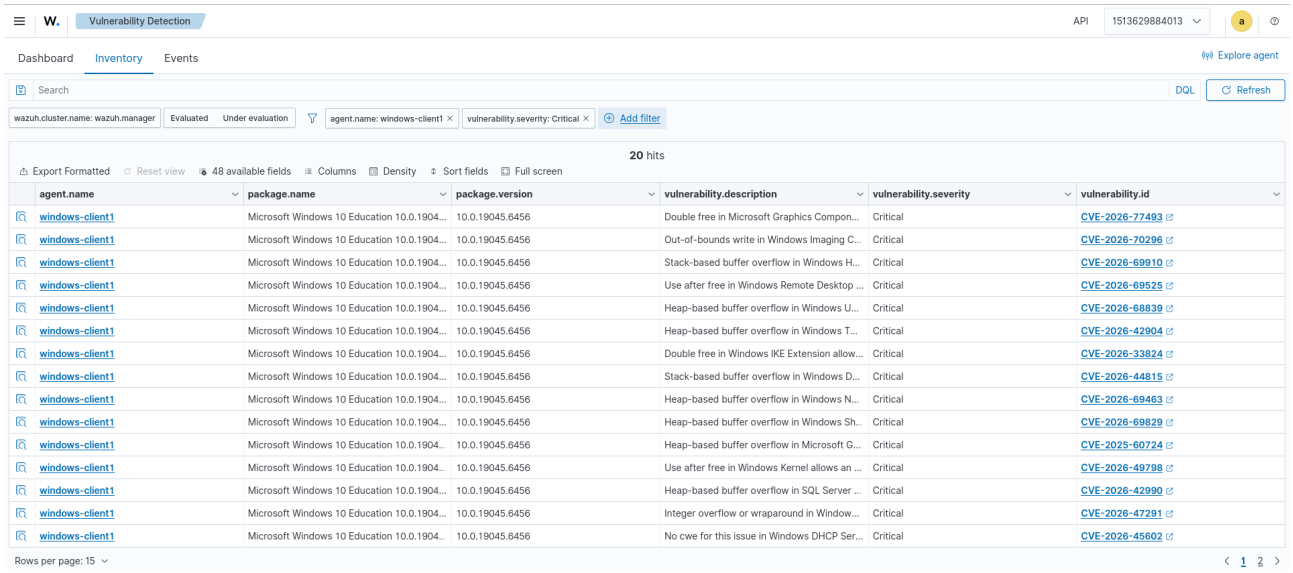
Wazuh récupère l'inventaire logiciel de l'agent via Syscollector, puis le compare avec sa base de vulnérabilités CTI (Cyber Threat Intelligence, renseignement sur les cybermenaces) pour identifier les logiciels/version affectés par des CVE connues.
Dans le Dashboard, on va dans : Threat intelligence → Vulnerability Detection

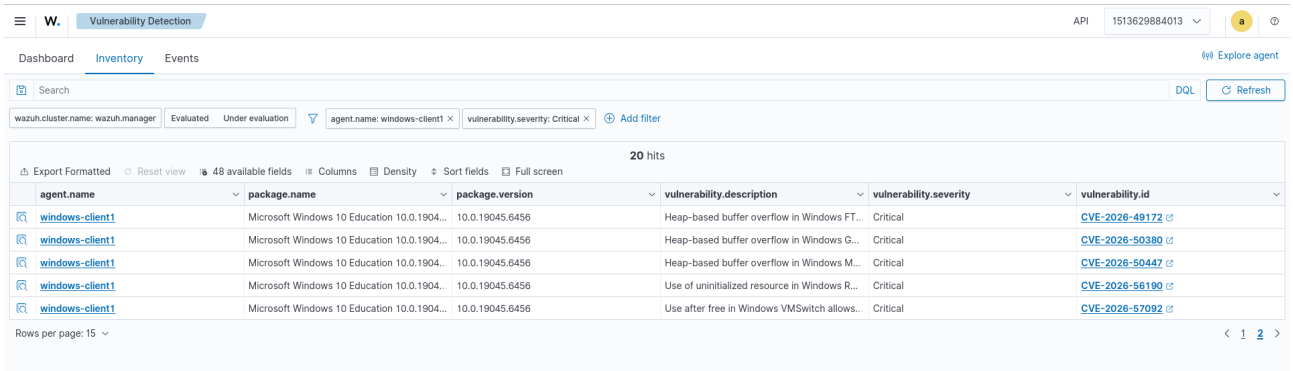


Le Dashboard montre actuellement **5 534 résultats au total** dans l'inventaire, dont **286 Critical, 3 030 High, 1 445 Medium et 28 Low**, avec **745 éléments encore en évaluation**.

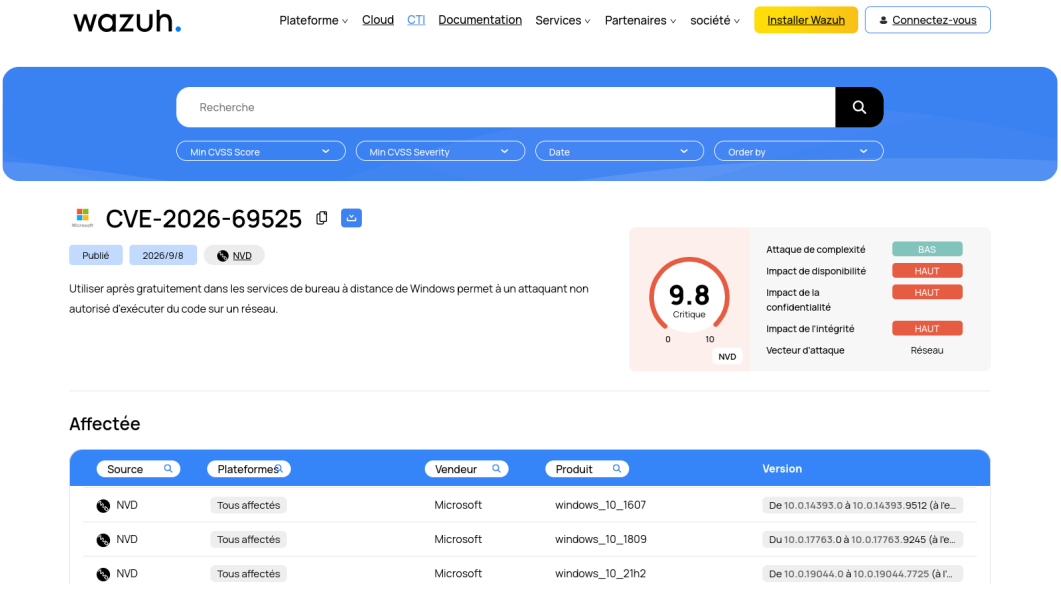
Les résultats concernent les 3 VM, donc on ajoute un filtre pour la VM windows-client1 + Critical.

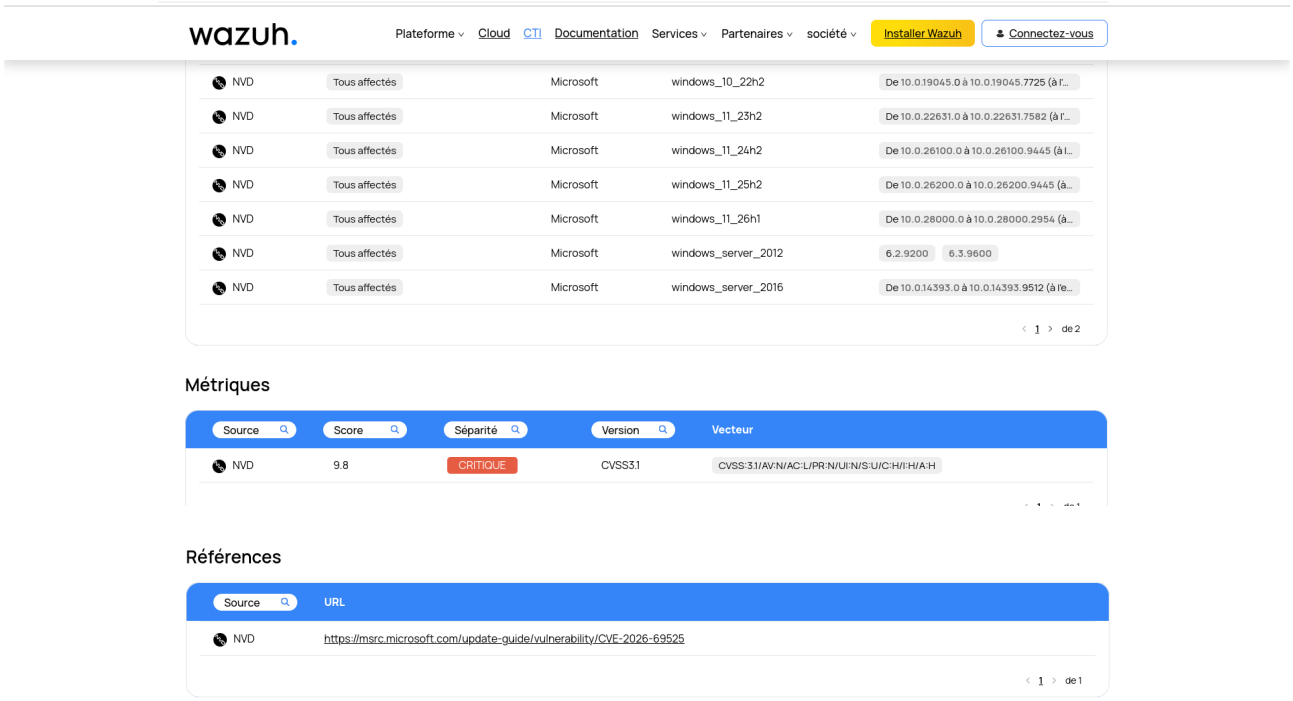
Avec les deux filtres agent.name: windows-client1 et vulnerability.severity: Critical, Wazuh retourne 20 vulnérabilités critiques associées à windows-client1. Elles sont rattachées au composant Windows 10 Education, version affichée 10.0.19045.6456. C'est donc une vraie preuve que Vulnerability Detection est opérationnel sur l'endpoint Windows.





On va choisir une vulnérabilité représentative et l'analyser. On choisit CVE-2026-69525, parce que la description indique un problème de type Use After Free dans Windows Remote Desktop, ça touche un service d'accès distant.



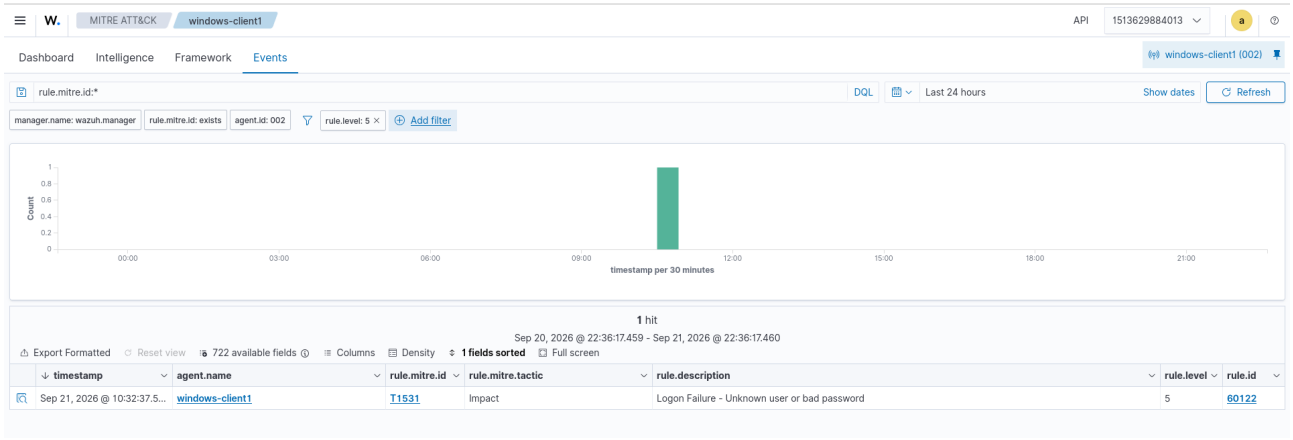


Analyse de la vulnérabilité CVE-2026-69525
Wazuh Vulnerability Detection a identifié sur l'endpoint windows-client1 la vulnérabilité CVE-2026-69525 affectant les services Bureau à distance de Microsoft Windows. La vulnérabilité est classée critique avec un score CVSS v3.1 de 9,8/10.

Il s'agit d'une vulnérabilité de type *Use After Free* susceptible de permettre l'exécution de code à distance. Le vecteur d'attaque est réseau (AV:N), la complexité est faible (AC:L), aucun privilège préalable n'est nécessaire (PR:N) et aucune interaction utilisateur n'est requise (UI:N). Les impacts sur la confidentialité, l'intégrité et la disponibilité sont tous évalués comme élevés.

Cette vulnérabilité doit donc être traitée avec une priorité élevée. Les mesures de remédiation consistent notamment à appliquer les mises à jour de sécurité Microsoft correspondantes et, dans l'attente du correctif, à limiter l'exposition des services Bureau à distance aux seuls systèmes et utilisateurs autorisés.

On utilise **MITRE ATT&CK,** on utilise ce framework pour **catégoriser les techniques d'attaque observées** après l'analyse des alertes.



Filtrage des événements disposant d'une correspondance MITRE ATT&CK dans Wazuh.

Analyse MITRE ATT&CK

Wazuh permet d'associer certaines alertes de sécurité au framework MITRE ATT&CK. Une recherche sur windows-client1 filtrée sur les événements possédant un identifiant MITRE et un niveau de criticité supérieur ou égal à 5 a permis d'identifier une alerte de niveau 5 correspondant à un échec d'authentification (Logon Failure - Unknown user or bad password). La règle Wazuh 60122 associe cet événement à la technique MITRE T1531 et à la tactique Impact. Cette corrélation permet à l'analyste SOC de replacer l'événement dans un référentiel commun de comportements adverses. Un événement isolé ne permet toutefois pas, à lui seul, de conclure à une attaque.

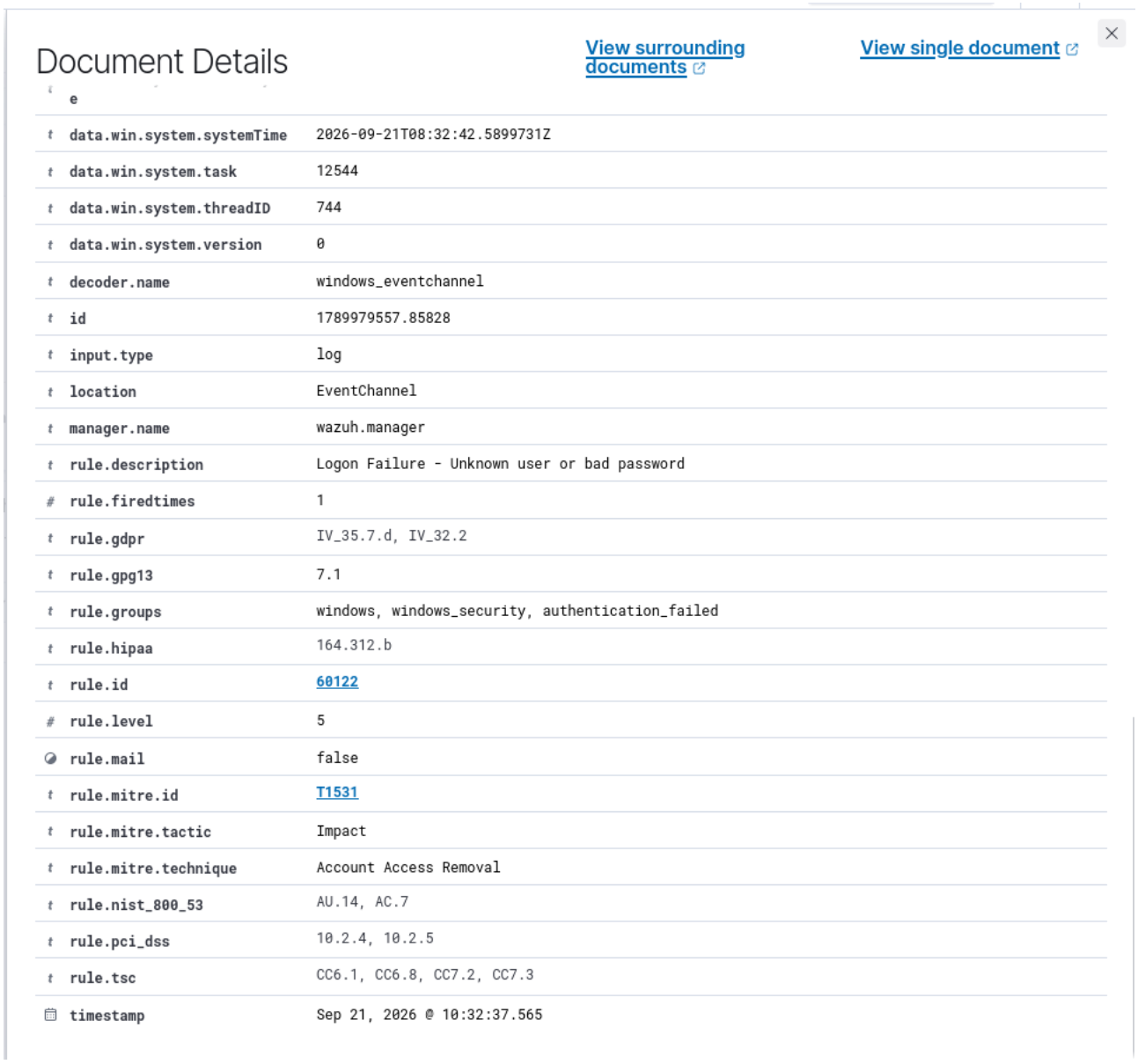
Document Details

View surrounding documents

View single document

Table | JSON

_index	wazuh-alerts-4.x-2026.09.21
agent.id	002
agent.ip	172.16.121.145
agent.name	windows-client1
data.win.eventdata.authenticationPackageName	Negotiate
data.win.eventdata.failureReason	%%2313
data.win.eventdata.keyLength	0
data.win.eventdata.logonProcessName	Advapi
data.win.eventdata.logonType	2
data.win.eventdata.processId	0x18d8
data.win.eventdata.processName	C:\\Program Files (x86)\\Microsoft\\EdgeWebView\\Application\\153.0.4234.46\\msedgewebview2.exe
data.win.eventdata.status	0xc000006e
data.win.eventdata.subStatus	0xc000006e
data.win.eventdata.subjectDomainName	DESKTOP-EHQ529H
data.win.eventdata.subjectLogonId	0x1ddad
data.win.eventdata.subjectUserName	Safia
data.win.eventdata.subjectUserSid	S-1-5-21-1941201515-170622419-332697004-1000
data.win.eventdata.targetDomainName	DESKTOP-EHQ529H
data.win.eventdata.targetUserName	Safia



Document Details

View surrounding documents | View single document

e	
data.win.system.systemTime	2026-09-21T08:32:42.5899731Z
data.win.system.task	12544
data.win.system.threadID	744
data.win.system.version	0
decoder.name	windows_eventchannel
id	1789979557.85828
input.type	log
location	EventChannel
manager.name	wazuh.manager
rule.description	Logon Failure - Unknown user or bad password
rule.firedtimes	1
rule.gdpr	IV_35.7.d, IV_32.2
rule.gpg13	7.1
rule.groups	windows, windows_security, authentication_failed
rule.hipaa	164.312.b
rule.id	60122
rule.level	5
rule.mail	false
rule.mitre.id	T1531
rule.mitre.tactic	Impact
rule.mitre.technique	Account Access Removal
rule.nist_800_53	AU.14, AC.7
rule.pci_dss	10.2.4, 10.2.5
rule.tsc	CC6.1, CC6.8, CC7.2, CC7.3
timestamp	Sep 21, 2026 @ 10:32:37.565

Détail de l’alerte et correspondance avec la technique MITRE ATT&CK T1531.

Analyse d’une alerte MITRE ATT&CK

Wazuh permet de corréler certaines alertes avec le référentiel MITRE ATT&CK.

- Un filtre rule.mitre.id:* permet d’afficher uniquement les événements possédant un mapping MITRE.
- Sur l’agent **windows-client1**, Wazuh a détecté une alerte :
- **ANALYSE D’INCIDENTSANALYSE D’INCIDENTS**rule.id : 60122
- rule.level : 5
- description : **Logon Failure – Unknown user or bad password**
- groupe : authentication_failed
- Cette alerte est associée dans Wazuh à :
- **MITRE ID : T1531**
- **Tactique : Impact**

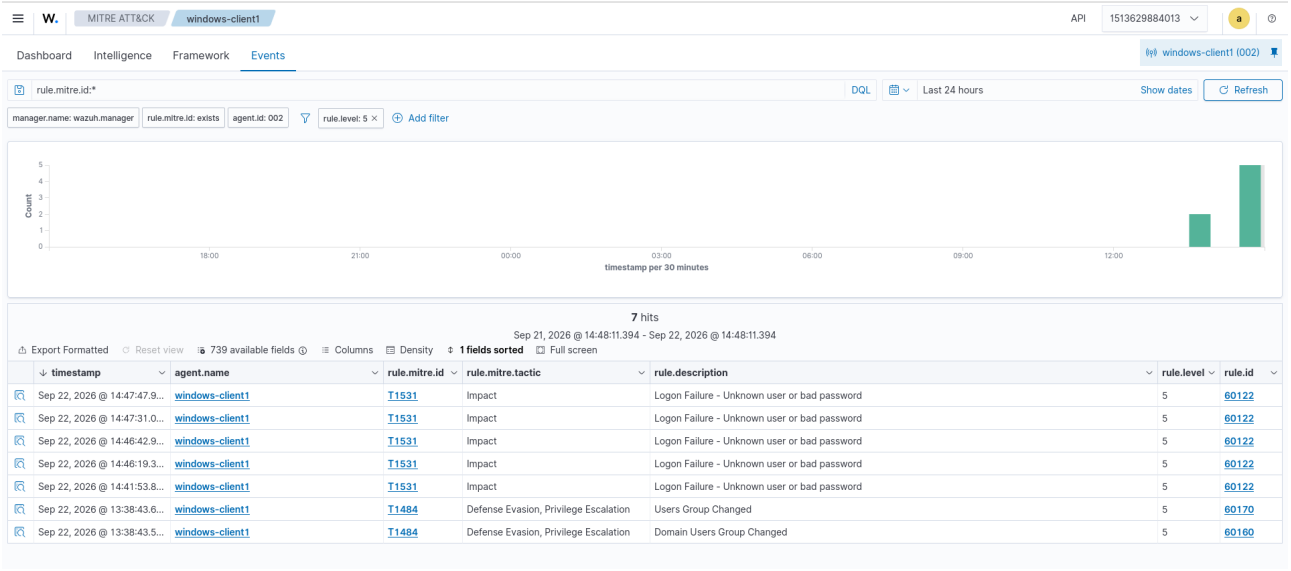
- **Technique : Account Access Removal**
- **MITRE ATT&CK sert à contextualiser et classifier les événements**, mais que cette classification ne suffit pas à elle seule à prouver qu'une attaque correspondant exactement à cette technique a réellement eu lieu. L'analyste doit toujours examiner le contexte.

Wazuh a détecté sur windows-client1 un échec d'authentification associé à la règle 60122, de niveau 5. L'événement est classé dans les groupes windows_security et authentication_failed. La règle Wazuh lui associe la technique MITRE ATT&CK T1531 – Account Access Removal, rattachée à la tactique Impact.
L'analyse détaillée montre toutefois qu'il s'agit d'une tentative de connexion interactive locale isolée. Aucun élément supplémentaire ne permet de conclure à une attaque. L'événement est donc conservé comme exemple de détection et de classification MITRE, mais ne constitue pas à lui seul un incident de sécurité confirmé.

15. ANALYSE D'INCIDENTS

15.1. Échecs répétés d'authentification

On essaie de se connecter avec un faux mot de passe sur la VM windows-client1, puis on observe le retour sur l'interface web de Wazuh afin de voir l'incident d'authentification.



timestamp	agent.name	rule.mitre.id	rule.mitre.tactic	rule.description	rule.level	rule.id
Sep 22, 2026 @ 14:47:47.9...	windows-client1	T1531	Impact	Logon Failure - Unknown user or bad password	5	60122
Sep 22, 2026 @ 14:47:31.0...	windows-client1	T1531	Impact	Logon Failure - Unknown user or bad password	5	60122
Sep 22, 2026 @ 14:46:42.9...	windows-client1	T1531	Impact	Logon Failure - Unknown user or bad password	5	60122
Sep 22, 2026 @ 14:46:19.3...	windows-client1	T1531	Impact	Logon Failure - Unknown user or bad password	5	60122
Sep 22, 2026 @ 14:41:53.8...	windows-client1	T1531	Impact	Logon Failure - Unknown user or bad password	5	60122
Sep 22, 2026 @ 13:38:43.6...	windows-client1	T1484	Defense Evasion, Privilege Escalation	Users Group Changed	5	60170
Sep 22, 2026 @ 13:38:43.5...	windows-client1	T1484	Defense Evasion, Privilege Escalation	Domain Users Group Changed	5	60160

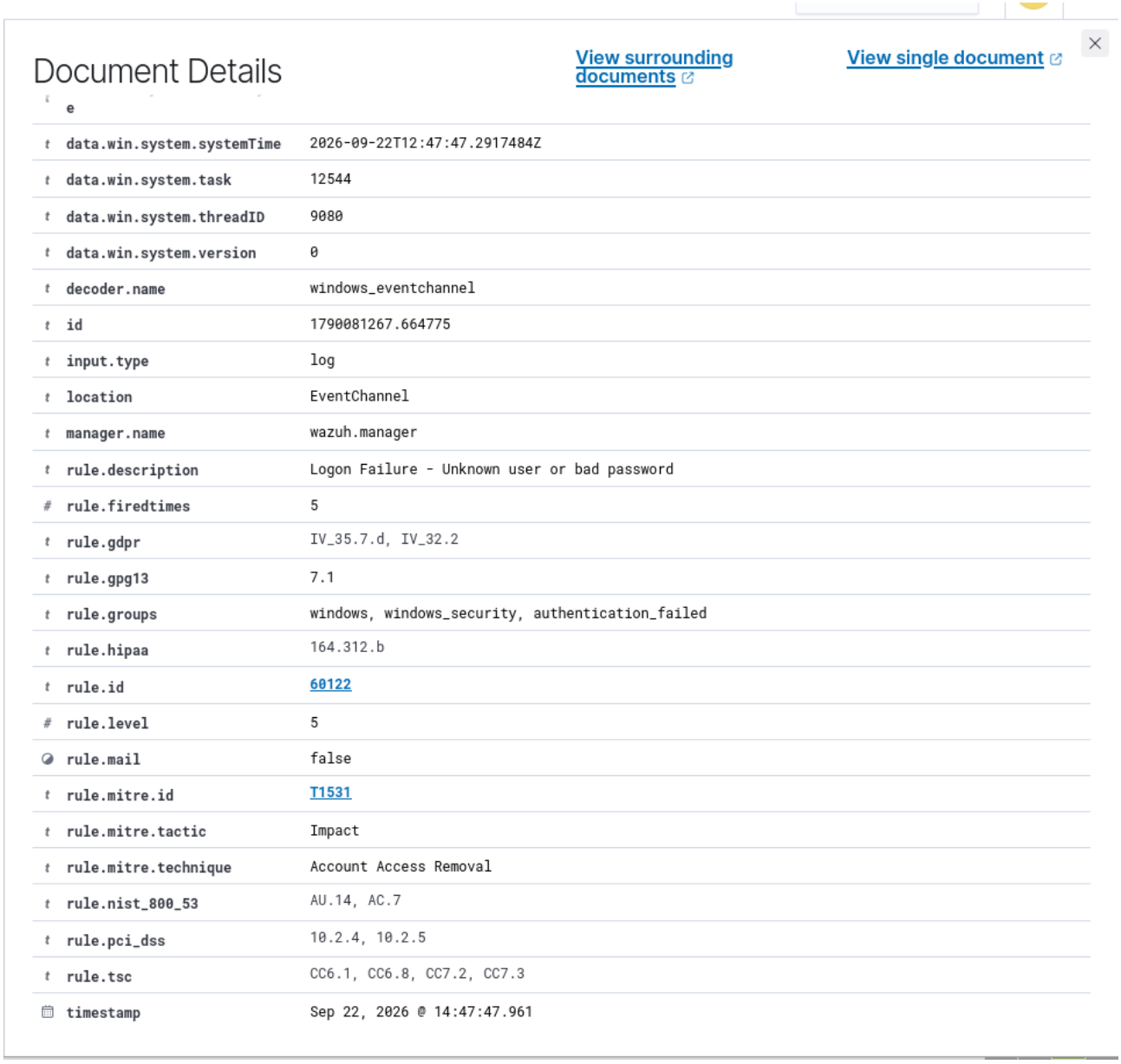
Document Details

View surrounding documents

View single document

Table | JSON

_index	wazuh-alerts-4.x-2026.09.22
agent.id	002
agent.ip	172.16.121.145
agent.name	windows-client1
data.win.eventdata.authenticationPackageName	Negotiate
data.win.eventdata.failureReason	%%2313
data.win.eventdata.ipAddress	::1
data.win.eventdata.ipPort	0
data.win.eventdata.keyLength	0
data.win.eventdata.logonProcessName	seclogo
data.win.eventdata.logonType	2
data.win.eventdata.processId	0x468
data.win.eventdata.processName	C:\\Windows\\System32\\svchost.exe
data.win.eventdata.status	0xc000006d
data.win.eventdata.subStatus	0xc000006a
data.win.eventdata.subjectDomainName	DESKTOP-EHQ529H
data.win.eventdata.subjectLogonId	0x3fd2a
data.win.eventdata.subjectUserName	Safia
data.win.eventdata.subjectUserSid	S-1-5-21-1941201515-170622419-332697004-1000
data.win.eventdata.targetDom	%COMPUTERNAME%



Document Details

View surrounding documents | View single document

data.win.system.systemTime	2026-09-22T12:47:47.2917484Z
data.win.system.task	12544
data.win.system.threadID	9080
data.win.system.version	0
decoder.name	windows_eventchannel
id	1790081267.664775
input.type	log
location	EventChannel
manager.name	wazuh.manager
rule.description	Logon Failure - Unknown user or bad password
rule.firedtimes	5
rule.gdpr	IV_35.7.d, IV_32.2
rule.gpg13	7.1
rule.groups	windows, windows_security, authentication_failed
rule.hipaa	164.312.b
rule.id	60122
rule.level	5
rule.mail	false
rule.mitre.id	T1531
rule.mitre.tactic	Impact
rule.mitre.technique	Account Access Removal
rule.nist_800_53	AU.14, AC.7
rule.pci_dss	10.2.4, 10.2.5
rule.tsc	CC6.1, CC6.8, CC7.2, CC7.3
timestamp	Sep 22, 2026 @ 14:47:47.961

Agent Wazuh : windows-client1

- **IP de l'agent :** 172.16.121.145
- **Source de la tentative :** ::1
- **LogonType :** 2
- **Authentication Package :** Negotiate
- **Processus :** C:\Windows\System32\svchost.exe
- **Status :** 0xc000006d
- **SubStatus :** 0xc000006a
- **Utilisateur à l'origine de l'action :** Safia
- **Règle Wazuh :** 60122
- **Niveau :** 5
- **Description :** Logon Failure - Unknown user or bad password
- **Nombre affiché dans rule.firedtimes :** 5
- **Mapping Wazuh :** T1531 – Account Access Removal
- **Tactique :** Impact

Wazuh a correctement collecté les événements correspondants et déclenché la règle 60122, de niveau 5, décrite comme *Logon Failure – Unknown user or bad password*. Plusieurs événements rapprochés ont ainsi pu être observés dans la console.

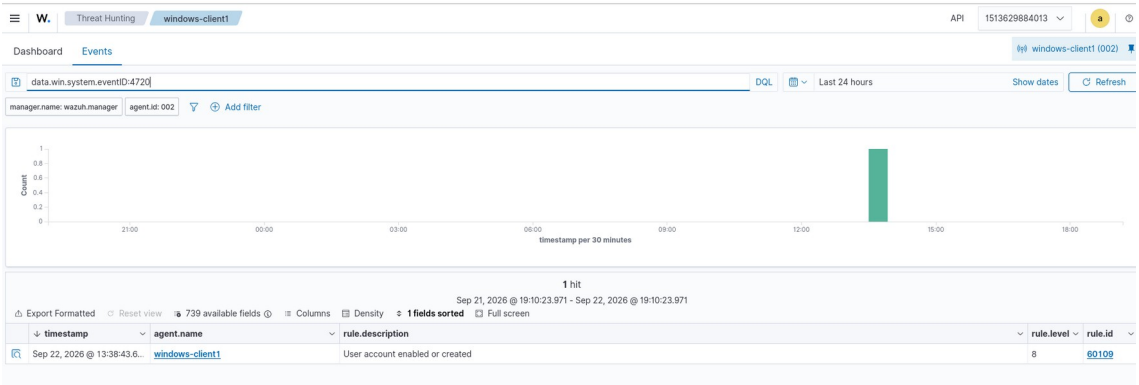
L'analyse détaillée de l'événement indique une adresse source ::1, correspondant à l'interface de bouclage IPv6. Cela confirme que les tentatives ont été effectuées localement sur le poste supervisé. Le type de connexion LogonType 2 correspond à une authentification interactive.

Les valeurs 0xc000006d et 0xc000006a associées au statut de l'événement permettent également de caractériser l'échec d'authentification comme résultant d'identifiants non valides, notamment d'un mot de passe incorrect.

Wazuh associe cette alerte à la technique MITRE ATT&CK T1531 – Account Access Removal, tactique Impact. Cette classification doit toutefois être interprétée avec prudence : les événements observés correspondent ici à des échecs d'authentification provoqués volontairement et ne constituent pas, à eux seuls, la preuve d'une action d'Account Access Removal.

15.2. Incident contôlé n°2 : création d'un compte utilisateur.

Ici, nous savons qu'il s'agit d'un test contrôlé, Le compte concerné est bien soc-test, et l'opération a été réalisée depuis le compte Safia sur DESKTOP-EHQ529H.



Wazuh a ensuite transformé cet événement Windows en alerte avec les éléments suivants :

- **Event ID Windows :** 4720
- **Compte créé :** soc-test
- **Compte ayant effectué l'action :** Safia
- **Agent :** windows-client1
- **IP de l'agent :** 172.16.121.145
- **Règle Wazuh :** 60109
- **Niveau :** 8
- **Description :** User account enabled or created
- **Groupes :** windows, windows_security, adduser, account_changed

- **MITRE ATT&CK :** T1098
- **Technique :** Account Manipulation
- **Tactique :** Persistence
- **Heure affichée par Wazuh :** 22 septembre 2026 à **13:38:43**

La création du compte a généré l'événement Windows **4720**, correspondant à la création d'un compte utilisateur. Wazuh a correctement collecté cet événement et déclenché la règle **60109**, de niveau **8**, avec la description *User account enabled or created*.

L'analyse détaillée de l'alerte permet d'identifier le compte créé (soc-test) ainsi que le compte ayant effectué l'opération (Safia). L'événement appartient notamment aux groupes adduser et account_changed.

Wazuh associe cet événement à la technique MITRE ATT&CK **T1098 – Account Manipulation**, relevant de la tactique **Persistence**. Dans un environnement réel, la création non autorisée d'un compte peut constituer un mécanisme permettant à un attaquant de maintenir un accès persistant au système.

Dans le cadre de cette expérimentation, l'action était volontaire et contrôlée. L'alerte démontre néanmoins la capacité du SIEM à identifier et contextualiser une modification sensible des comptes Windows.

15.3. Incident contrôlé n°3 : FIM (File Integrity Monitoring)

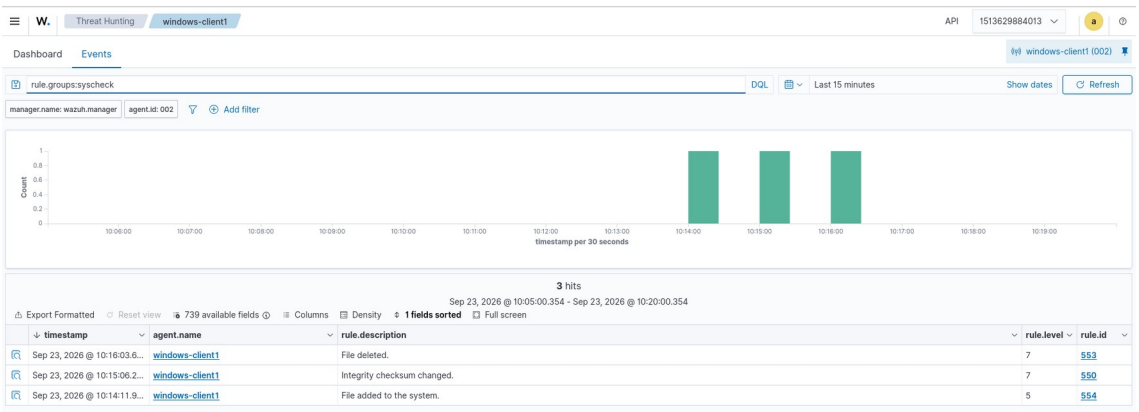
Étape 1 — créer un répertoire de test

Sur windows-client1, on ouvre PowerShell en administrateur et on lance la commande :

New-Item -Path C:\fim-test -ItemType Directory

On crée le fichier, puis on le supprime. On observe ensuite ce que Wazuh détecte.

```
PS C:\Windows\system32> Set-Content C:\fim-test\incident-test.txt "Version initiale du fichier FIM"
PS C:\Windows\system32> Add-Content C:\fim-test\incident-test.txt "Modification contrôlée pour le test SOC"
PS C:\Windows\system32> Remove-Item C:\fim-test\incident-test.txt
PS C:\Windows\system32>
```



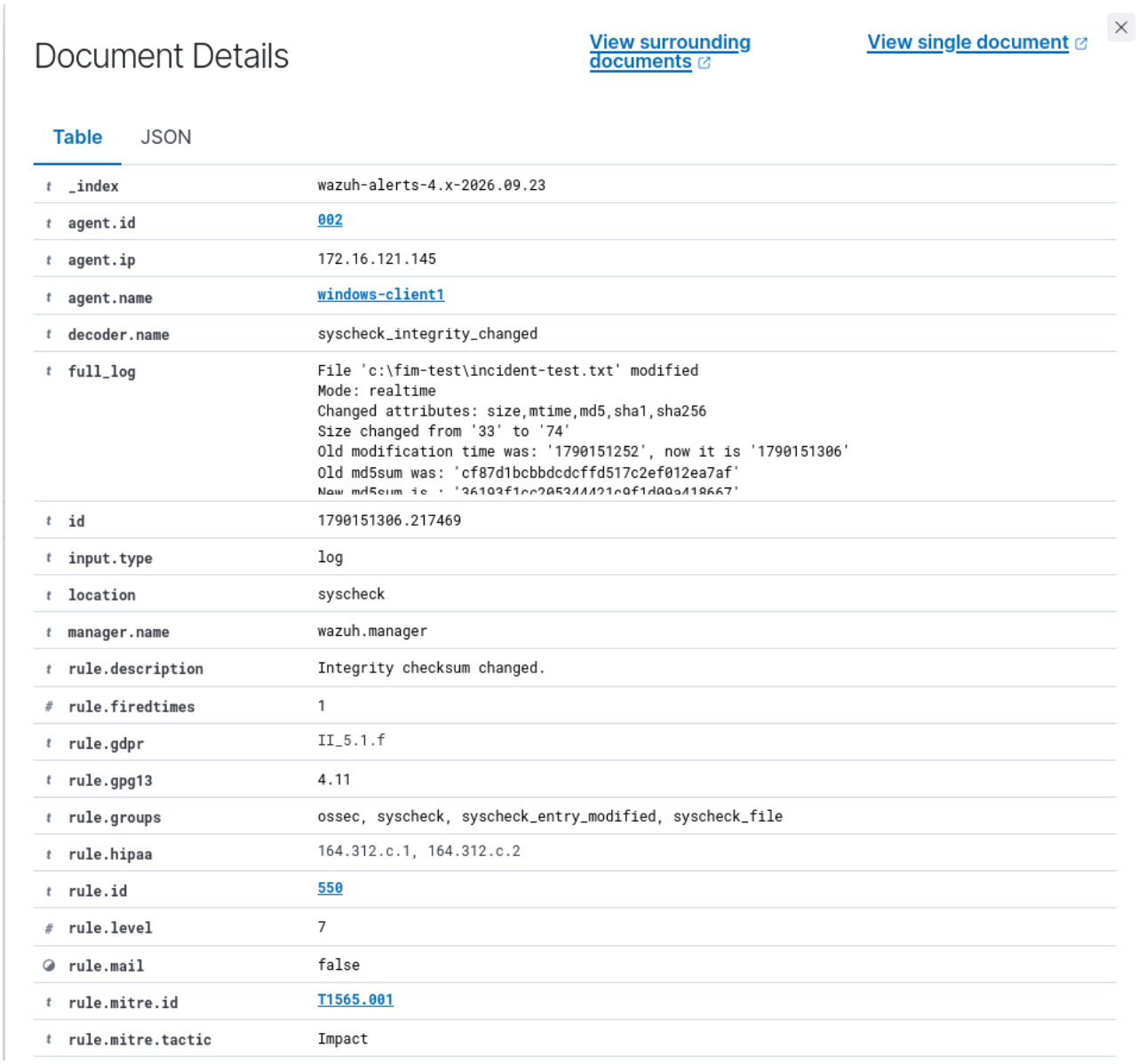
Wazuh a détecté les **trois étapes de notre scénario contrôlé** :

Heure	Événement Wazuh	Règle	Niveau
~10:14:11	File added to the system.	**554**	**5**
~10:15:06	Integrity checksum changed.	**550**	**7**
~10:16:03	File deleted.	**553**	**7**

Cela correspond exactement à ce que nous avons fait :

création du fichier → modification du contenu → suppression du fichier.

C’est une démonstration très claire du **File Integrity Monitoring** : Wazuh ne se contente pas de voir qu’un fichier existe, il détecte aussi qu’un fichier surveillé a changé puis qu’il a disparu.



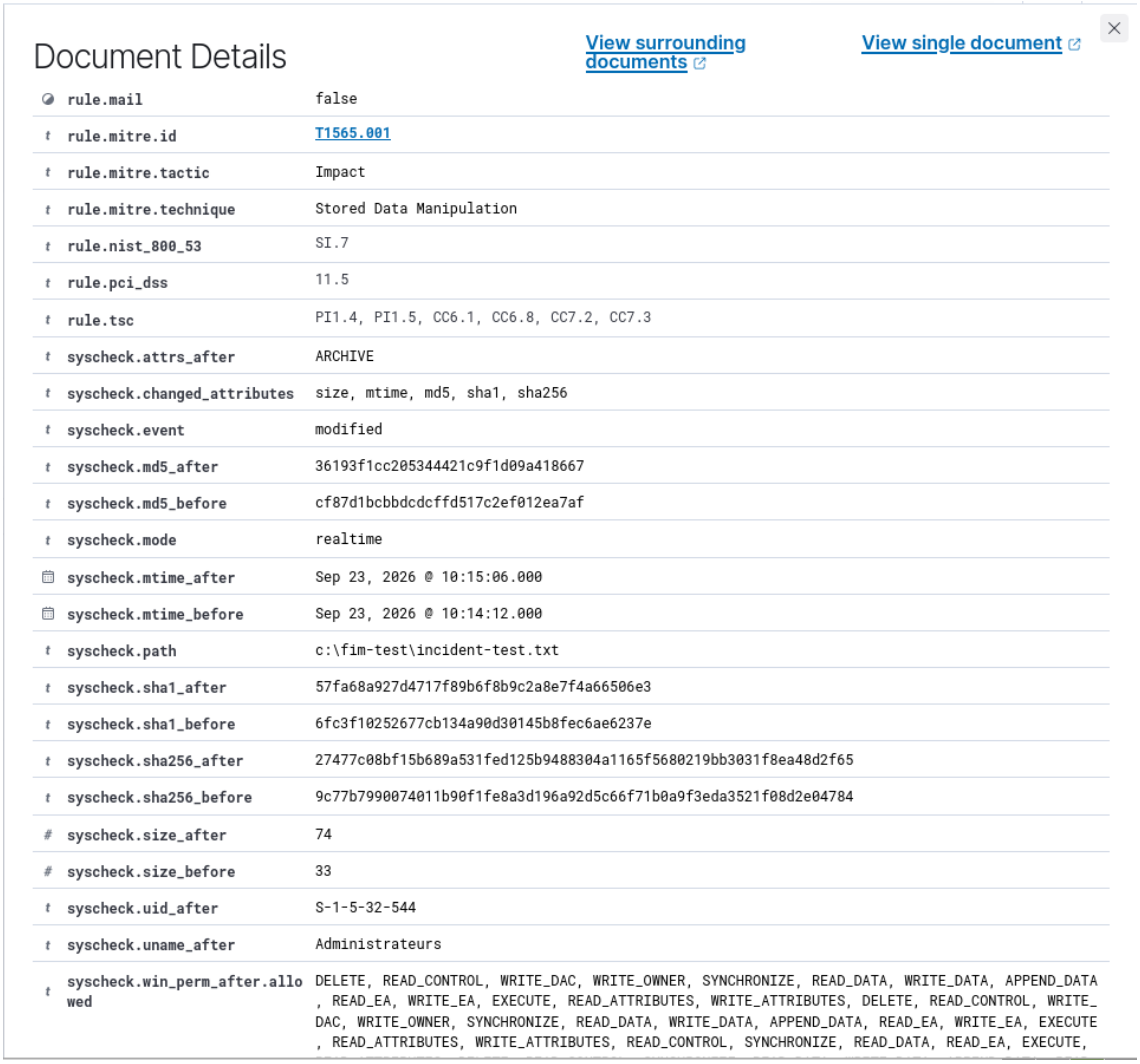
Document Details

View surrounding documents

View single document

Table JSON

t _index	wazuh-alerts-4.x-2026.09.23
t agent.id	002
t agent.ip	172.16.121.145
t agent.name	windows-client1
t decoder.name	syscheck_integrity_changed
t full_log	File 'c:\fim-test\incident-test.txt' modified Mode: realtime Changed attributes: size,mtime,md5,sha1,sha256 Size changed from '33' to '74' Old modification time was: '1790151252', now it is '1790151306' Old md5sum was: 'cf87d1bcbbdcdcffd517c2ef012ea7af' New md5sum is : '36193f1cc205344421c9f1d09a418667'
t id	1790151306.217469
t input.type	log
t location	syscheck
t manager.name	wazuh.manager
t rule.description	Integrity checksum changed.
# rule.firedtimes	1
t rule.gdpr	II_5.1.f
t rule.gpg13	4.11
t rule.groups	ossec, syscheck, syscheck_entry_modified, syscheck_file
t rule.hipaa	164.312.c.1, 164.312.c.2
t rule.id	550
# rule.level	7
rule.mail	false
t rule.mitre.id	T1565.001
t rule.mitre.tactic	Impact



Document Details

View surrounding documents | View single document

rule.mail	false
rule.mitre.id	T1565.001
rule.mitre.tactic	Impact
rule.mitre.technique	Stored Data Manipulation
rule.nist_800_53	SI.7
rule.pci_dss	11.5
rule.tsc	PI1.4, PI1.5, CC6.1, CC6.8, CC7.2, CC7.3
syscheck.attrs_after	ARCHIVE
syscheck.changed_attributes	size, mtime, md5, sha1, sha256
syscheck.event	modified
syscheck.md5_after	36193f1cc205344421c9f1d09a418667
syscheck.md5_before	cf87d1bcbbdcdcffd517c2ef012ea7af
syscheck.mode	realtime
syscheck.mtime_after	Sep 23, 2026 @ 10:15:06.000
syscheck.mtime_before	Sep 23, 2026 @ 10:14:12.000
syscheck.path	c:\fim-test\incident-test.txt
syscheck.sha1_after	57fa68a927d4717f89b6f8b9c2a8e7f4a66506e3
syscheck.sha1_before	6fc3f10252677cb134a90d30145b8fec6ae6237e
syscheck.sha256_after	27477c08bf15b689a531fed125b9488304a1165f5680219bb3031f8ea48d2f65
syscheck.sha256_before	9c77b7990074011b90f1fe8a3d196a92d5c66f71b0a9f3eda3521f08d2e04784
syscheck.size_after	74
syscheck.size_before	33
syscheck.uid_after	S-1-5-32-544
syscheck.uname_after	Administrateurs
syscheck.win_perm_after.allowed	DELETE, READ_CONTROL, WRITE_DAC, WRITE_OWNER, SYNCHRONIZE, READ_DATA, WRITE_DATA, APPEND_DATA, READ_EA, WRITE_EA, EXECUTE, READ_ATTRIBUTES, WRITE_ATTRIBUTES, DELETE, READ_CONTROL, WRITE_DAC, WRITE_OWNER, SYNCHRONIZE, READ_DATA, WRITE_DATA, APPEND_DATA, READ_EA, WRITE_EA, EXECUTE, READ_ATTRIBUTES, WRITE_ATTRIBUTES, READ_CONTROL, SYNCHRONIZE, READ_DATA, READ_EA, EXECUTE,

Wazuh montre clairement que le fichier C:\fim-test\incident-test.txt a été **modifié en temps réel** :

- syscheck.event = modified
- syscheck.mode = realtime
- taille passée de **33 à 74 octets**
- mtime modifié
- **MD5, SHA-1 et SHA-256 avant/après différents**
- règle Wazuh **550**
- niveau **7**
- description : Integrity checksum changed
- mapping MITRE : **T1565.001 – Stored Data Manipulation**
- tactique : **Impact**

Incident contrôlé n°3 – Surveillance de l'intégrité d'un fichier

Afin de tester la fonctionnalité FIM (*File Integrity Monitoring*) de Wazuh, un fichier de test a été créé dans le répertoire surveillé C:\fim-test, puis modifié et supprimé.

Wazuh a détecté successivement la création du fichier, sa modification puis sa suppression. La modification a déclenché la règle **550**, de niveau **7**, avec la description *Integrity checksum changed*.

L’analyse de l’événement montre que la taille du fichier est passée de **33 à 74 octets** et que ses
Incident contrôlé n°3 – Surveillance de l’intégrité d’un fichier

Afin de tester la fonctionnalité FIM (*File Integrity Monitoring*) de Wazuh, un fichier de test a été créé dans le répertoire surveillé C:\fim-test, puis modifié et supprimé.

Wazuh a détecté successivement la création du fichier, sa modification puis sa suppression. La modification a déclenché la règle **550**, de niveau **7**, avec la description *Integrity checksum changed*.

Les empreintes cryptographiques MD5, SHA-1 et SHA-256 ont été modifiées. Ces différences confirment une altération effective du contenu du fichier.

La surveillance était réalisée en mode realtime, permettant à Wazuh de détecter rapidement les modifications effectuées sur le répertoire supervisé.

Wazuh associe cet événement à la technique MITRE ATT&CK **T1565.001 – Stored Data Manipulation**, relevant de la tactique **Impact**. Dans un environnement réel, ce type d’alerte pourrait permettre d’identifier une modification non autorisée d’un fichier sensible.

Dans le cadre de ce test, les modifications étaient volontaires et contrôlées ; elles permettent néanmoins de démontrer la capacité du FIM à détecter des altérations d’intégrité.

16. Élasticsearch

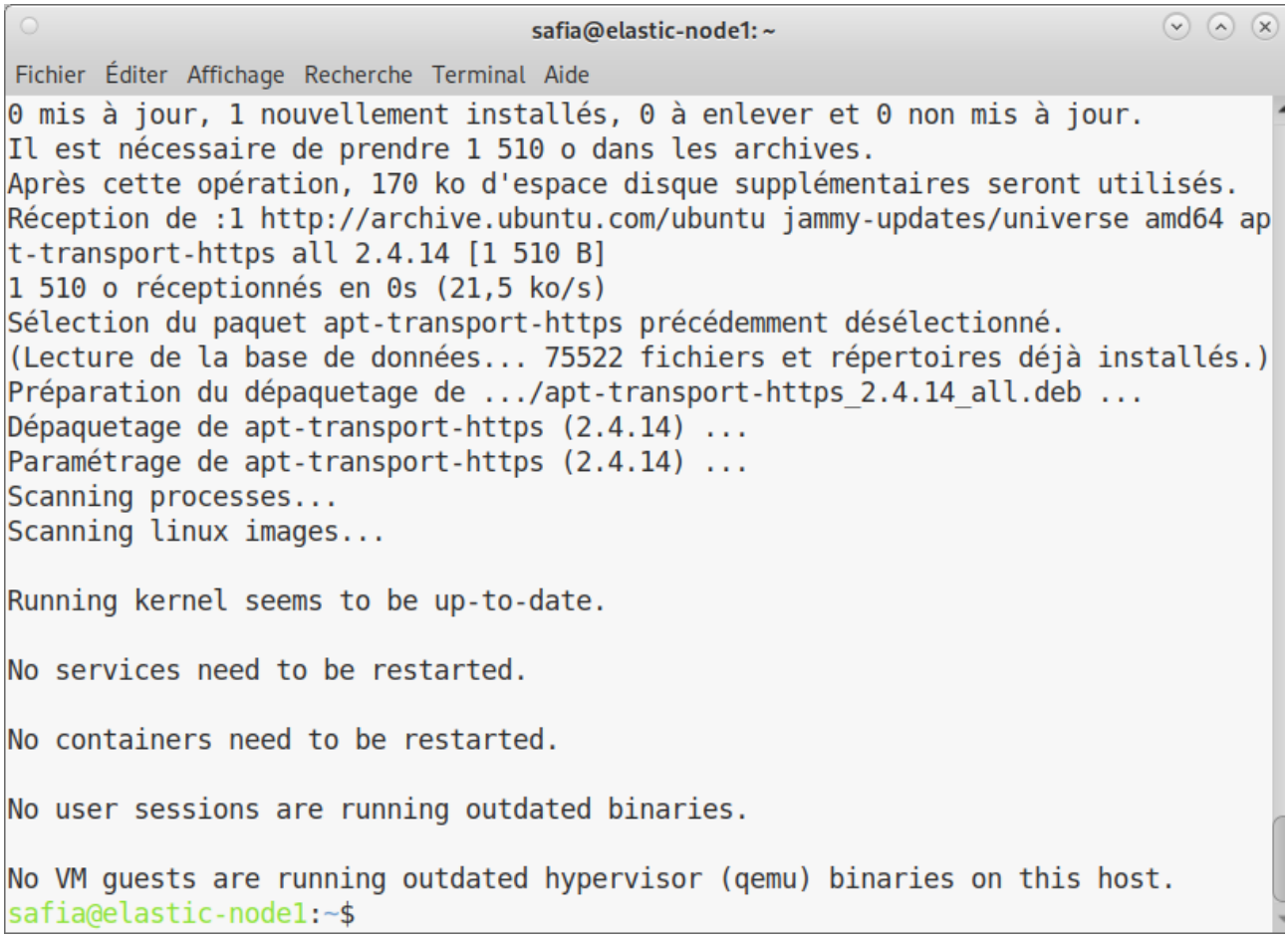
16.1 Installation d’Élasticsearch

On crée deux VM, elastic-node1 et elastic-node2, pour installer Élasticsearch 9.5.4, la version stable proposée par le dépôt officiel. L’installation se fait d’abord avec elastic-node1, qui rejoindra le cluster que l’on va créer avec elastic-node2.

16.1.1. Sur elastic-node1 :

Pour installer les outils nécessaires et pour préparer le dépôt Elastic, on exécute la commande :

sudo apt install -y wget gnupg apt-transport-https



Puis on importe la clé officielle Elastic avec la commande :

wget -qO - https://artifacts.elastic.co/GPG-KEY-elasticsearch \
| sudo gpg --dearmor -o /usr/share/keyrings/elasticsearch-keyring.gpg

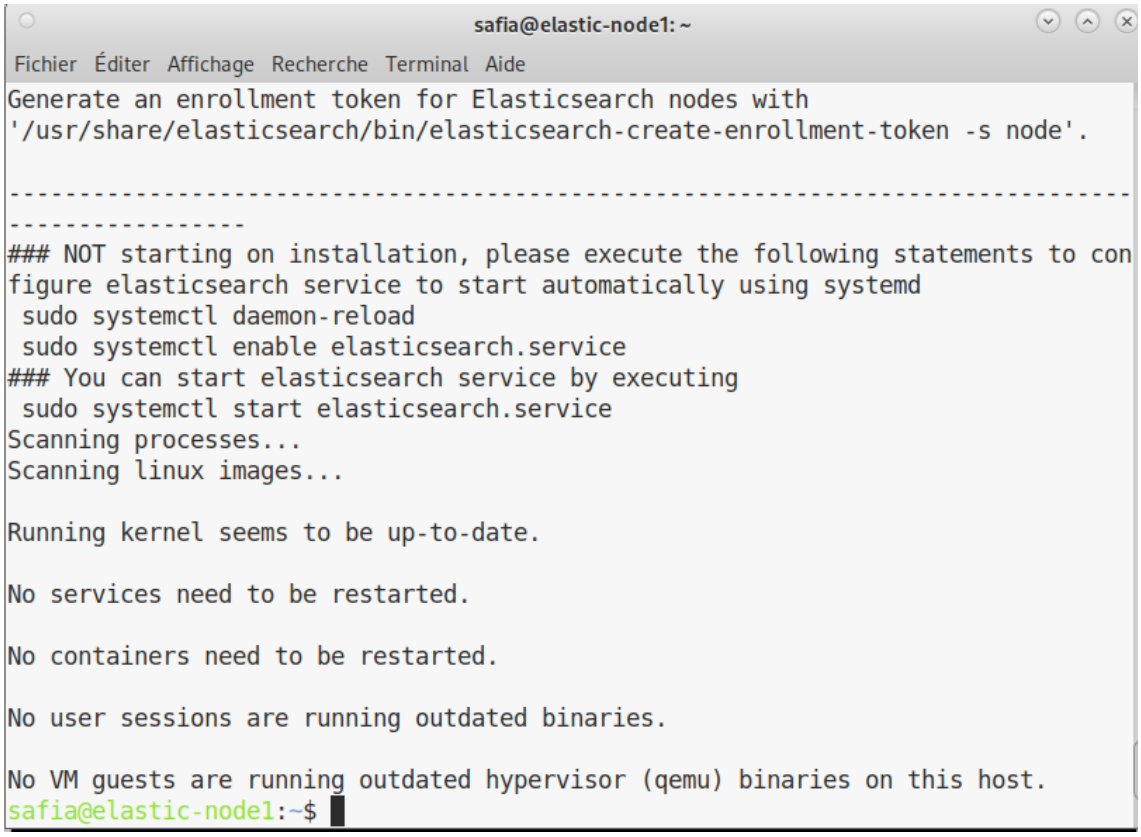
Et on l'ajoute à Elasticsearch :
echo "deb [signed-by=/usr/share/keyrings/elasticsearch-keyring.gpg]
https://artifacts.elastic.co/packages/9.x/apt stable main" \

| sudo tee /etc/apt/sources.list.d/elastic-9.x.list

```
safia@elastic-node1:~$ wget -qO - https://artifacts.elastic.co/GPG-KEY-elasticse
arch \
| sudo gpg --dearmor -o /usr/share/keyrings/elasticsearch-keyring.gpg
[sudo] password for safia:
safia@elastic-node1:~$ echo "deb [signed-by=/usr/share/keyrings/elasticsearch-ke
yring.gpg] https://artifacts.elastic.co/packages/9.x/apt stable main" \
| sudo tee /etc/apt/sources.list.d/elastic-9.x.list
deb [signed-by=/usr/share/keyrings/elasticsearch-keyring.gpg] https://artifacts.
elastic.co/packages/9.x/apt stable main
```

On installe ensuite Élasticsearch avec la commande :

sudo apt install elasticsearch



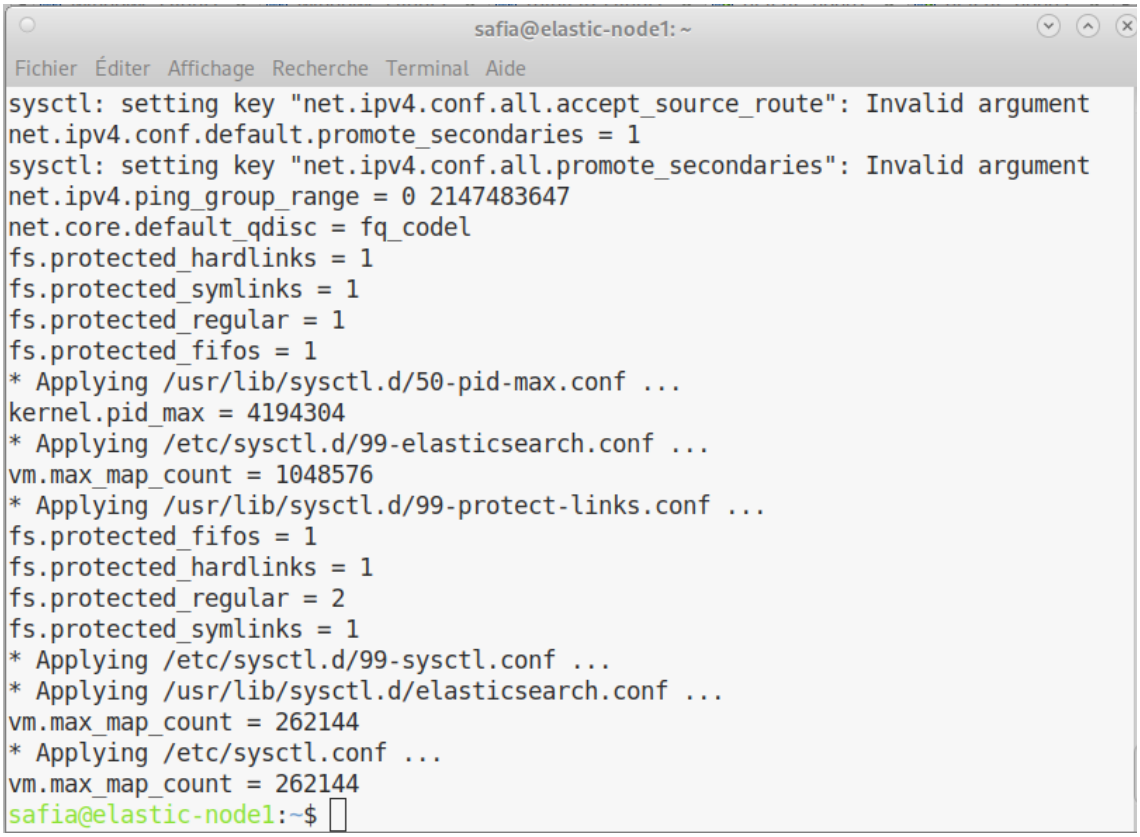
On doit d'abord configurer le premier nœud pour qu'il puisse former un cluster multi-nœuds.

D'abord, on prépare le noyau avec la commande :

echo 'vm.max_map_count=1048576' | sudo tee /etc/sysctl.d/99-elasticsearch.conf

sudo sysctl --system

sysctl vm.max_map_count



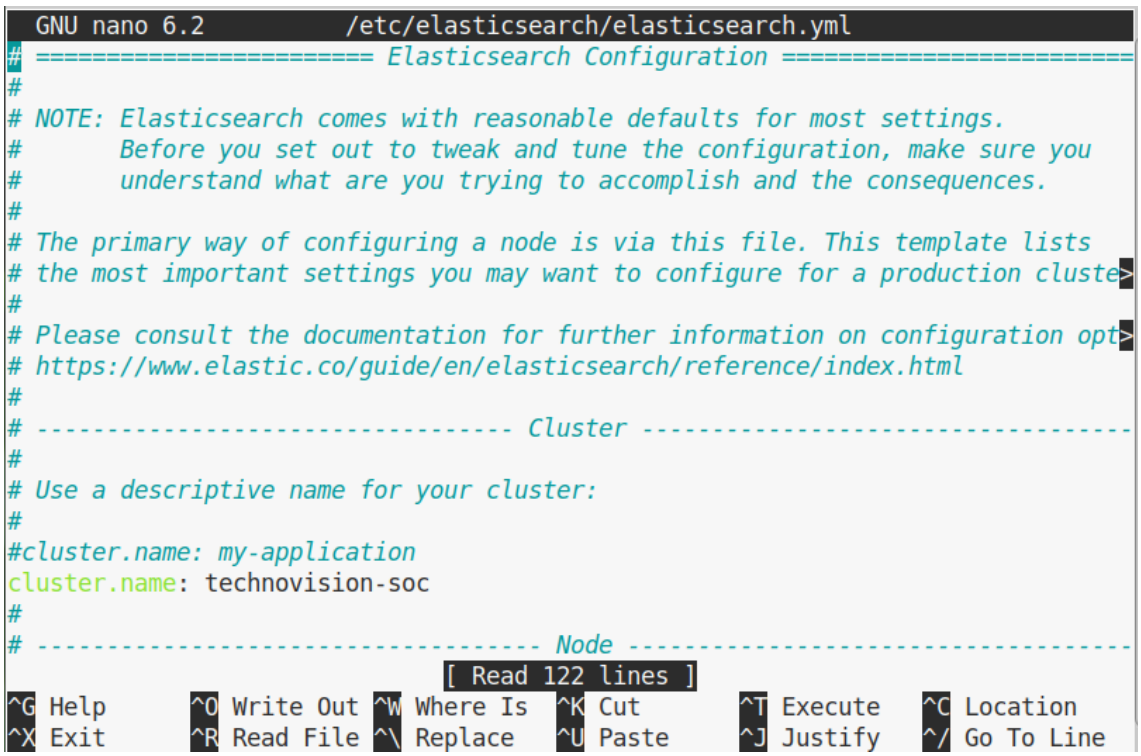
Ensuite, on configure le **premier nœud Elasticsearch**.

On ajoute les paramètres suivants au fichier

sudo nano /etc/elasticsearch/elasticsearch.yml

cluster.name: technovision-soc
node.name: elastic-node1
network.host: 172.16.121.147
transport.host: 172.16.121.147

Puis on démarre le premier nœud :
sudo systemctl start elasticsearch.service



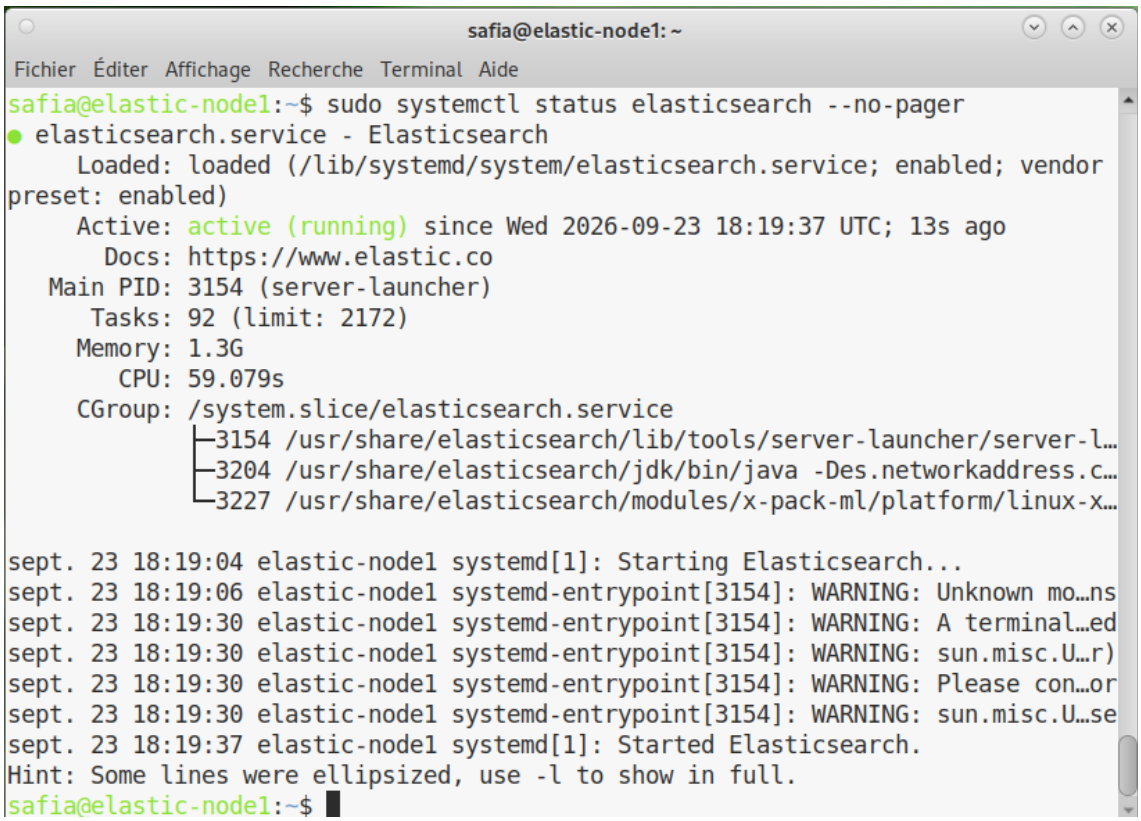
Enfin, on active le service avec la commande :

sudo systemctl daemon-reload
sudo systemctl enable elasticsearch.service

```
safia@elastic-node1:~$ sudo systemctl daemon-reload
sudo systemctl enable elasticsearch.service
Created symlink /etc/systemd/system/multi-user.target.wants/elasticsearch.servic
e → /lib/systemd/system/elasticsearch.service.
safia@elastic-node1:~$
```

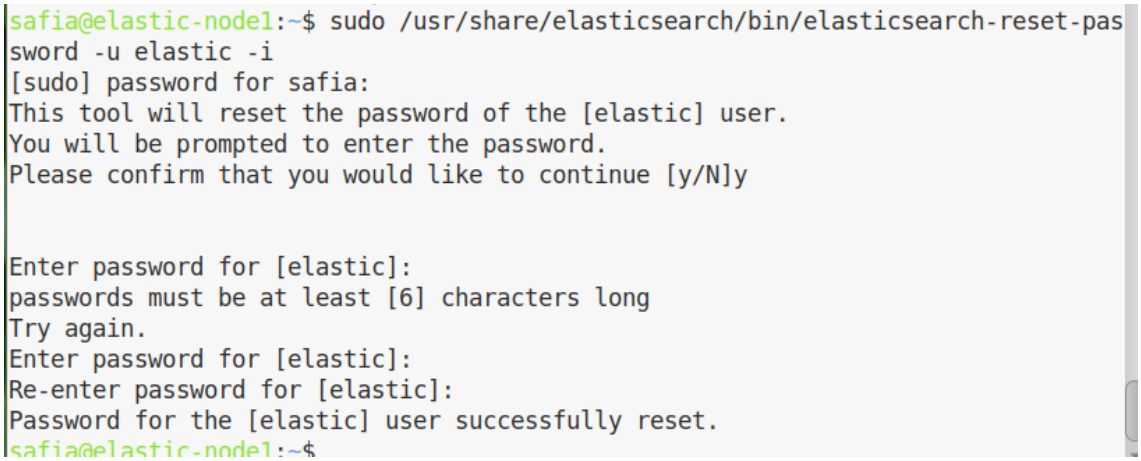
On vérifie que le système est actif :

sudo systemctl status elasticsearch –no-pager



Avant de créer le deuxième nœud , on défini le mot de passe du compte administrateur intégré elastic :

sudo /usr/share/elasticsearch/bin/elasticsearch-reset-password -u elastic -i



Le premier cluster est installé « technovision-soc » :

```
safia@elastic-node1:~$ sudo curl --cacert /etc/elasticsearch/certs/http_ca.crt \
-u elastic https://localhost:9200/_cluster/health?pretty
Enter host password for user 'elastic':
{
  "cluster_name" : "technovision-soc",
  "status" : "green",
  "timed_out" : false,
  "number_of_nodes" : 1,
  "number_of_data_nodes" : 1,
  "active_primary_shards" : 3,
  "active_shards" : 3,
  "relocating_shards" : 0,
  "initializing_shards" : 0,
  "unassigned_shards" : 0,
  "unassigned_primary_shards" : 0,
  "delayed_unassigned_shards" : 0,
  "number_of_pending_tasks" : 0,
  "number_of_in_flight_fetch" : 0,
  "task_max_waiting_in_queue_millis" : 0,
  "active_shards_percent_as_number" : 100.0
}
safia@elastic-node1:~$
```

On peut maintenant installer le second nœud.

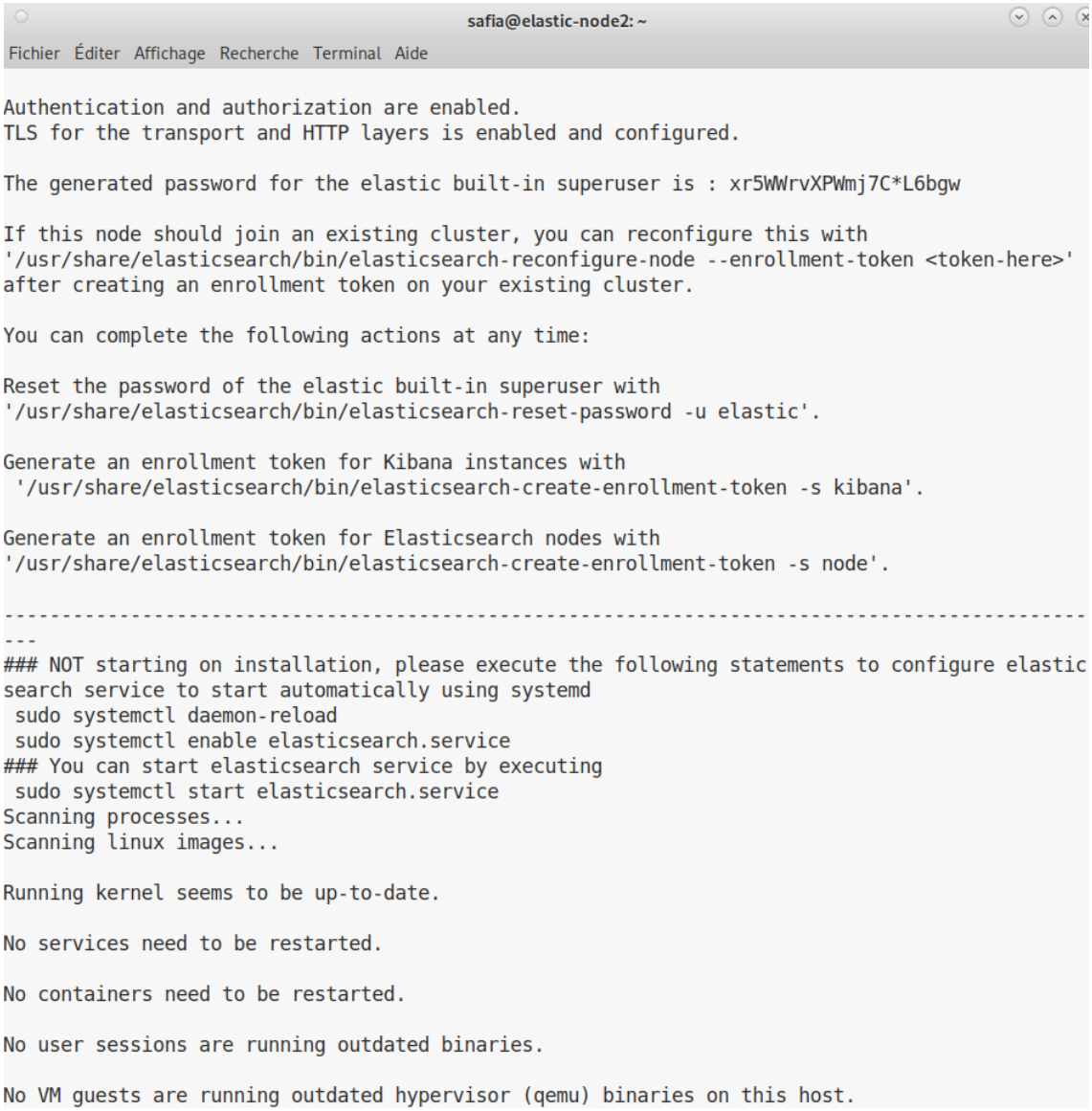
16.1.2. elastic-node2

sudo apt install -y wget gnupg apt-transport-https

wget -qO - https://artifacts.elastic.co/GPG-KEY-elasticsearch \
| sudo gpg --dearmor -o /usr/share/keyrings/elasticsearch-keyring.gpg

echo "deb [signed-by=/usr/share/keyrings/elasticsearch-keyring.gpg]
https://artifacts.elastic.co/packages/9.x/apt stable main" \
| sudo tee /etc/apt/sources.list.d/elastic-9.x.list

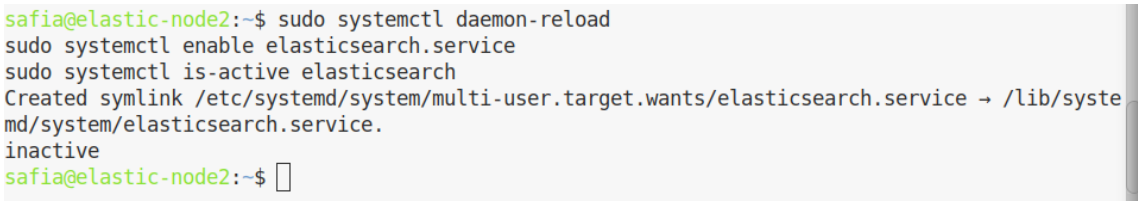
sudo apt update
sudo apt install elasticsearch=9.5.4



17. Installation du cluster

Le nouvel hôte doit être reconfiguré pour rejoindre le cluster existant avant son premier démarrage.

sudo systemctl daemon-reload
sudo systemctl enable elasticsearch.service
sudo systemctl is-active elasticsearch



On fait la commande :

curl -k https://172.16.121.147:9200

Afin de voir si elastic-node2 peut joindre elastic-node1 pour obtenir

status: 401 missing authentication credentials.

```
safia@elastic-node2:~$ curl -k https://172.16.121.147:9200
{"error":{"root_cause":[{"type":"security_exception","reason":"missing authentication credenti
als for REST request [/]","header":{"WWW-Authenticate":["Basic realm=\"security\", charset=\"U
TF-8\"","Bearer realm=\"security\"","ApiKey"]}}],"type":"security_exception","reason":"missing
 authentication credentials for REST request [/]","header":{"WWW-Authenticate":["Basic realm=\
"security\", charset=\"UTF-8\"","Bearer realm=\"security\"","ApiKey"]}},"status":401}safia@ela
stic-node2:~$
```

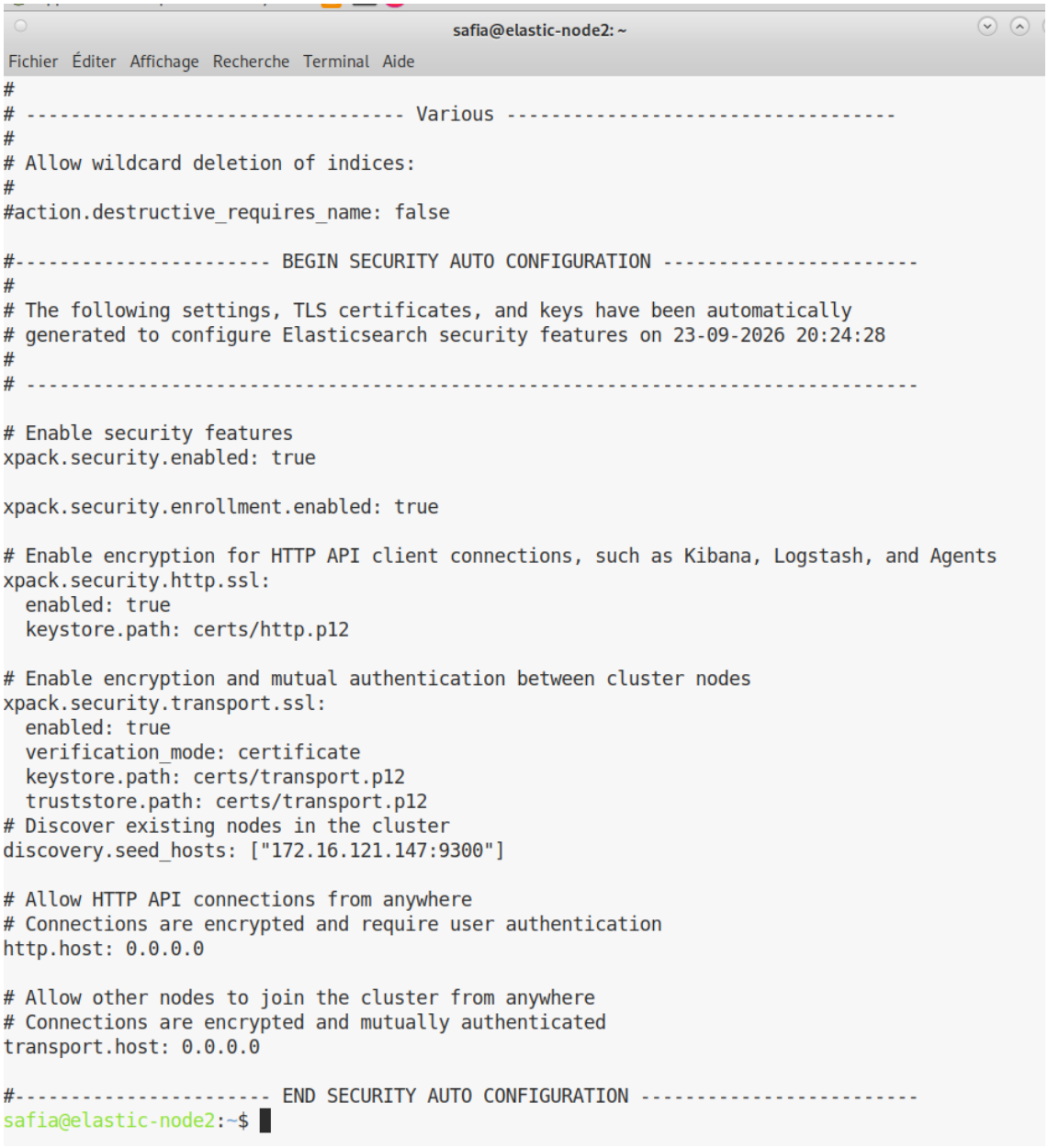
Donc elastic-node2 arrive bien à joindre **Elasticsearch sur 172.16.121.147:9200**, et Elasticsearch répond en exigeant une authentification. Autrement dit :

réseau OK → port 9200 OK → HTTPS OK → Elasticsearch répond.

Depuis elastic-node1, on génère le token :
sudo /usr/share/elasticsearch/bin/elasticsearch-create-enrollment-token -s node

Puis on l’enregistre sur elastic-node2

```
safia@elastic-node1:~$ sudo /usr/share/elasticsearch/bin/elasticsearch-create-enrollment-token
 -s node
[sudo] password for safia:
eyJ2ZXIiOiI4LjE0LjAiLCJhZHIiOlsiMTcyLjE2LjEyMS4xNDc6OTIwMCJdLCJmZ3IiOiI5OWIzY2M4ZTk1NmYwNTFlYj
Q0Y2U4N2M1YzdjZjY4ZDE0MTk5ZmM4ZmE4MTRjZTliMWYzNjQ3NGNlNzBhNzhjIiwia2V5IjoiSnNIc3o2QUI1R3hKeVpX
Z2thaEk6UTcxaGVaSlZnRWlaYWRoTDdiM2JOUSJ9
safia@elastic-node1:~$
```



discovery.seed_hosts: ["172.16.121.147:9300"]

Donc elastic-node2 sait maintenant qu’il doit chercher le cluster existant sur **elastic-node1**.

On enregistre sur le fichier :

sudo nano /etc/elasticsearch/elasticsearch.yml

le nom du cluster et le nom du nœud.



Ensuite, on peut démarrer le second nœud :

sudo systemctl daemon-reload
sudo systemctl enable elasticsearch
sudo systemctl start elasticsearch

```
safia@elastic-node2:~$ sudo systemctl daemon-reload
sudo systemctl enable elasticsearch
sudo systemctl start elasticsearch
safia@elastic-node2:~$ sudo systemctl status elasticsearch --no-pager
● elasticsearch.service - Elasticsearch
     Loaded: loaded (/lib/systemd/system/elasticsearch.service; enabled; vendor preset: enable
d)
     Active: active (running) since Wed 2026-09-23 20:36:35 UTC; 23s ago
       Docs: https://www.elastic.co
   Main PID: 3513 (server-launcher)
      Tasks: 64 (limit: 2172)
     Memory: 1.2G
        CPU: 56.640s
     CGroup: /system.slice/elasticsearch.service
             ├─3513 /usr/share/elasticsearch/lib/tools/server-launcher/server-launcher -p /ru…
             ├─3563 /usr/share/elasticsearch/jdk/bin/java -Des.networkaddress.cache.ttl=60 -D…
             └─3586 /usr/share/elasticsearch/modules/x-pack-ml/platform/linux-x86_64/bin/cont…

sept. 23 20:35:59 elastic-node2 systemd[1]: Starting Elasticsearch...
sept. 23 20:36:01 elastic-node2 systemd-entrypoint[3513]: WARNING: Unknown module: org.apa…ens
sept. 23 20:36:25 elastic-node2 systemd-entrypoint[3513]: WARNING: A terminally deprecated…led
sept. 23 20:36:25 elastic-node2 systemd-entrypoint[3513]: WARNING: sun.misc.Unsafe::arrayB…ar)
sept. 23 20:36:25 elastic-node2 systemd-entrypoint[3513]: WARNING: Please consider reporti…sor
sept. 23 20:36:25 elastic-node2 systemd-entrypoint[3513]: WARNING: sun.misc.Unsafe::arrayB…ase
sept. 23 20:36:35 elastic-node2 systemd[1]: Started Elasticsearch.
Hint: Some lines were ellipsized, use -l to show in full.
safia@elastic-node2:~$
```

Puis on retourne sur elastic-node1 afin de vérifier la mise en place du cluster :

sudo curl --cacert /etc/elasticsearch/certs/http_ca.crt \
-u elastic https://localhost:9200/_cat/nodes?v

```
safia@elastic-node1:~$ sudo curl --cacert /etc/elasticsearch/certs/http_ca.crt \
-u elastic https://localhost:9200/_cat/nodes?v
[sudo] password for safia:
Enter host password for user 'elastic':
ip            heap.percent ram.percent cpu load_1m load_5m load_15m node.role   master name
172.16.121.147           21          94   1    0.20    0.08     0.02 cdfhilmrstw *      elasti
c-node1
172.16.121.148           29          96  12    0.07    0.11     0.05 cdfhilmrstw -      elasti
c-node2
safia@elastic-node1:~$
```

Le cluster Elasticsearch à 2 nœuds fonctionne.

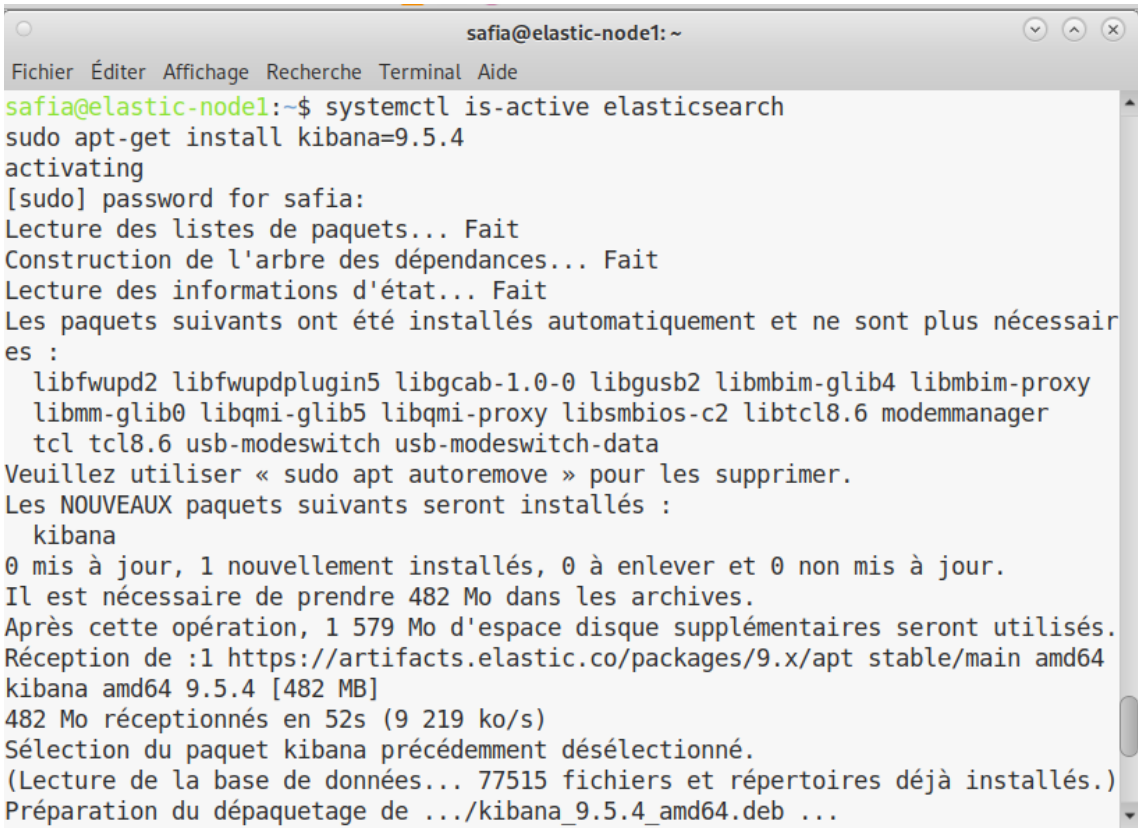
```
safia@elastic-node1:~$ sudo curl --cacert /etc/elasticsearch/certs/http_ca.crt \
-u elastic https://localhost:9200/_cluster/health?pretty
Enter host password for user 'elastic':
{
  "cluster_name" : "technovision-soc",
  "status" : "green",
  "timed_out" : false,
  "number_of_nodes" : 2,
  "number_of_data_nodes" : 2,
  "active_primary_shards" : 3,
  "active_shards" : 6,
  "relocating_shards" : 0,
  "initializing_shards" : 0,
  "unassigned_shards" : 0,
  "unassigned_primary_shards" : 0,
  "delayed_unassigned_shards" : 0,
  "number_of_pending_tasks" : 0,
  "number_of_in_flight_fetch" : 0,
  "task_max_waiting_in_queue_millis" : 0,
  "active_shards_percent_as_number" : 100.0
}
safia@elastic-node1:~$
```

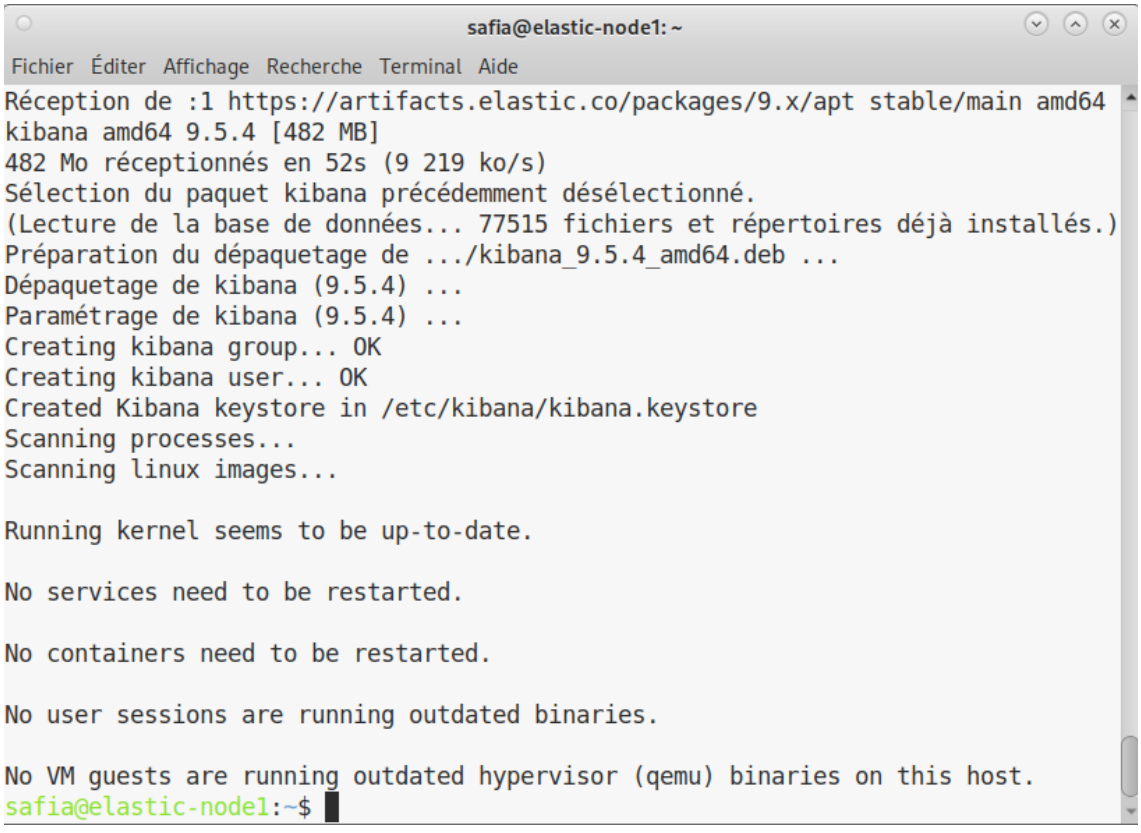
18. Installation de Kibana

Pour installer kibana, on utilise la VM **elastic-node1** qui est prévue pour héberger Kibana, avec elastic-node2 comme second nœud Elasticsearch. Kibana doit avoir **exactement la même version** qu'Elasticsearch.

On lance les commandes pour installer kibana :

systemctl is-active elasticsearch
sudo apt-get install kibana=9.5.4





Avant de connecter kibana on veut voir les deux nœuds du cluster, on lance la commande :

sudo curl --cacert /etc/elasticsearch/certs/http_ca.crt -u elastic 'https://localhost:9200/_cat/nodes?v'

```
safia@elastic-node1:~$ sudo curl --cacert /etc/elasticsearch/certs/http_ca.crt -
u elastic 'https://localhost:9200/_cat/nodes?v'
Enter host password for user 'elastic':
ip             heap.percent ram.percent cpu load_1m load_5m load_15m node.role
 master name
172.16.121.147           21          96   5    0.02    0.02     0.06 cdfhilmrstw
 *      elastic-node1
172.16.121.148           40          93   1    0.00    0.02     0.00 cdfhilmrstw
 -      elastic-node2
safia@elastic-node1:~$
```

la commande affiche bien elastic-node1 (172.16.121.147) et elastic-node2 (172.16.121.148) dans le même cluster. Nous pouvons connecter Kibana.

sudo nano /etc/kibana/kibana.yml

Onajoute l’adresse IP du serveur

server.host: "172.16.121.147"

```
  GNU nano 6.2                        /etc/kibana/kibana.yml *
# For more configuration options see the configuration guide for Kibana in
# https://www.elastic.co/guide/index.html

# ================== System: Kibana Server ==================
# Kibana is served by a back end server. This setting specifies the port to use.
#server.port: 5601

# Specifies the address to which the Kibana server will bind. IP addresses and host names are>
# The default is 'localhost', which usually means remote machines will not be able to connect.
# To allow connections from remote users, set this parameter to a non-loopback address.
#server.host: "localhost"
server.host: "172.16.121.147"
# Enables you to specify a path to mount Kibana at if you are running behind a proxy.
# Use the `server.rewriteBasePath` setting to tell Kibana if it should remove the basePath
# from requests it receives, and to prevent a deprecation warning at startup.
# This setting cannot end in a slash.
#server.basePath: ""

# Specifies whether Kibana should rewrite requests that are prefixed with
# `server.basePath` or require that they are rewritten by your reverse proxy.
# Defaults to `false`.
#server.rewriteBasePath: false

# Specifies the public URL at which Kibana is available for end users. If
# `server.basePath` is configured this URL should end with the same basePath.
#server.publicBaseUrl: ""

# The maximum payload size in bytes for incoming server requests.
#server.maxPayload: 1048576

# The Kibana server's name. This is used for display purposes.
#server.name: "your-hostname"

# Block requests to specific routes (exact path match, evaluated before auth).
# server.excludeRoutes: ["/api/status"]

# ================== System: Kibana Server (Optional) ==================
# Enables SSL and paths to the PEM-format SSL certificate and SSL key files, respectively.
# These settings enable SSL for outgoing requests from the Kibana server to the browser.

^G Help        ^O Write Out   ^W Where Is    ^K Cut         ^T Execute     ^C Location
^X Exit        ^R Read File   ^\ Replace     ^U Paste       ^J Justify     ^/ Go To Line
```

Puis on démarre kibana :

sudo systemctl enable --now kibana
sudo systemctl status kibana –no-pager

```
safia@elastic-node1:~$ sudo systemctl enable --now kibana
sudo systemctl status kibana --no-pager
Created symlink /etc/systemd/system/multi-user.target.wants/kibana.service → /lib/systemd/syst
em/kibana.service.
● kibana.service - Kibana
     Loaded: loaded (/lib/systemd/system/kibana.service; enabled; vendor preset: enabled)
     Active: active (running) since Thu 2026-09-24 08:55:06 UTC; 17ms ago
       Docs: https://www.elastic.co
   Main PID: 1613 (kibana)
      Tasks: 2 (limit: 6505)
     Memory: 532.0K
        CPU: 6ms
     CGroup: /system.slice/kibana.service
             ├─1613 /bin/sh /usr/share/kibana/bin/kibana
             └─1617 /usr/share/kibana/bin/../node/default/bin/node --version

sept. 24 08:55:06 elastic-node1 systemd[1]: Started Kibana.
safia@elastic-node1:~$
```

Une fois que le service est actif, sur la page du navigateur, on lance l’interface graphique http://172.16.121.147:5601

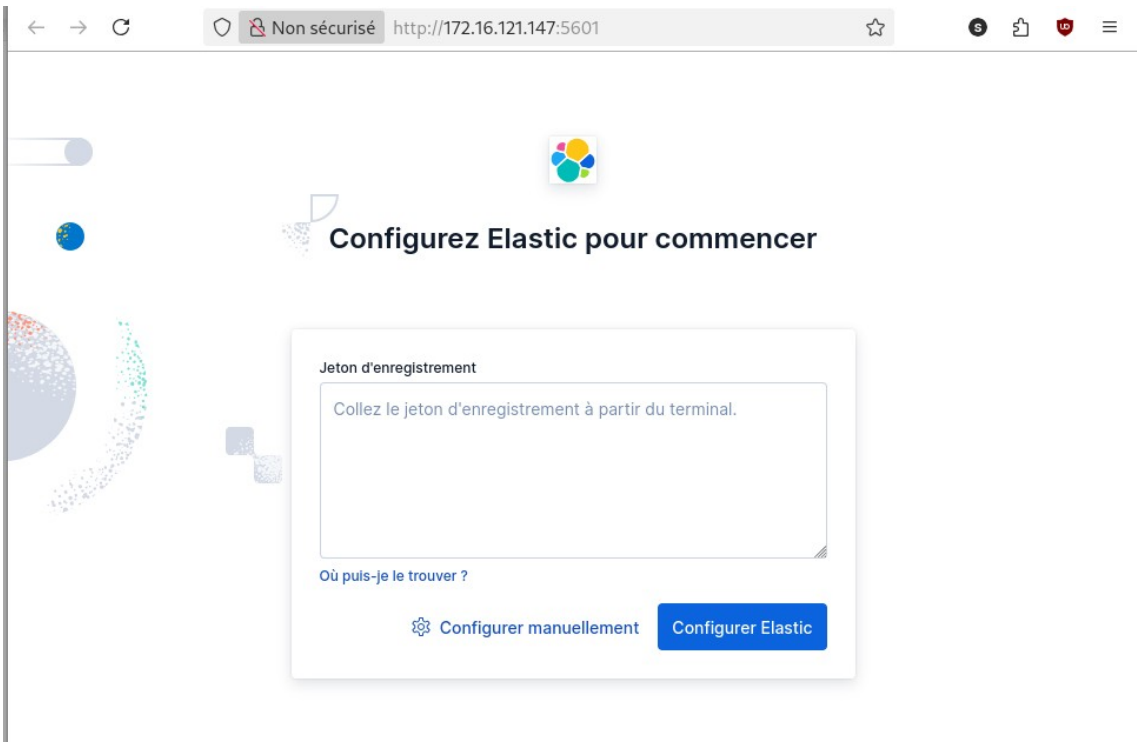
On récupère un jeton d’inscription que l’on obtient en allant sur elastic-node1 avec la commande :

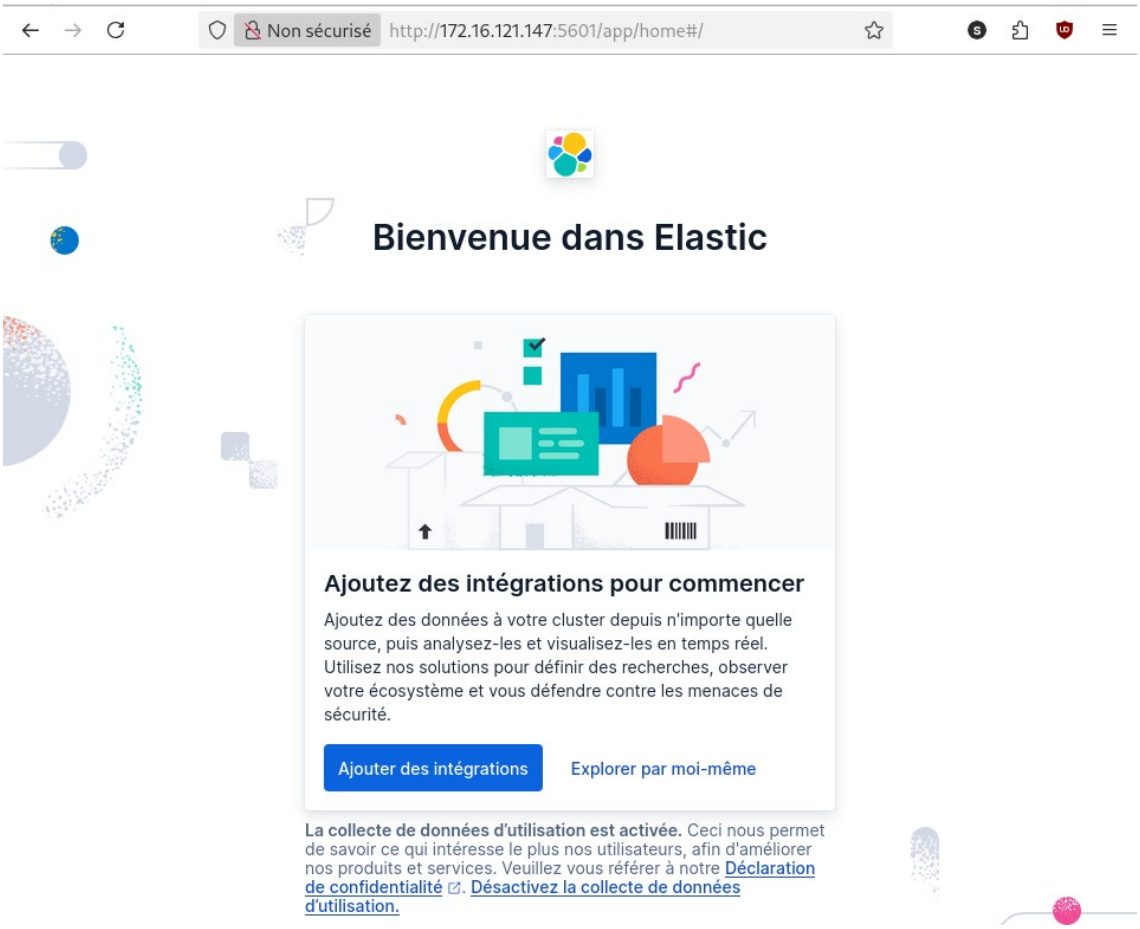
sudo /usr/share/elasticsearch/bin/elasticsearch-create-enrollment-token -s kibana --url https://localhost:9200

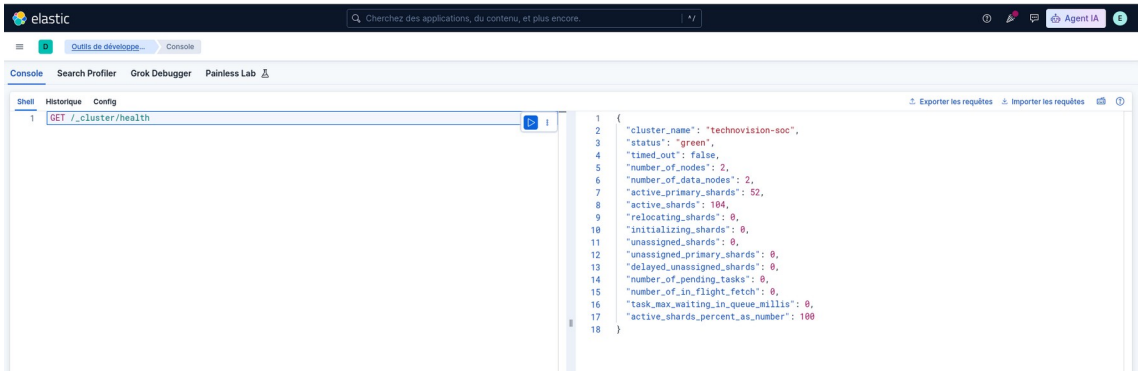

```
{
  "cluster_name": "technovision-soc",
  "status": "green",
  "timed_out": false,
  "number_of_nodes": 2,
  "number_of_data_nodes": 2,
  "active_primary_shards": 52,
  "active_shards": 104,
  "relocating_shards": 0,
  "initializing_shards": 0,
  "unassigned_shards": 0,
  "unassigned_primary_shards": 0,
  "delayed_unassigned_shards": 0,
  "number_of_pending_tasks": 0,
  "number_of_in_flight_fetch": 0,
```


Effacer cette sortie Filtrer les réponses 200 - OK 52 ms

L’installation de kibana est terminée.


Non sécurisé
http://172.16.121.147:5601
Configurez Elastic pour commencer
Jeton d'enregistrement
Collez le jeton d'enregistrement à partir du terminal.
Où puis-je le trouver ?
Configurer manuellement
Configurer Elastic


Non sécurisé
http://172.16.121.147:5601/app/home#/
Bienvenue dans Elastic
Ajoutez des intégrations pour commencer
Ajoutez des données à votre cluster depuis n'importe quelle source, puis analysez-les et visualisez-les en temps réel. Utilisez nos solutions pour définir des recherches, observer votre écosystème et vous défendre contre les menaces de sécurité.
Ajouter des intégrations
Explorer par moi-même
La collecte de données d'utilisation est activée. Ceci nous permet de savoir ce qui intéresse le plus nos utilisateurs, afin d'améliorer nos produits et services. Veuillez vous référer à notre Déclaration de confidentialité. Désactivez la collecte de données d'utilisation.



On retrouve notre cluster avec les 2 nœuds.

19. Installation de filebeat

On utilise la même version que pour le reste de la pile 9.5.4

Sur la VM elastic-node1, on utilise la commande :

sudo apt-get install filebeat=9.5.4

```
safia@elastic-node1:~$ sudo apt-get install filebeat=9.5.4
[sudo] password for safia:
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
Les paquets suivants ont été installés automatiquement et ne sont plus nécessaires :
  libfwupd2 libfwupdplugin5 libgcab-1.0-0 libgusb2 libmbim-glib4 libmbim-proxy libmm-glib0
  libqmi-glib5 libqmi-proxy libsmbios-c2 libtcl8.6 modemmanager tcl tcl8.6 usb-modeswitch
  usb-modeswitch-data
Veuillez utiliser « sudo apt autoremove » pour les supprimer.
Les NOUVEAUX paquets suivants seront installés :
  filebeat
0 mis à jour, 1 nouvellement installés, 0 à enlever et 2 non mis à jour.
Il est nécessaire de prendre 71,9 Mo dans les archives.
Après cette opération, 272 Mo d'espace disque supplémentaires seront utilisés.
Réception de :1 https://artifacts.elastic.co/packages/9.x/apt stable/main amd64 filebeat amd64
 9.5.4 [71,9 MB]
71,9 Mo réceptionnés en 7s (10,6 Mo/s)
Sélection du paquet filebeat précédemment désélectionné.
(Lecture de la base de données... 198819 fichiers et répertoires déjà installés.)
Préparation du dépaquetage de .../filebeat_9.5.4_amd64.deb ...
Dépaquetage de filebeat (9.5.4) ...
Paramétrage de filebeat (9.5.4) ...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
safia@elastic-node1:~$
```

On installe le certificat ssl avec la clé d’authentification avec les commandes suivantes :

sudo install -D -m 600 /etc/elasticsearch/certs/http_ca.crt /etc/filebeat/certs/http_ca.crt
sudo filebeat keystore create
sudo filebeat keystore add ES_PWD

```
safia@elastic-node1:~$ sudo install -D -m 600 /etc/elasticsearch/certs/http_ca.crt /etc/filebe
at/certs/http_ca.crt
sudo filebeat keystore create
sudo filebeat keystore add ES_PWD
Created filebeat keystore
Enter value for ES_PWD:
Successfully updated the keystore
```

Après avoir enregistré la clé générée sur kibana, on modifie le fichier yml pour y ajouter ces paramètres :

sudo nano /etc/filebeat/filebeat.yml

```
# ================================== Outputs ===================================

# Configure what output to use when sending the data collected by the beat.

# ---------------------------- Elasticsearch Output ----------------------------
output.elasticsearch:
  # Array of hosts to connect to.
  hosts: ["https://localhost:9200"]
  username: "elastic"
  password: "${ES_PWD}"
  ssl.certificate_authorities: ["/etc/filebeat/certs/http_ca.crt"]


  # Performance preset - one of "balanced", "throughput", "scale",
  # "latency", or "custom".
  preset: balanced

  # Protocol - either `http` (default) or `https`.
  #protocol: "https"
```

Puis on teste le changement et l’enregistrement avec les commandes :

sudo filebeat test config
sudo filebeat test output

```
safia@elastic-node1:~$ sudo filebeat test config
sudo filebeat test output
Config OK
elasticsearch: https://localhost:9200...
  parse url... OK
  connection...
    parse host... OK
    dns lookup... OK
    addresses: 127.0.0.1
    dial up... OK
  TLS...
    security: server's certificate chain verification is enabled
    handshake... OK
    TLS version: TLSv1.3
    dial up... OK
  talk to server... OK
  version: 9.5.4
safia@elastic-node1:~$
```

Les tests sont réussis : Config OK, certificat vérifié, TLS 1.3 et connexion à Elasticsearch 9.5.4.

On doit vérifier où Ubuntu conserve ses journaux avant de configurer le module system. Donc on lance la commande :

sudo ls -l /var/log/auth.log /var/log/syslog

```
safia@elastic-node1:~$ sudo ls -l /var/log/auth.log /var/log/syslog
-rw-r----- 1 syslog adm   38340 sept. 24 13:14 /var/log/auth.log
-rw-r----- 1 syslog adm 1470051 sept. 24 13:08 /var/log/syslog
safia@elastic-node1:~$
```

Les deux journaux sont présents. Il faut activer le module system pour lire les événements de connexion dans auth.log et les événements système dans syslog. Donc on modifie le fichier de configuration.

sudo filebeat modules enable system
sudo nano /etc/filebeat/modules.d/system.yml

```
 GNU nano 6.2                  /etc/filebeat/modules.d/system.yml *
# Module: system
# Docs: https://www.elastic.co/guide/en/beats/filebeat/main/filebeat-module-system.html

- module: system
  # Syslog
  syslog:
    enabled: true
    var.paths: ["/var/log/syslog*"]

    # Set custom paths for the log files. If left empty,
    # Filebeat will choose the paths depending on your OS.
    #var.paths:

    # Use journald to collect system logs
    #var.use_journald: false

  # Authorization logs
  auth:
    enabled: true
    var.paths: ["/var/log/auth.log*"]


    # Set custom paths for the log files. If left empty,
    # Filebeat will choose the paths depending on your OS.
    #var.paths:

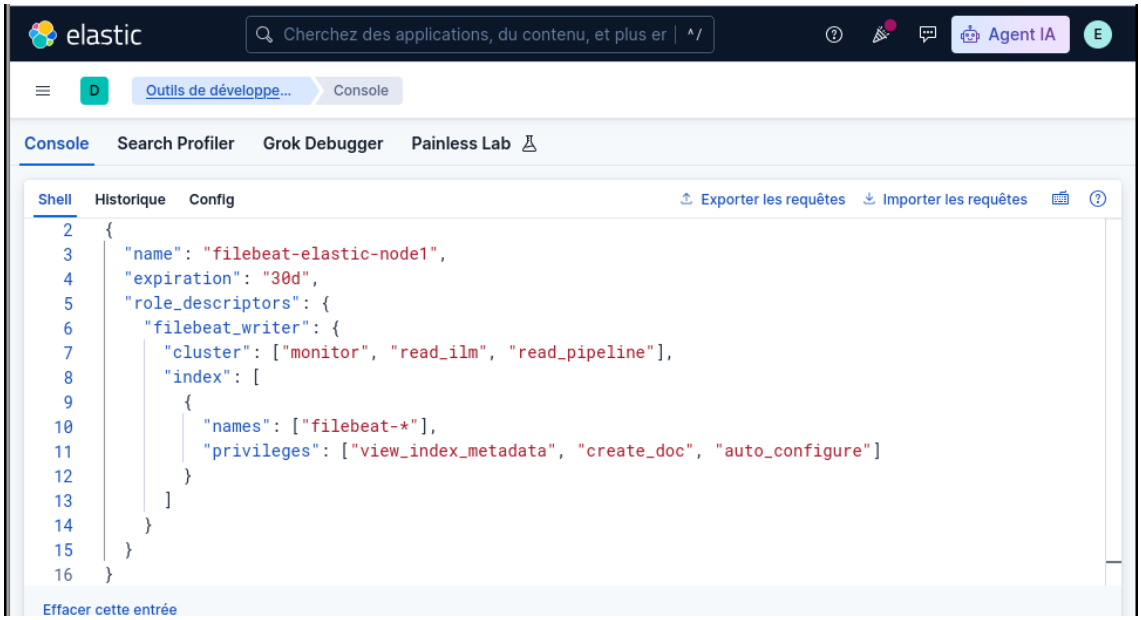
    # Use journald to collect auth logs
    #var.use_journald: false
```

On lance la commande pour charger les pipelines :

sudo filebeat setup --pipelines --modules system –force-enable-module-filesets

```
safia@elastic-node1:~$ sudo filebeat setup --pipelines --modules system --force-enable-module-
filesets
Loaded Ingest pipelines
safia@elastic-node1:~$
```

«Loaded Ingest pipelines » confirme que les traitements des journaux sont installés.
Maintenant on va générer une clé depuis la console kibana afin de la limiter à la publication de filebeat :



On enregistre la clé générée par la console kibana :

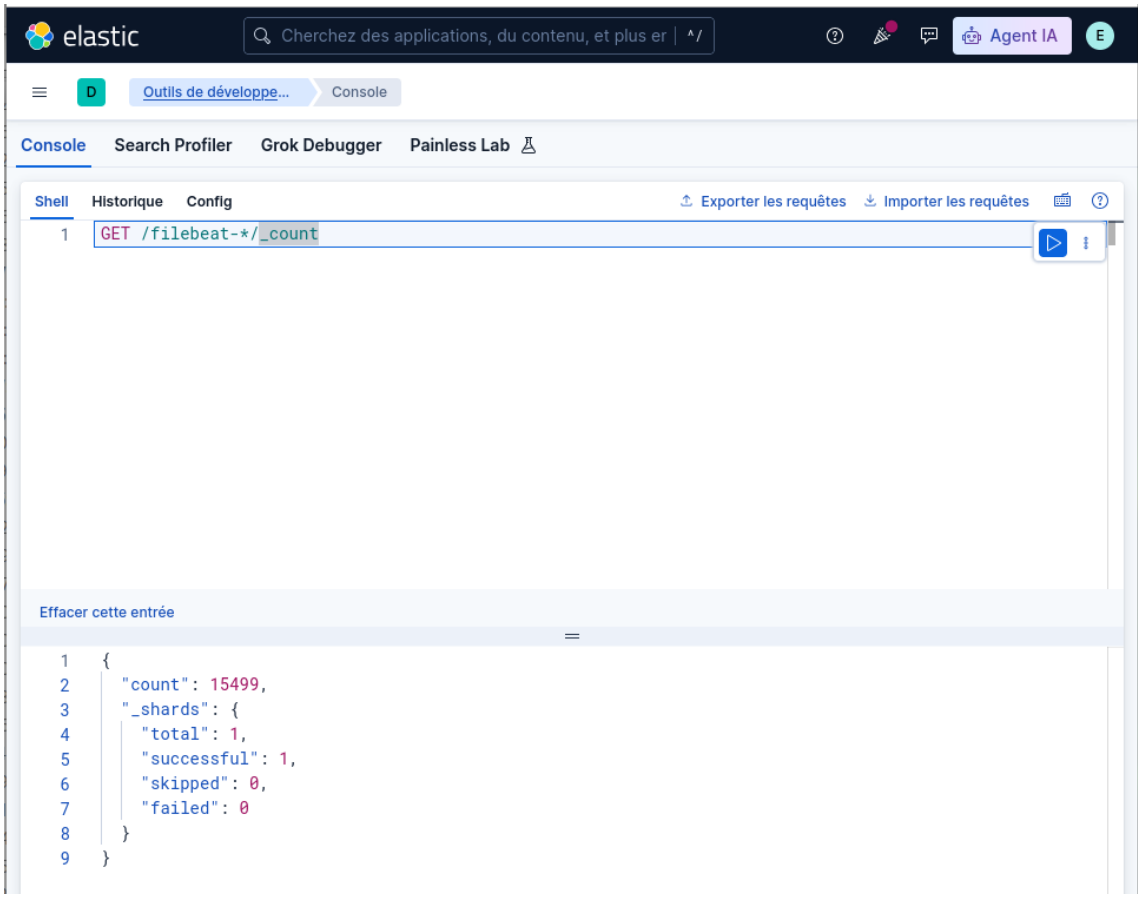
sudo filebeat keystore add FILEBEAT_API_KEY --force

```
safia@elastic-node1:~$ sudo filebeat keystore add FILEBEAT_API_KEY --force
Enter value for FILEBEAT_API_KEY:
Successfully updated the keystore
```

Puis on relance pour voir si la connexion HTTPS fonctionne et si les journaux Ubuntu arrivent bien dans Elasticsearch.

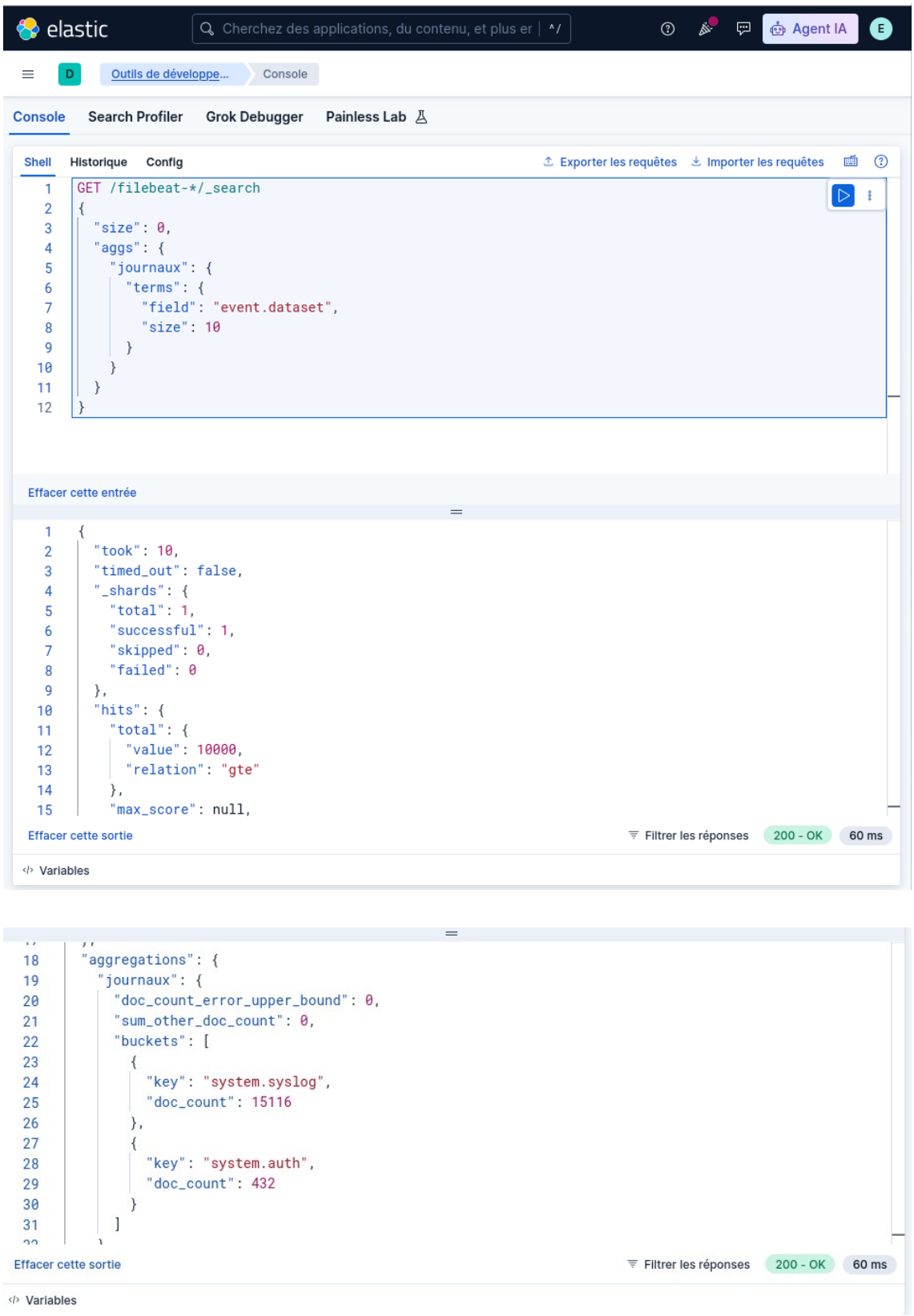
sudo filebeat keystore remove ES_PWD
sudo systemctl enable --now filebeat
sudo systemctl status filebeat –no-pager

Sur la console kibana, on vérifie avec la commande :
GET /filebeat-*/_count



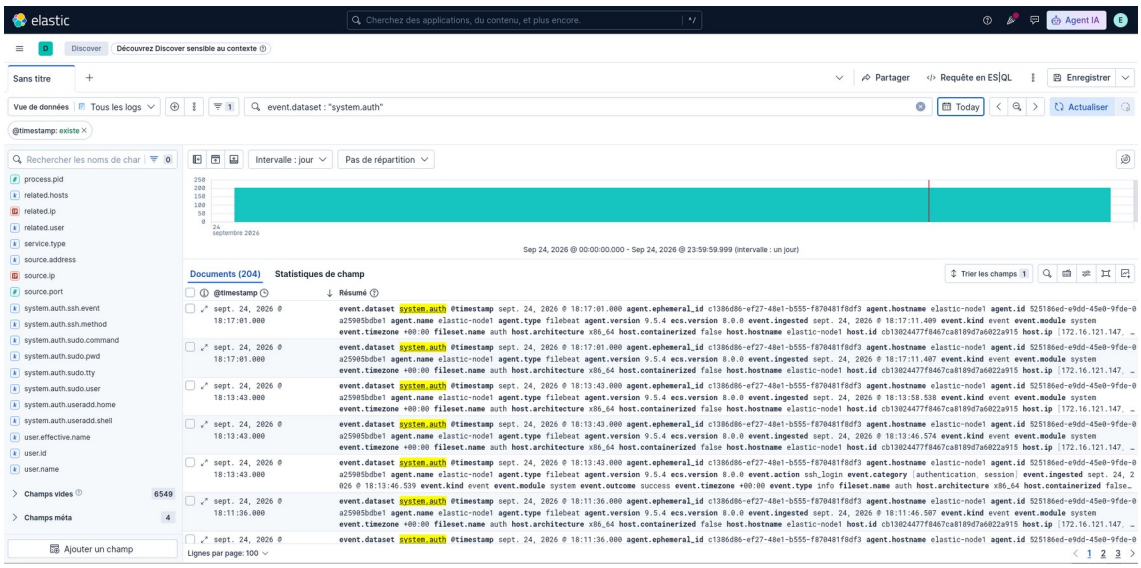
la réponse montre 15 499 événements Filebeat indexés, avec failed: 0. La collecte des journaux de elastic-node1 fonctionne.

Pour vérifier que les deux sources arrivent bien, on exécute dans la console Kibana :

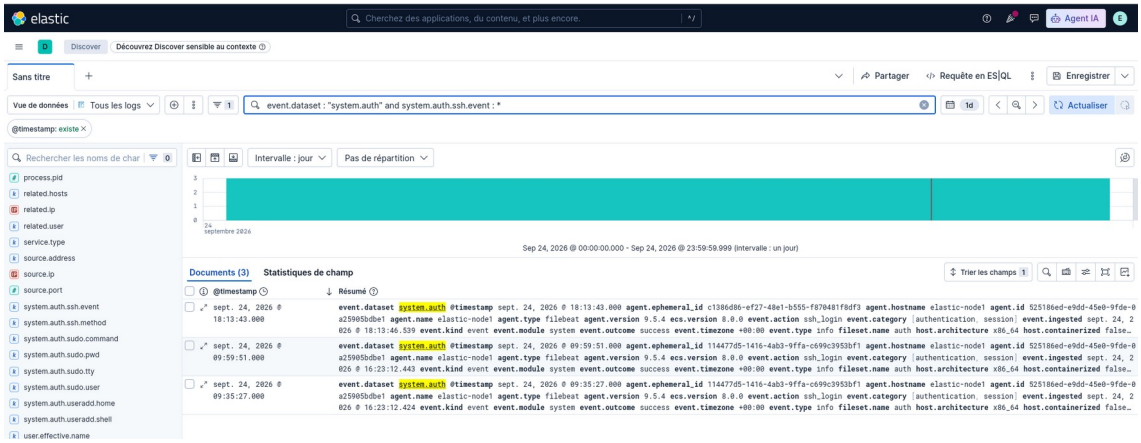


Filebeat a envoyé 15 116 événements system.syslog et 432 événements system.auth. Les deux types de journaux arrivent bien dans Elasticsearch.

Pour voir les journaux d’authentification, on renseigne dans la barre de recherche :
event.dataset : "system.auth"

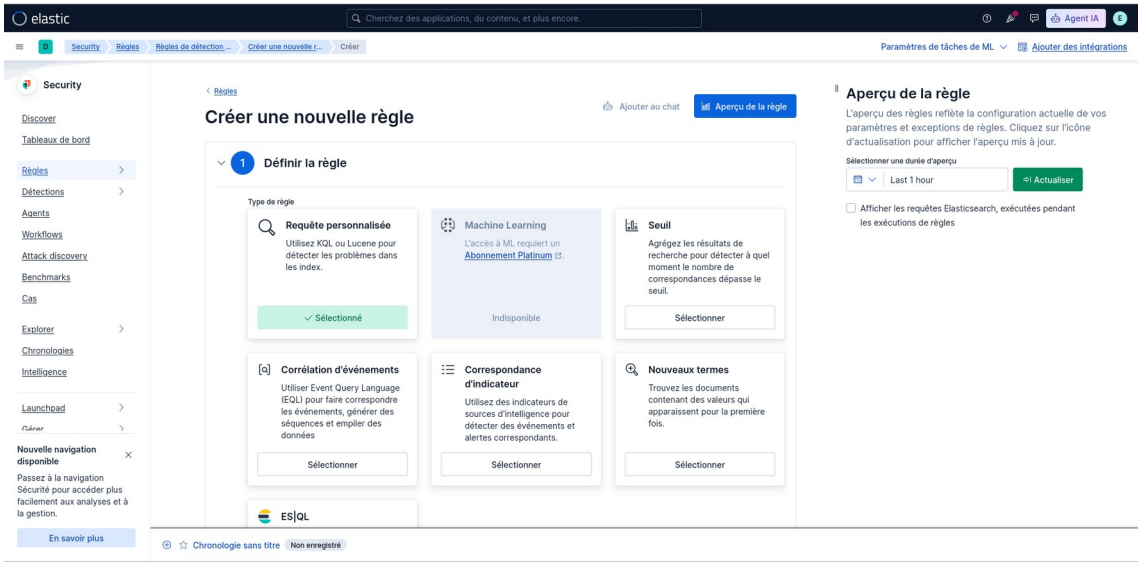


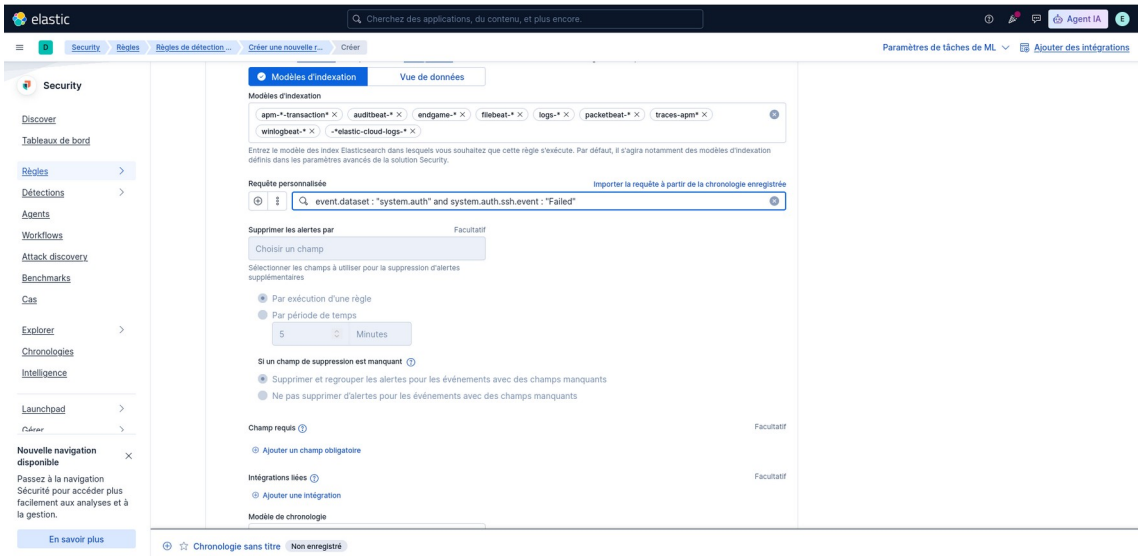
Pour voir les connexions ssh :
event.dataset : "system.auth" and system.auth.ssh.event : *



20. Création de la règle SSH

On crée une nouvelle règle sur kibana pour la détection SSH, Cette première règle détectera **chaque échec de connexion SSH** enregistré par Filebeat.





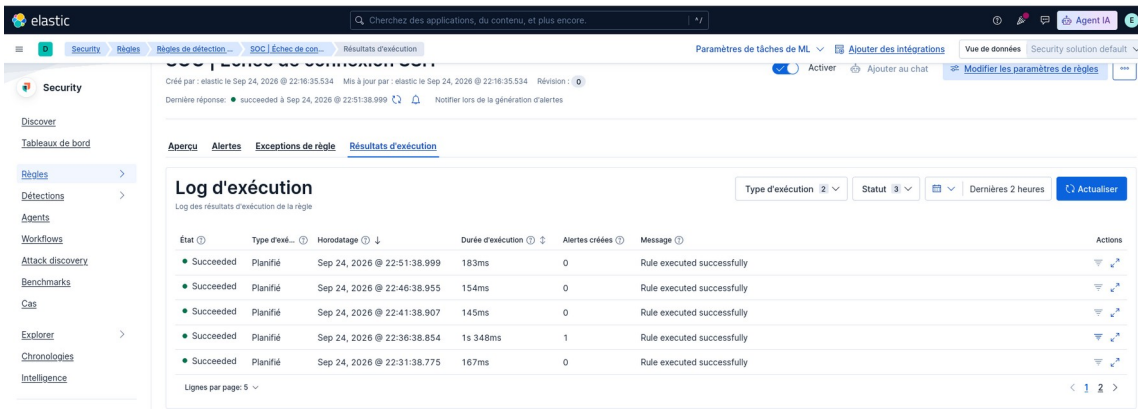


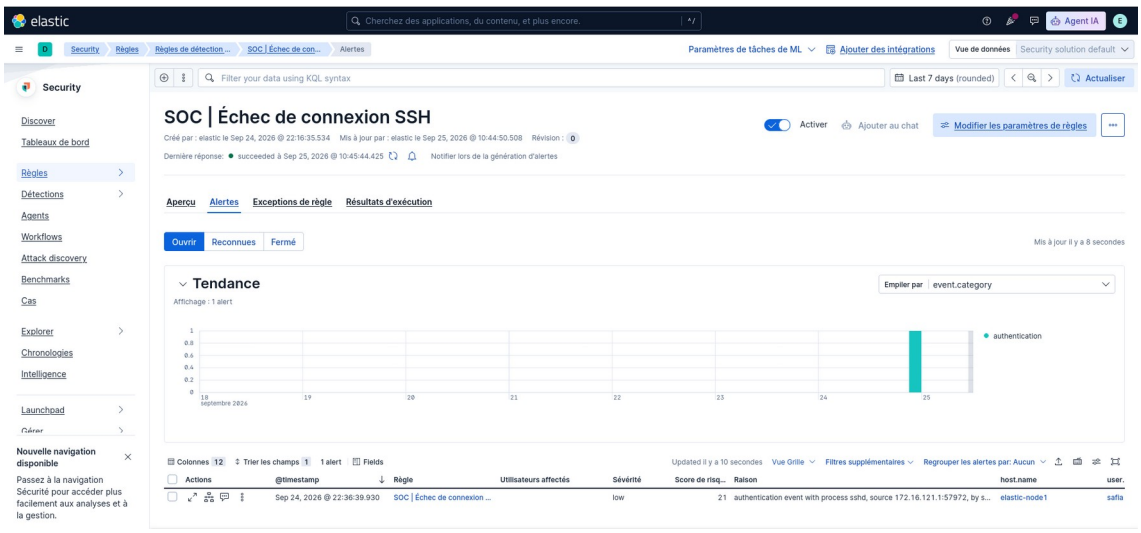
On teste la règle avec des mots de passe erronés

```
safia@localhost:~$ ssh safia@172.16.121.147
safia@172.16.121.147's password:
Permission denied, please try again.
safia@172.16.121.147's password:
Permission denied, please try again.
safia@172.16.121.147's password:
safia@172.16.121.147: Permission denied (publickey,password).
safia@localhost:~$
```

La règle a bien fonctionné et la détection est visible sur kibana.

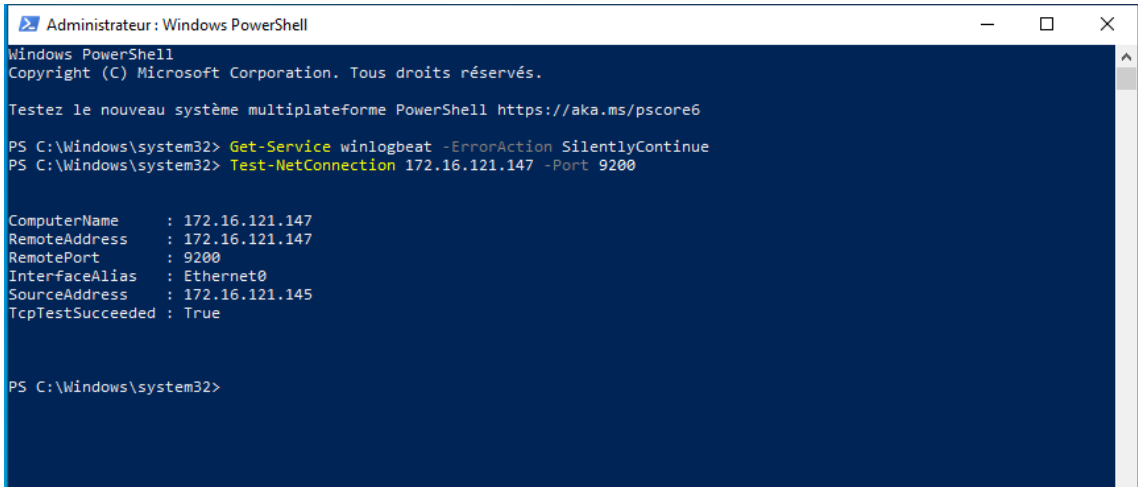

elastic
Cherchez des applications, du contenu, et plus encore.
Agent IA
Discover
Découvrez Discover sensible au contexte
Sans titre
Partager
Requête en ES|QL
Enregistrer
Vue de données
Tous les logs
event.dataset : "system.auth" and system.auth.ssh.event : "Failed"
Last 15 minutes
Actualiser
Rechercher les noms de char
Intervalle automatique
Pas de répartition
Champs disponibles
61
@timestamp
agent.ephemeral_id
agent.hostname
agent.id
agent.name
agent.type
agent.version
data_stream.dataset
data_stream.namespace
data_stream.type
ecs.version
event.action
Sep 24, 2026 @ 22:17:56.471 - Sep 24, 2026 @ 22:32:56.471 (intervalle : Auto - 30 secondes)
Documents (1)
Statistiques de champ
Trier les champs
@timestamp
Résumé
sept. 24, 2026 @ 22:32:09.000
event.dataset system.auth system.auth.ssh.event Failed @timestamp sept. 24, 2026 @ 22:32:09.000 agent.ephemeral_id c1386d86-ef27-48e1-b555-f870481f8df3 agent.hostname elastic-node1 agent.id 525186ed-e9dd-45e0-9fde-0a25905bdbe1 agent.name elastic-node1 agent.type filebeat agent.version 9.5.4 ecs.version 8.0.0 event.action ssh_login event.category authentication event.ingested sept. 24, 2026 @ 22:32:25.581 event.kind event event.module system event.outcome failure event.timezone +00:00 event.type info fileset.name auth host.architecture x86_64…


elastic
Cherchez des applications, du contenu, et plus encore.
Agent IA
Security
Règles
Règles de détection…
SOC | Échec de con…
Résultats d'exécution
Paramètres de tâches de ML
Ajouter des intégrations
Vue de données
Security solution default
Activer
Ajouter au chat
Modifier les paramètres de règles
Discover
Tableaux de bord
Règles
Détections
Agents
Workflows
Attack discovery
Benchmarks
Cas
Explorer
Chronologies
Intelligence
Aperçu
Alertes
Exceptions de règle
Résultats d'exécution
Log d'exécution
Log des résultats d'exécution de la règle
Type d'exécution 2
Statut 3
Dernières 2 heures
Actualiser
État
Type d'exé…
Horodatage
Durée d'exécution
Alertes créées
Message
Actions
Succeeded
Planifié
Sep 24, 2026 @ 22:51:38.999
183ms
0
Rule executed successfully
Succeeded
Planifié
Sep 24, 2026 @ 22:46:38.955
154ms
0
Rule executed successfully
Succeeded
Planifié
Sep 24, 2026 @ 22:41:38.907
145ms
0
Rule executed successfully
Succeeded
Planifié
Sep 24, 2026 @ 22:36:38.854
1s 348ms
1
Rule executed successfully
Succeeded
Planifié
Sep 24, 2026 @ 22:31:38.775
167ms
0
Rule executed successfully
Lignes par page: 5

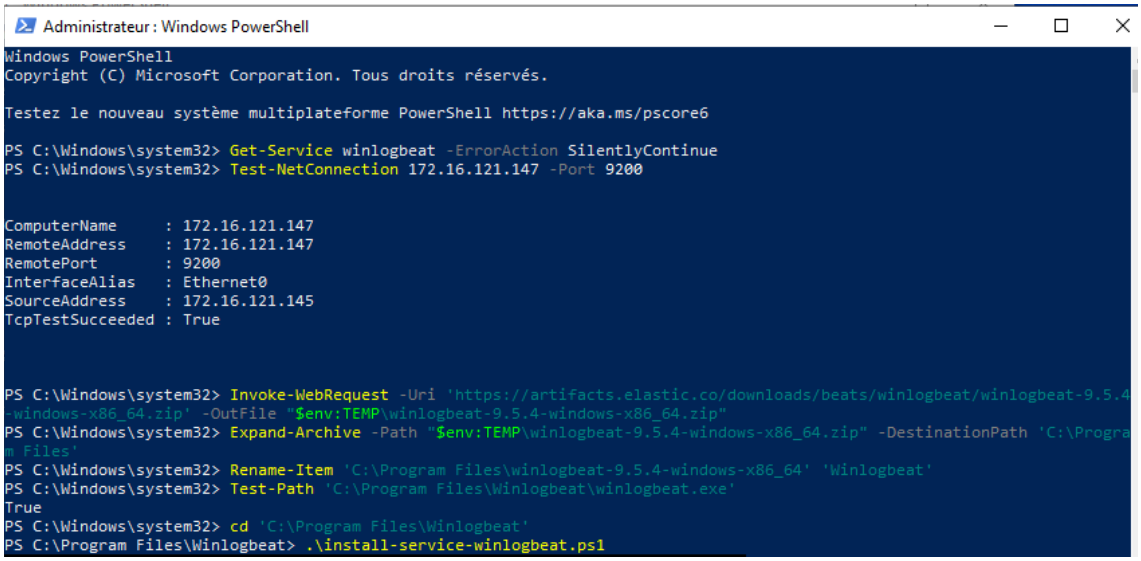

elastic
Cherchez des applications, du contenu, et plus encore.
Agent IA
Security
Règles
Règles de détection…
SOC | Échec de con…
Alertes
Paramètres de tâches de ML
Ajouter des intégrations
Vue de données
Security solution default
Filter your data using KQL syntax
Last 7 days (rounded)
Actualiser
SOC | Échec de connexion SSH
Activer
Ajouter au chat
Modifier les paramètres de règles
Aperçu
Alertes
Exceptions de règle
Résultats d'exécution
Ouvrir
Reconnues
Fermé
Mis à jour il y a 8 secondes
Tendance
Affichage : 1 alert
Empiler par
event.category
authentication
Colonnes 12
Trier les champs 1
1 alert
Fields
Updated il y a 10 secondes
Vue Grille
Filtres supplémentaires
Regrouper les alertes par: Aucun
Actions
@timestamp
Règle
Utilisateurs affectés
Sévérité
Score de risq…
Raison
host.name
user.
Sep 24, 2026 @ 22:36:39.930
SOC | Échec de connexion …
low
21
authentication event with process sshd, source 172.16.121.1:57972, by s…
elastic-node1
safia
Launchpad
Nouvelle navigation disponible
Passez à la navigation Sécurité pour accéder plus facilement aux analyses et à la gestion.

21. Tests sur windows-client1 :

Get-Service winlogbeat -ErrorAction SilentlyContinue
Test-NetConnection 172.16.121.147 -Port 9200



On active winlogbeat avec les commandes suivantes :
Invoke-WebRequest -Uri 'https://artifacts.elastic.co/downloads/beats/winlogbeat/winlogbeat-9.5.4-windows-x86_64.zip' -OutFile "$env:TEMP\winlogbeat-9.5.4-windows-x86_64.zip"
Expand-Archive -Path "$env:TEMP\winlogbeat-9.5.4-windows-x86_64.zip" -DestinationPath 'C:\Program Files'
Rename-Item 'C:\Program Files\winlogbeat-9.5.4-windows-x86_64' 'Winlogbeat'
Test-Path 'C:\Program Files\Winlogbeat\winlogbeat.exe'



winlogbeat est téléchargé.

Avec la commande : Get-Service winlogbeat

On peut voir que le service winlogbeat est installé à l’état « stopped ».

```
PS C:\Windows\system32> Get-Service winlogbeat

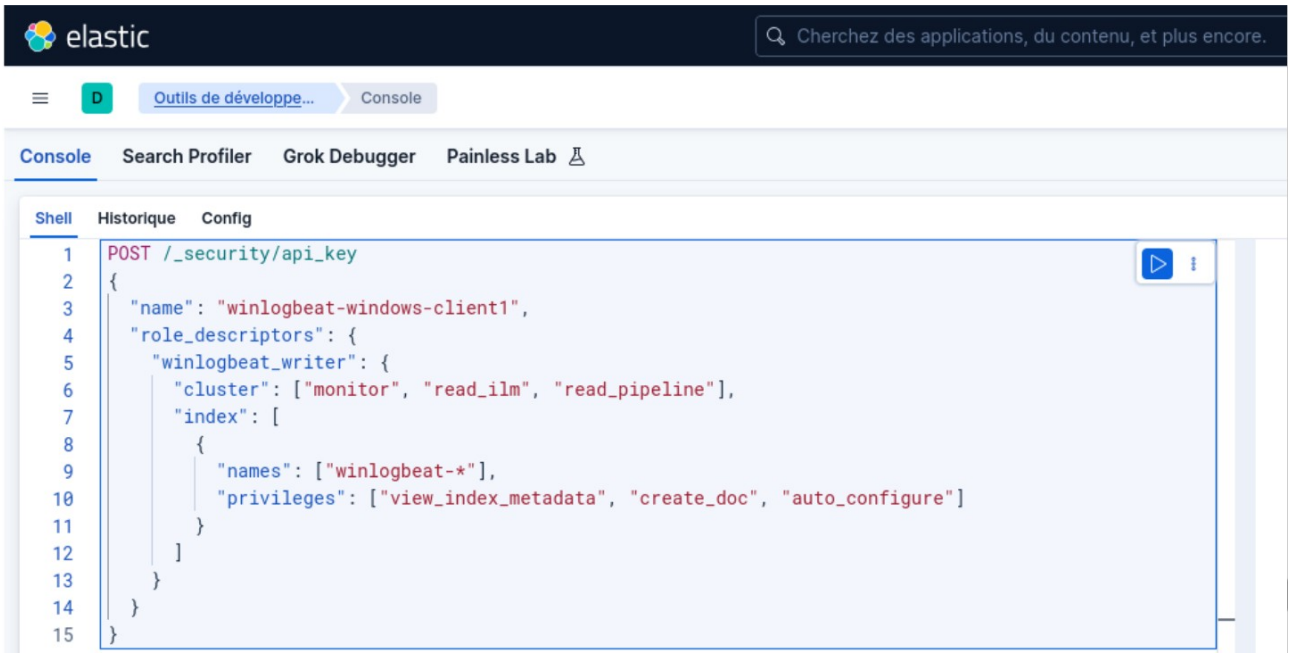
Status   Name               DisplayName
------   ----               -----------
Stopped  winlogbeat         winlogbeat


PS C:\Windows\system32>
```

Pour préparer sa connexion HTTPS, on passe sur la VM elastic-node1 et on lance la commande :
sudo openssl x509 -in /etc/elasticsearch/certs/http_ca.crt -noout -fingerprint -sha256

```
Last login: Fri Sep 25 08:04:39 2026
safia@elastic-node1:~$ sudo openssl x509 -in /etc/elasticsearch/certs/http_ca.cr
t -noout -fingerprint -sha256
[sudo] password for safia:
sha256 Fingerprint=99:B3:CC:8E:95:6F:05:1E:B4:4C:E8:7C:5C:7C:F6:8D:14:19:9F:C8:F
A:81:4C:E9:B1:F3:64:74:CE:70:A7:8C
safia@elastic-node1:~$
```

L'empreinte est récupérée. Nous Créons une clé API limitée à winlogbeat.
Dans Kibana, dans Outils de développement, on exécute une requête pour obtenir la clé api.



Ensuite on enregistre la clé sur windows-client1 :

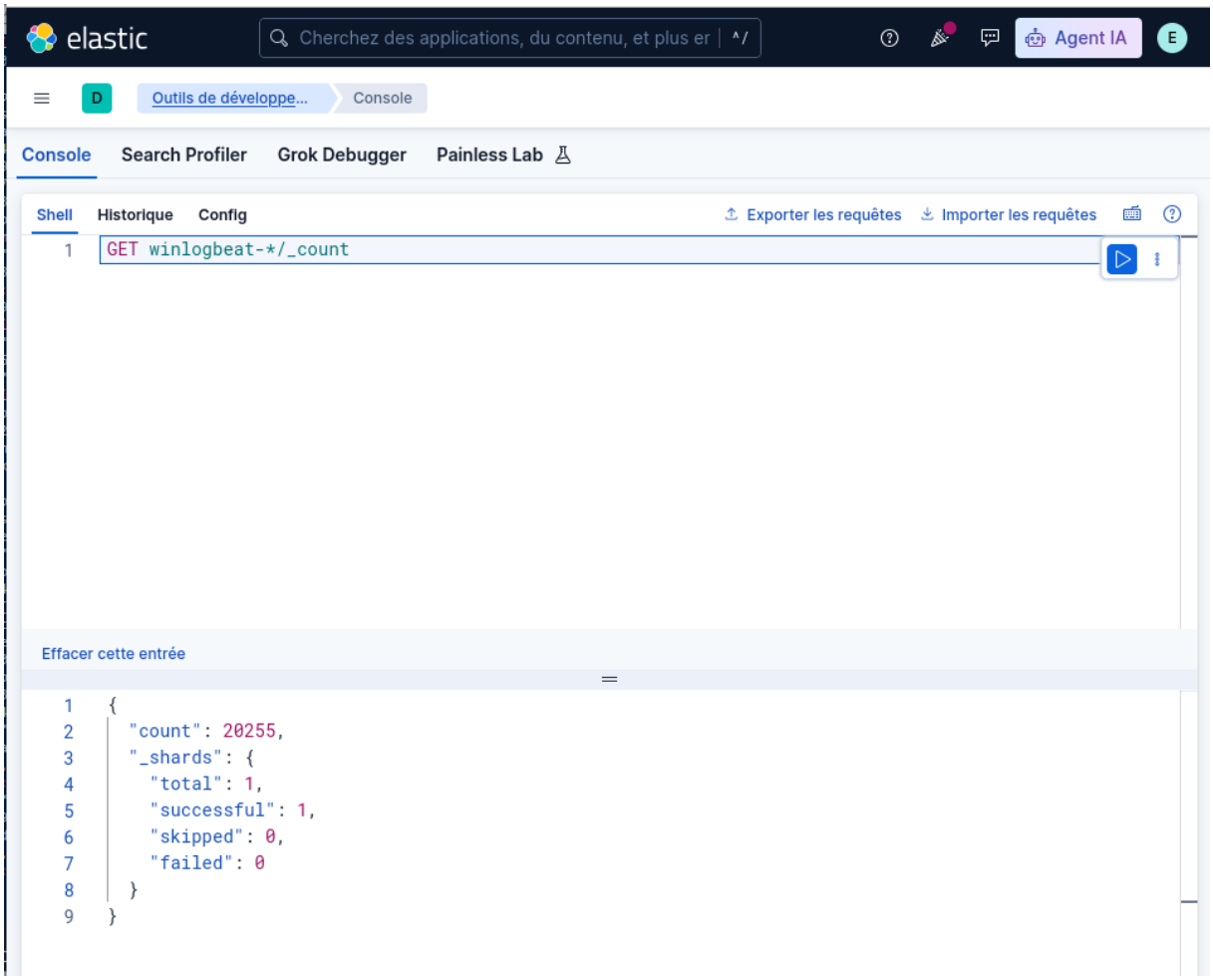
cd 'C:\Program Files\Winlogbeat'
.\winlogbeat.exe keystore create --path.data 'C:\Program Files\Winlogbeat-Data\data'
.\winlogbeat.exe keystore add WINLOGBEAT_API_KEY --path.data 'C:\Program Files\Winlogbeat-Data\data'

Nous effectuons ensuite deux vérifications :

.\winlogbeat.exe test config -e --path.data 'C:\Program Files\Winlogbeat-Data\data'
.\winlogbeat.exe test output -e --path.data 'C:\Program Files\Winlogbeat-Data\data'

Le premier teste la configuration, le deuxième teste la connexion à Élasticsearch.

On enregistre la clé API sur le portail kibana et on lance la requête pour la collecte des logs journaux : GET winlogbeat-*/_count



20 255 journaux Windows sont présents.
Avec l'Outil de développement de kibana, on lance cette requête pour voir les canaux et les événements les plus fréquents :

```
GET winlogbeat-*/_search
{
  "size": 0,
  "aggs": {
    "canaux": {
      "terms": { "field": "winlog.channel.keyword", "size": 10 }
    },
    "codes_evenements": {
      "terms": { "field": "event.code.keyword", "size": 15 }
    }
  }
}
```

```
{
  "took": 15,
  "timed_out": false,
  "_shards": {
    "total": 1,
    "successful": 1,
    "skipped": 0,
    "failed": 0
  },
  "hits": {
    "total": {
      "value": 10000,
      "relation": "gte"
    },
    "max_score": null,
    "hits": []
  },
  "aggregations": {
    "canaux": {
      "doc_count_error_upper_bound": 0,
      "sum_other_doc_count": 0,
      "buckets": [
        {
          "key": "Security",
          "doc_count": 16090
        },
        {
          "key": "System",
          "doc_count": 2118
        },
        {
          "key": "Application",
          "doc_count": 2063
        }
      ]
    },
    "codes_evenements": {
      "doc_count_error_upper_bound": 0,
      "sum_other_doc_count": 2427,
      "buckets": [
        {
          "key": "4907",
          "doc_count": 12018
        },
        {
          "key": "5379",
          "doc_count": 1221
        },
        {
          "key": "1004",
          "doc_count": 878
        },
        {
          "key": "4624",
          "doc_count": 712
        },
        {
          "key": "4672",
          "doc_count": 651
```

```
        {
          "key": "4798",
          "doc_count": 513
        },
        {
          "key": "4799",
          "doc_count": 339
        },
        {
          "key": "16",
          "doc_count": 315
        },
        {
          "key": "44",
          "doc_count": 275
```

```
          "key": "4688",
          "doc_count": 177
        },
        {
          "key": "6",
          "doc_count": 176
        },
        {
          "key": "1003",
          "doc_count": 150
        },
        {
          "key": "16394",
          "doc_count": 148
        },
```

```
          "key": "16384",
          "doc_count": 146
        },
        {
          "key": "10016",
          "doc_count": 125
        }
```


Les résultats sont exploitables : 16 090 événements Security, 2 118 System et 2 063 Application. On voit aussi 712 connexions réussies (4624). La prochaine vérification porte sur les échecs de connexion (4625), une base utile pour une alerte.
On lance donc une nouvelle requête :

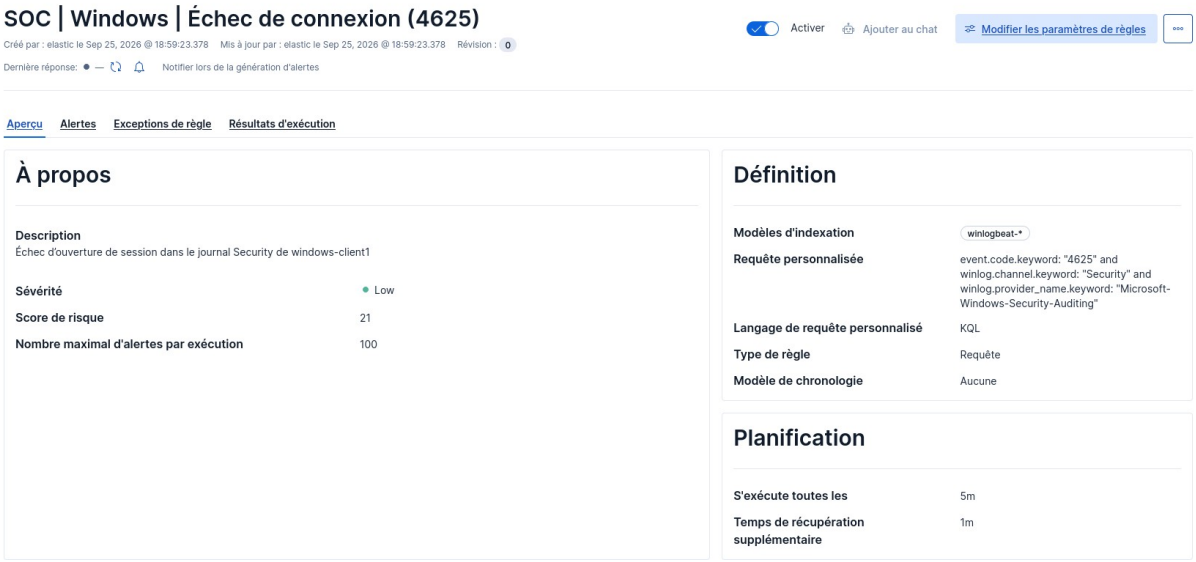
```
GET winlogbeat-*/_search
{
  "size": 3,
  "track_total_hits": true,
  "query": {
    "term": { "event.code.keyword": "4625" }
  },
  "_source": [
    "@timestamp",
    "host.name",
    "event.code",
    "user.name",
    "source.ip",
    "winlog.event_data.TargetUserName",
    "winlog.event_data.Status",
```

```
"took": 16,
"timed_out": false,
"_shards": {
  "total": 1,
  "successful": 1,
  "skipped": 0,
  "failed": 0
},
"hits": {
  "total": {
    "value": 18,
    "relation": "eq"
  },
  "max_score": 6.999324,
```

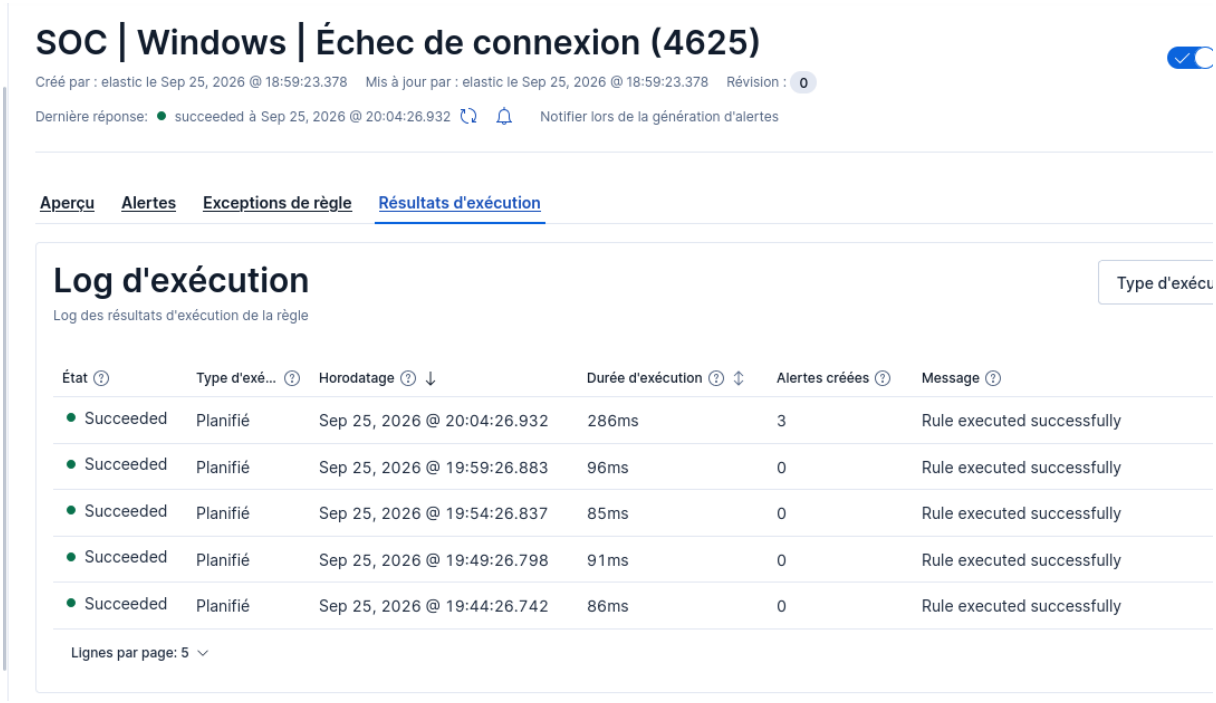
```
      "_index": "winlogbeat-9.5.4",
      "_id": "6ncp2aABjYdY-RATrpAo",
      "_score": 6.999324,
      "_ignored": [
        "message.keyword"
      ],
      "_source": {
        "@timestamp": "2026-09-21T15:40:28.062Z",
        "host": {
          "name": "DESKTOP-EHQ529H"
        },
        "event": {
          "code": "4625"
        },
        "message": """Le sous-système EventSystem va filtrer les doublons d'entrées du journal
```


22. Création de la règle échecs de connexion

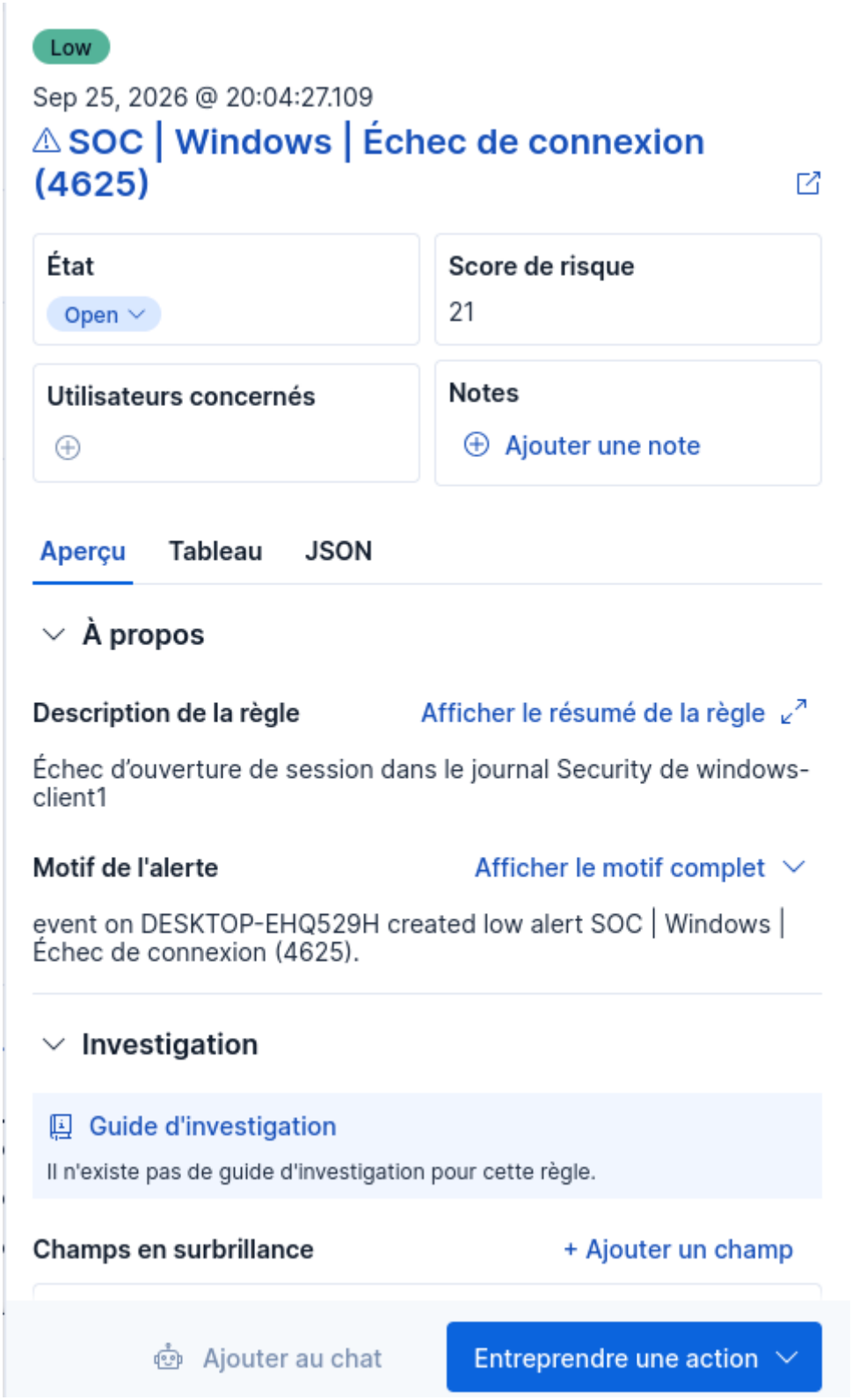
Pour filtrer les éléments que l'on veut observer, les échecs de connexion, on crée une nouvelle règle avec les paramètres suivants :



On fait un test en lançant une exécution avec un faux mot de passe pour voir si l'erreur est enregistré dans les logs d'exécution de la règle sur kibana.



La règle de détection des échecs de connexion Windows (événement 4625) a été validée par un test contrôlé. L’exécution planifiée du 25 septembre 2026 à 20 h 04 a généré trois alertes. Leur qualification nécessite l’examen des événements associés ; une alerte ne suffit pas, à elle seule, à conclure à une attaque.



L'analyse de l'événement 4625 montre une tentative de connexion interactive au compte local soc-test, initiée depuis DESKTOP-EHQ529H avec un mot de passe incorrect. L'adresse source ::1 indique une origine locale. Cet événement correspond au test contrôlé réalisé pour valider la détection : Winlogbeat l'a transmis et la règle Elastic a généré une alerte environ 18 secondes après l'événement. Aucune tentative distante n'est établie par cette alerte.

23. Enquête soc-test :

On lance une requête afin de relever des informations concernant la règle SOC Windows Échec de connexion (4625).

Shell Historique Config

```
GET winlogbeat-*/_search
{
  "size": 30,
  "sort": [{ "@timestamp": "asc" }],
  "_source": [
    "@timestamp",
    "host.name",
    "event.code",
    "winlog.event_data.SubjectUserName",
    "winlog.event_data.TargetUserName",
    "winlog.event_data.IpAddress",
    "winlog.event_data.LogonType",
    "winlog.event_data.Status",
    "winlog.event_data.SubStatus",
    "winlog.event_data.ProcessName"
  ],
  "query": {
    "bool": {
      "filter": [
        { "term": { "winlog.channel.keyword": "Security" } },
        { "terms": { "event.code.keyword": ["4720", "4625", "4624"] } },
        { "match_phrase": { "winlog.event_data.TargetUserName": "soc-test" } },
        {
          "range": {
            "@timestamp": {
              "gte": "2026-09-22T11:30:00Z",
              "lte": "2026-09-22T13:00:00Z"
            }
          }
        }
      ]
    }
  }
```

Effacer cette entrée

22 septembre	Événement	Fait observé
13 h 38 min 42 s	**4720**	Safia crée le compte soc-test sur DESKTOP-EHQ529H.
14 h 41 min 53 s	**4625**	Premier échec de connexion à soc-test.
14 h 46 min 18 s 14 h 47 min 47 s	**4 × 4625**	Quatre autres échecs rapprochés pour le même compte.

Parsing et qualification : sur les cinq échecs, TargetUserName désigne le compte visé (soc-test), tandis que SubjectUserName désigne la session qui a demandé l'opération (Safia). LogonType=2 signifie *interactif*, IpAddress=::1 indique la machine locale et SubStatus=0xc000006a précise un mot de passe incorrect. Le processus indiqué est C:\Windows\System32\svchost.exe.

La recherche ne montre **aucun 4624 pour soc-test dans cette plage de 90 minutes** ; cela ne suffit pas à affirmer qu'aucune connexion n'a réussi ensuite. Nous n'avons pas non plus d'indice d'une origine distante. Le rapprochement des événements est plus instructif que chacune des alertes prise seule.
Résultat de l'enquête 1 : aucun événement **4624** de connexion réussie à soc-test n'apparaît dans les données Winlogbeat collectées depuis sa création. Nous avons donc une création de compte suivie de **cinq échecs locaux**, sans succès observé. Comme nous avons réalisé ces essais, la qualification est : **test contrôlé détecté correctement**.

24. Enquête SSH :

Nous cherchons si des échecs précèdent un accès réussi et si les tentatives viennent d'une même adresse. Sur kibana, nous lançons la requête pour extraire les informations IP → compte → échecs → éventuel succès → délai.

```
GET filebeat-*/_search
{
  "size": 20,
  "track_total_hits": true,
  "sort": [{ "@timestamp": "desc" }],
  "_source": [
    "@timestamp",
    "host.name",
    "source.ip",
    "user.name",
    "event.action",
    "event.outcome",
    "system.auth.ssh.event",
    "system.auth.ssh.method",
    "message"
  ],
  "query": {
    "bool": {
      "filter": [
        { "term": { "event.dataset": "system.auth" } },
        { "exists": { "field": "system.auth.ssh.event" } }
      ]
    }
  }
}
```

Période, heure de Paris	Logs SSH observés sur elastic-node1
23–25 septembre	**13 connexions acceptées** pour safia, depuis 172.16.121.1.
24 septembre, 22 h 32 min 09 s	**1 échec explicite** : Failed, event.outcome=failure, même compte et même IP.
24 septembre, 22 h 32 min 29 s	**1 ligne de répétition** : repeated 2 times: [Failed password…, classée system.auth.ssh.event=message, sans event.outcome.
25 septembre, 11 h 25 min 04 s	Nouvelle connexion **acceptée**, près de 13 heures après les échecs.

Les échecs concernent le même compte et la même IP que les connexions habituelles, et concordent avec le test SSH. Les logs fournis ne montrent **aucun succès immédiatement après les échecs**. Ils ne démontrent donc pas une prise de contrôle.

La ligne « repeated 2 times » signale que des messages ont été regroupés par la journalisation. Filebeat l'a rangée dans event=message et a laissé event.outcome vide. Une règle qui compte seulement event.outcome=failure **sous-comptera les échecs**. La documentation de rsyslog confirme que sa réduction des messages répétés remplace plusieurs lignes par un résumé.

Conclusion de l'enquête SSH : un échec est correctement normalisé à 22 h 32 min 09 s ; un second document signale des échecs répétés vingt secondes plus tard. Les 13 connexions acceptées visibles dans cette recherche concernent le même compte safia et la même IP 172.16.121.1, mais **aucune ne suit immédiatement ces échecs**. Dans le contexte de **test contrôlé**, sans preuve de compromission. Nous avons aussi trouvé une limite concrète de la détection : une règle filtrant uniquement event.outcome=failure ignorerait la ligne de répétition.

Enquête 3 — Changement d'un fichier surveillé

Wazuh a détecté la création, la modification puis la suppression d'un fichier dans C:\FIM-Test. Les alertes montrent que la surveillance fonctionne et que le fichier a bien changé. Nous avons ensuite replacé ces événements dans leur contexte : nous avions réalisé ces opérations pour tester l'outil, dans un dossier créé pour l'exercice. Nous classons donc ces alertes comme une activité de test attendue et nous n'ouvrons pas d'incident.

Un changement de fichier ne suffit pas, à lui seul, à conclure à une attaque. Si le fichier concerné était sensible, comme le fichier Windows hosts, il faudrait examiner précisément la modification, son auteur, sa date et son éventuelle justification avant de décider s'il s'agit d'un incident. Aucun changement suspect de ce type n'a été établi dans cette enquête.

Enquête 4 — Faire la différence entre une alerte et le fonctionnement normal de Windows

Windows enregistre un événement 4907 lorsque les paramètres d'audit d'un objet changent. Ces paramètres définissent quelles actions sur cet objet seront consignées. L'événement ne prouve pas que le contenu du fichier a changé. Il y a 12 018 occurrences sont une question d'enquête : quels objets et processus expliquent ce volume, et y a-t-il une exception sensible ?

Requête Kibana	Ce qu'elle permet de vérifier	Résultat à montrer
Comptage et regroupement par programme, compte et jour	Qui a produit les 4907, et à quelles dates ?	12 018 événements, principalement poqexec.exe et TiWorker.exe
Recherche des comptes autres que DESKTOP-EHQ529H$	Les huit événements restants racontent-ils une autre histoire ?	Deux événements OOBE et six événements TiWorker.exe

On lance une requête sur kibana pour voir 5 évènements. Ces cinq événements donnent une bonne piste pour l'enquête sur le **bruit du journal Security**. Ils surviennent tous le 22 septembre vers **13 h 33 min 52 s (heure de Paris)**, sous le compte système, avec le même processus C:\Windows\System32\poqexec.exe, sur des fichiers Windows. Le code **4907 évoque une opération de maintenance Windows.**

```
GET winlogbeat-*/_search
{
  "size": 5,
  "sort": [{ "@timestamp": "desc" }],
  "_source": [
    "@timestamp",
    "host.name",
    "winlog.event_data.SubjectUserName",
    "winlog.event_data.ObjectName",
    "winlog.event_data.ProcessName",
    "winlog.event_data.OldSd",
    "winlog.event_data.NewSd",
    "message"
  ],
  "query": {
    "bool": {
      "filter": [
        { "term": { "event.code.keyword": "4907" } },
        { "term": { "winlog.channel.keyword": "Security" } }
      ]
    }
  }
}GET winlogbeat-*/_search
```

Méthode d'analyse

Nous avons d'abord utilisé une requête Kibana pour compter les événements 4907, puis les regrouper par programme, par compte et par jour. Cela a permis de constater que la plupart étaient produits par les mêmes programmes Windows à des dates rapprochées. Une deuxième requête a isolé les huit événements associés à d'autres noms de compte machine. Leur examen montre qu'ils

s’inscrivent également dans des opérations de configuration ou de maintenance du système. Ce sont ces vérifications, et non le nombre d’événements à lui seul, qui motivent ce classement en bruit de fonctionnement probable.